

# DATA MINING 1

## Introduction Regression

---

Dino Pedreschi, Riccardo Guidotti

a.a. 2024/2025

Contains edited slides from StatQuest



UNIVERSITÀ DI PISA

# Linear Regression

---

# Regression

---

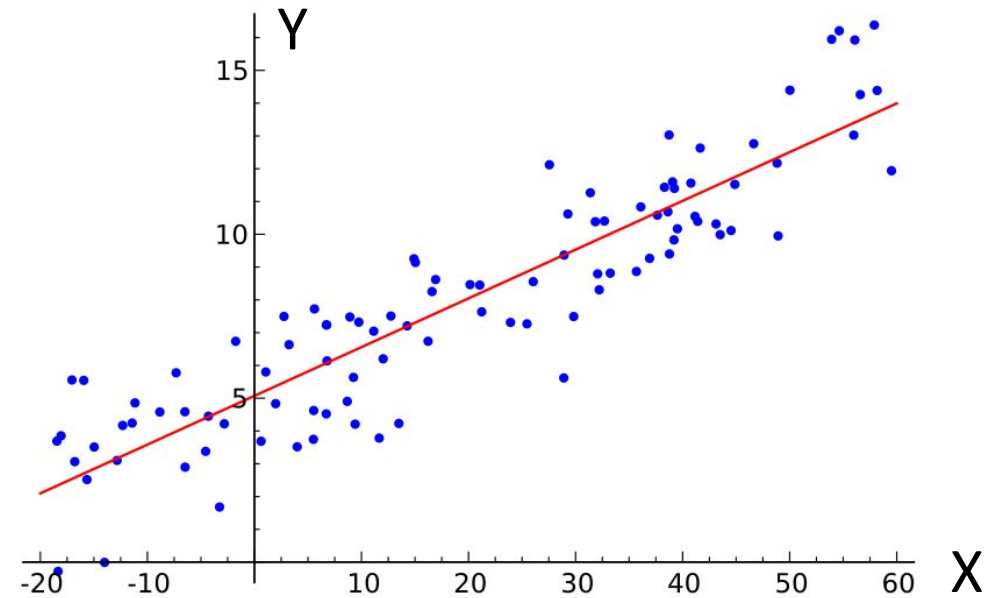
- Given a dataset containing  $N$  observations  $X_i, Y_i, i = 1, 2, \dots, N$
- **Regression** is the task of learning a target function  $f$  that maps each input attribute set  $X$  into an output  $Y$ .
- The goal is to find the target function that can fit the input data with minimum error.
- The error function can be expressed as
  - Absolute Error =  $\sum_i |y_i - f(x_i)|$
  - Squared Error =  $\sum_i (y_i - f(x_i))^2$



residuals

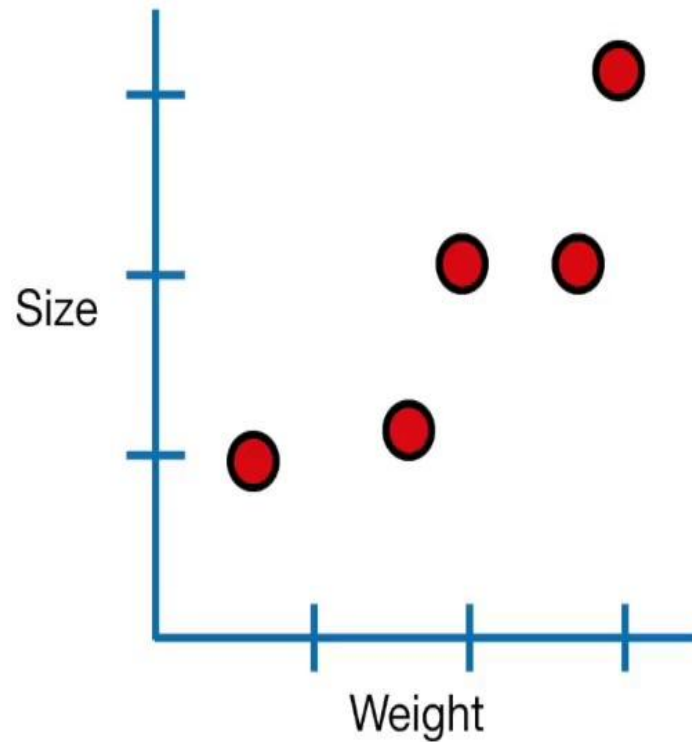
# Linear Regression

- **Linear regression** is a linear approach to modeling the relationship between a *dependent variable*  $Y$  and one or more *independent* (explanatory) variables  $X$ .
- The case of *one* explanatory variable is called **simple linear regression**.
- For *more than one* explanatory variable, the process is called **multiple linear regression**.
- For *multiple correlated dependent variables*, the process is called **multivariate linear regression**.

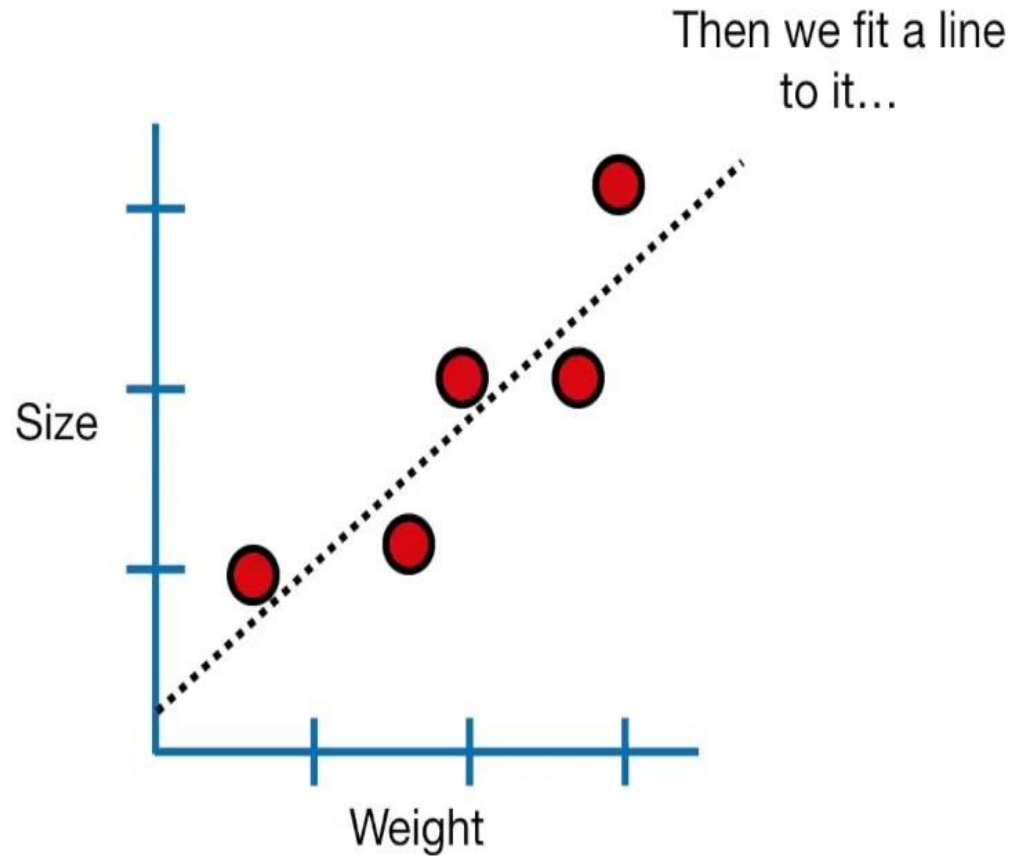


# What does it mean to predict Y?

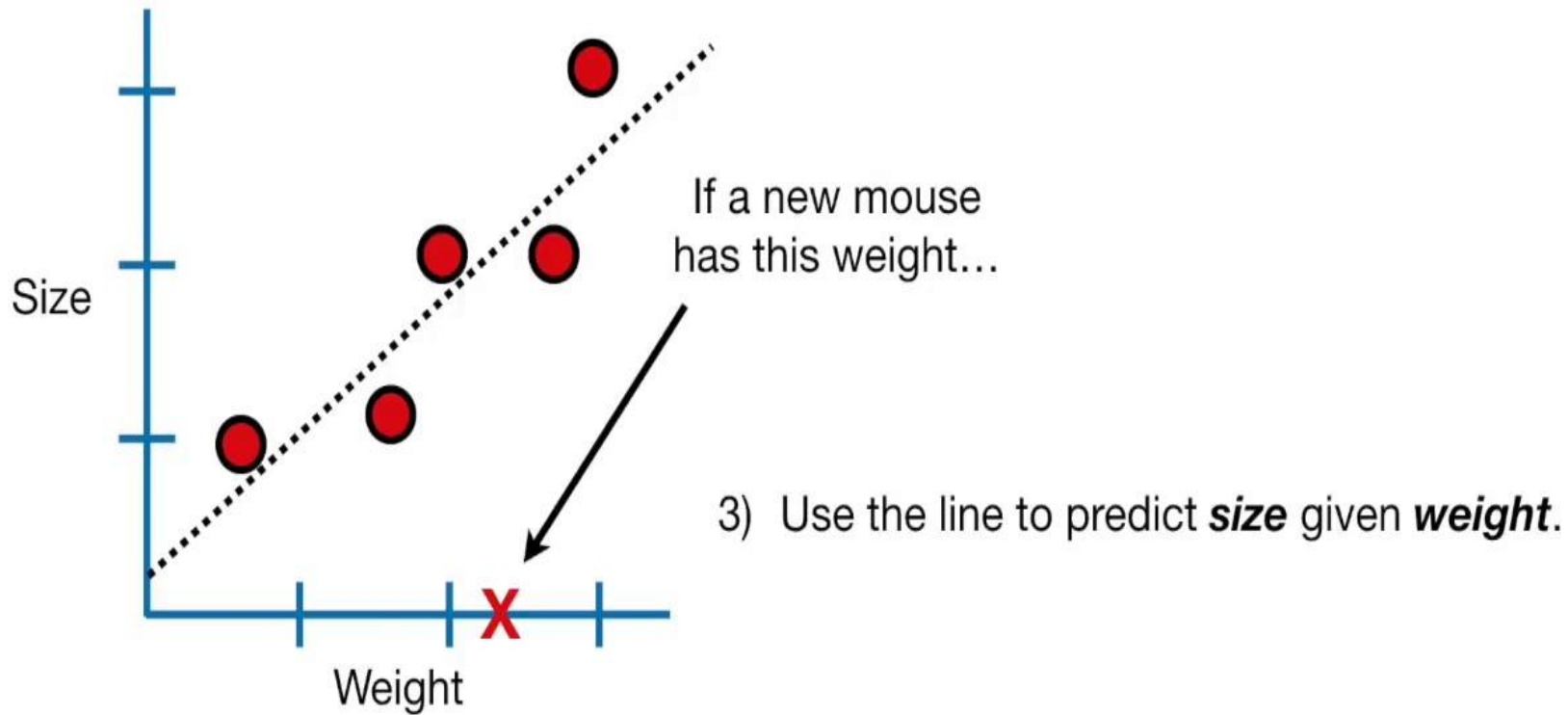
We had some data...



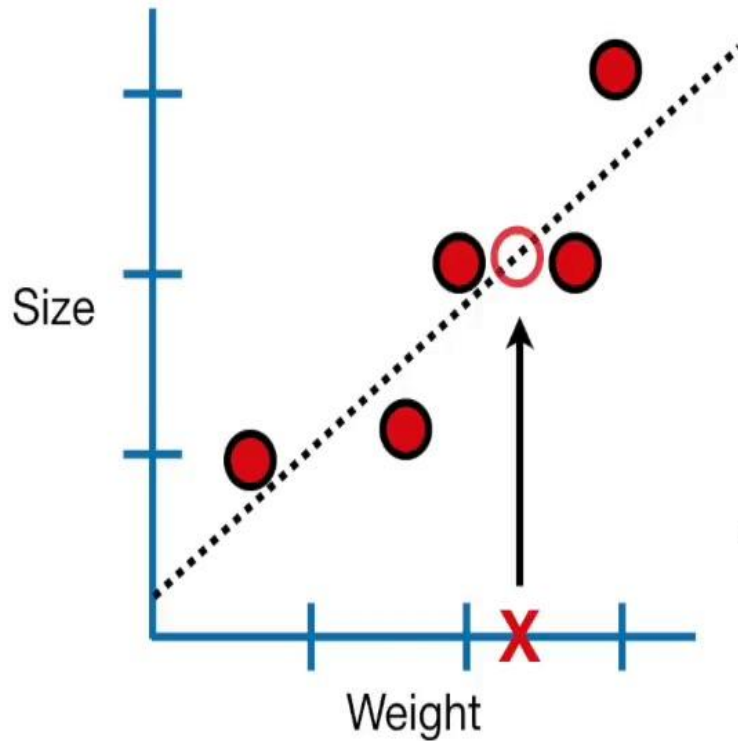
# What does it mean to predict Y?



# What does it mean to predict Y?



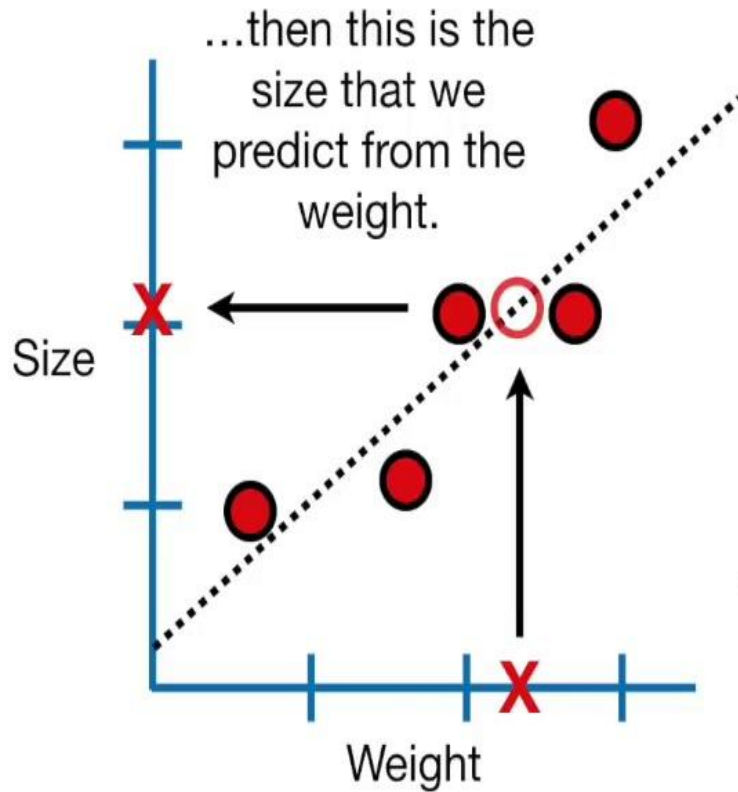
# What does it mean to predict Y?



3) Use the line to predict **size** given **weight**.



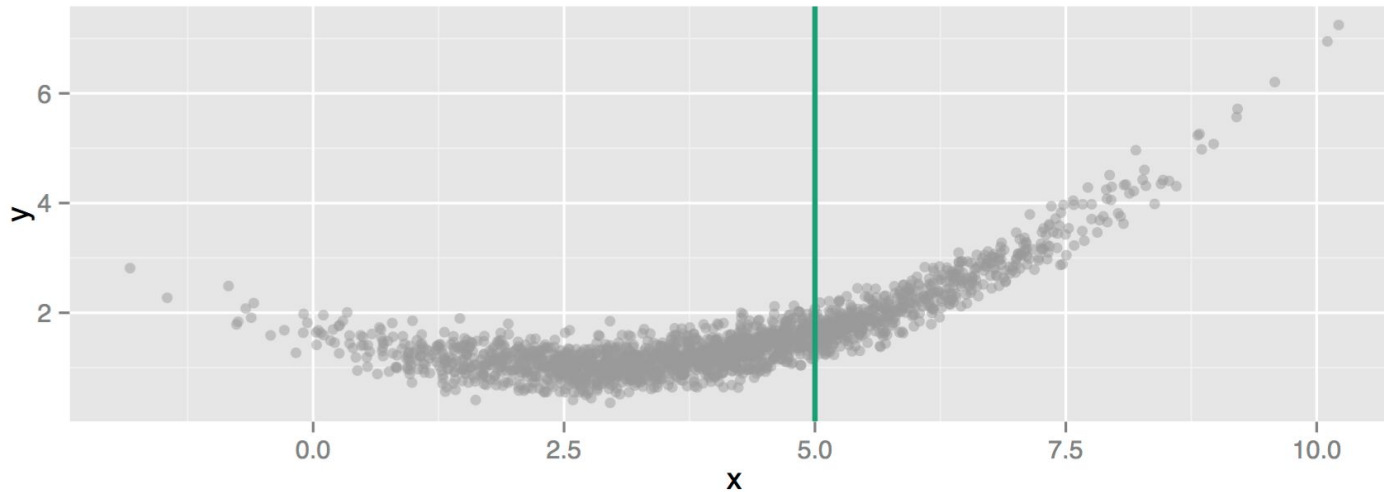
# What does it mean to predict Y?



3) Use the line to predict **size** given **weight**.

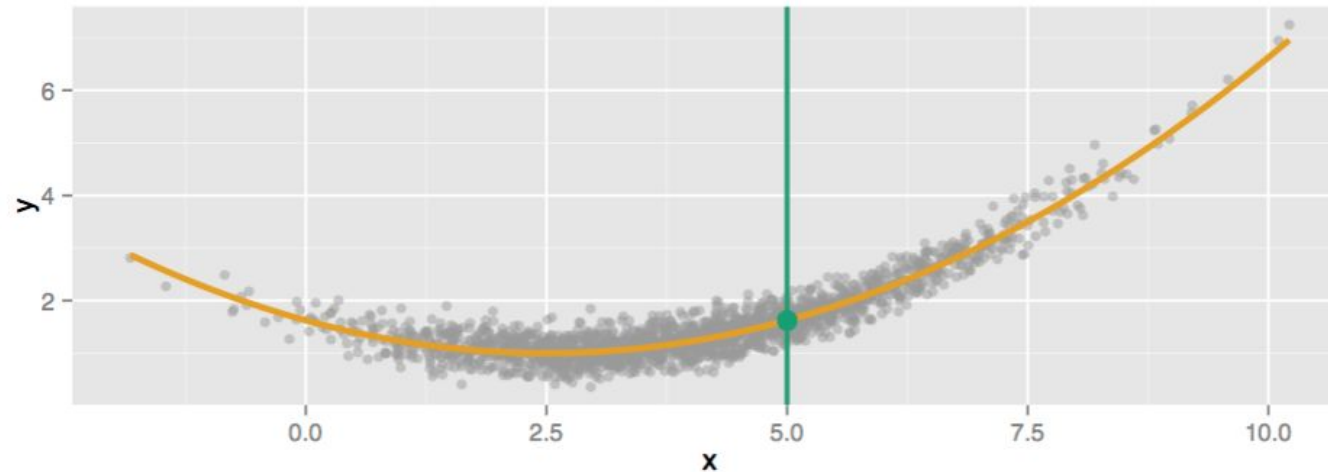
# What does it mean to predict Y?

- Look at  $X = 5$ . There are many different  $Y$  values at  $X=5$ .
- When we say predict  $Y$  at  $X = 5$ , we are really asking:
- What is the expected value (average) of  $Y$  at  $X = 5$ ?

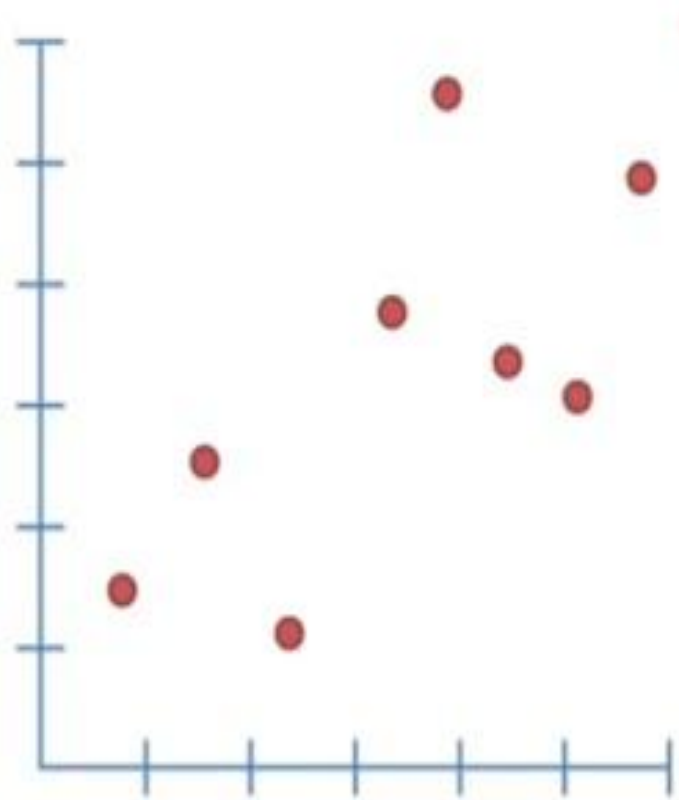


# What does it mean to predict Y?

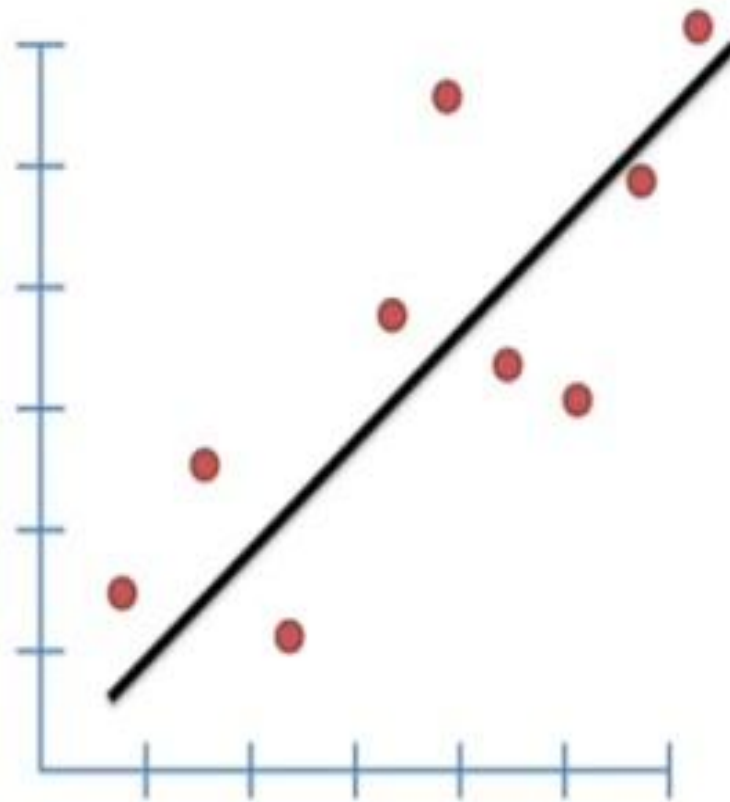
- Formally, the **regression function** is given by  $E(Y/X=x)$ . This is the expected value of  $Y$  at  $X=x$ .
- The ideal or optimal predictor of  $Y$  based on  $X$  is thus
  - $f(X) = E(Y \mid X=x)$



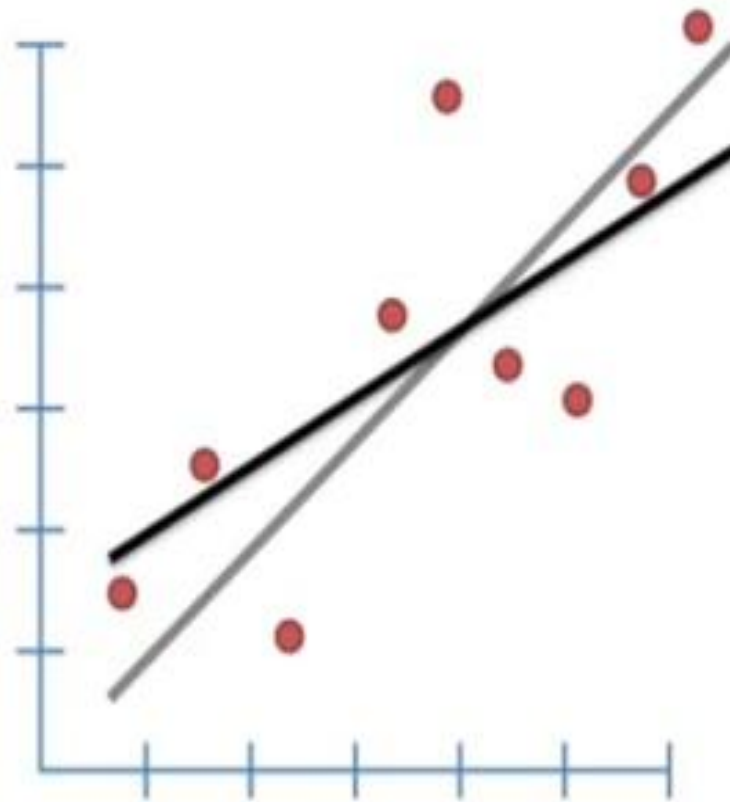
# Fitting a Line Example



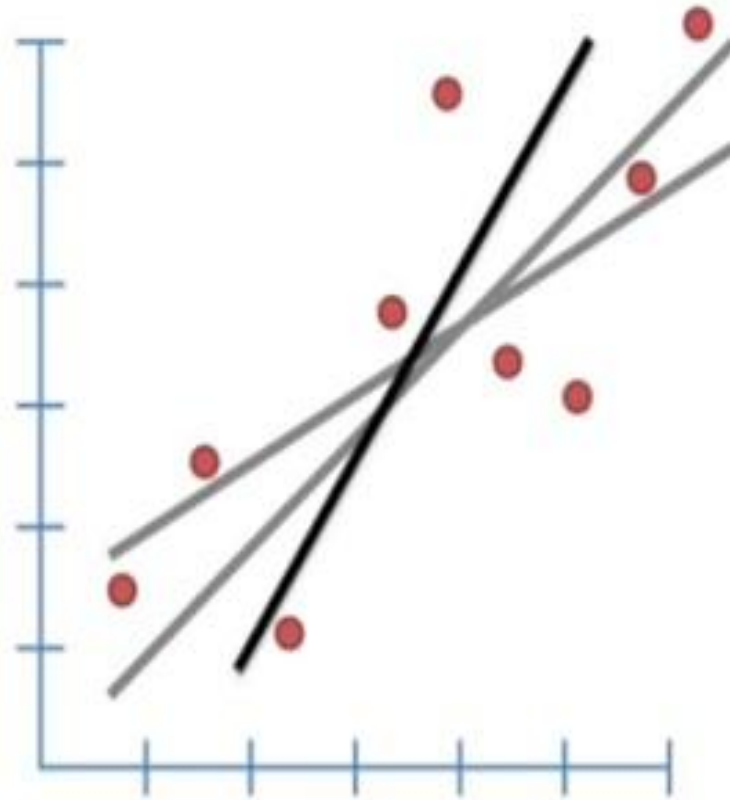
# Fitting a Line Example



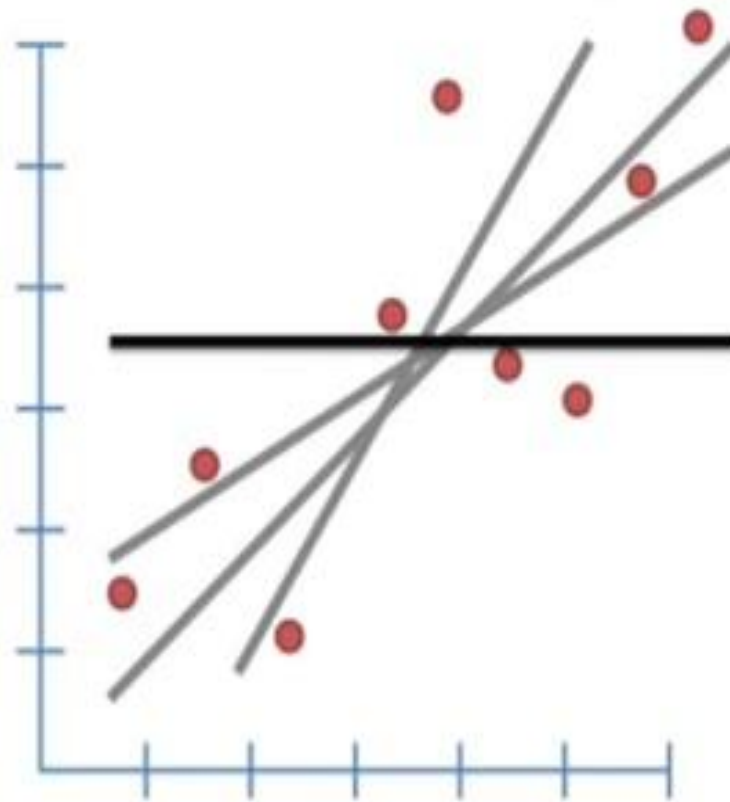
# Fitting a Line Example



# Fitting a Line Example

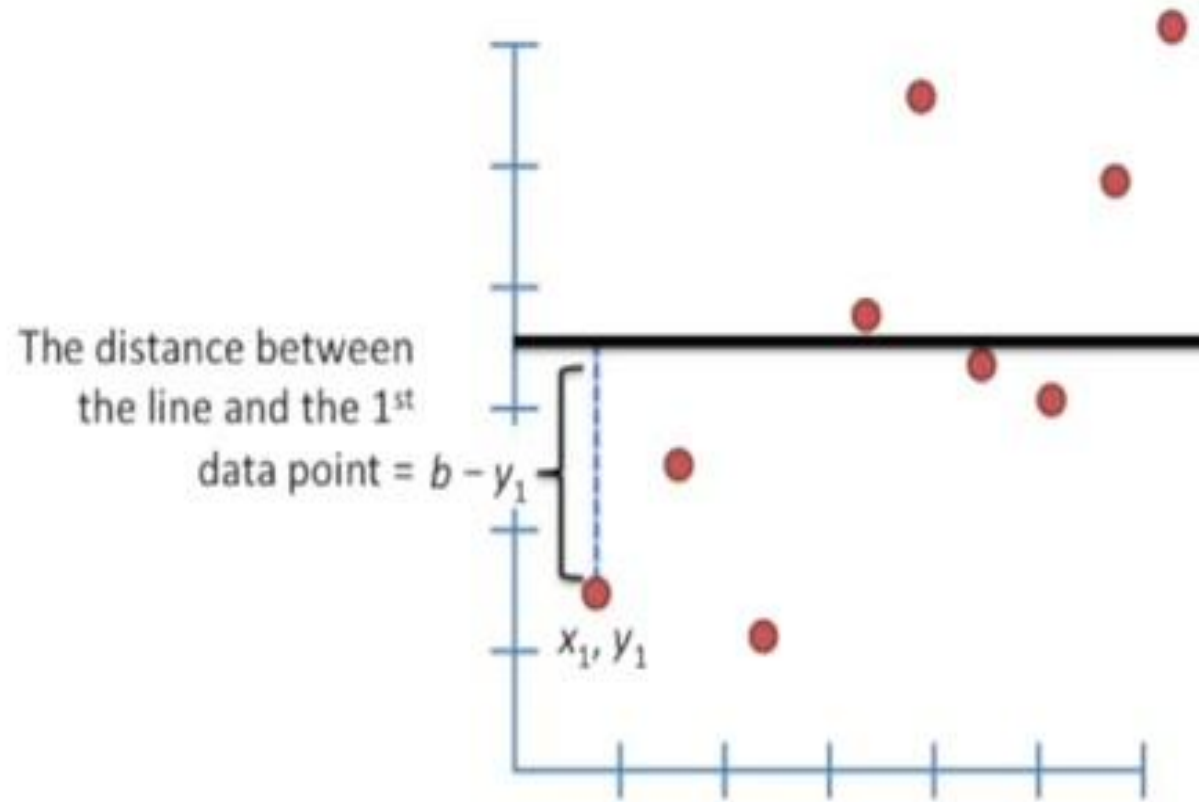


# Fitting a Line Example

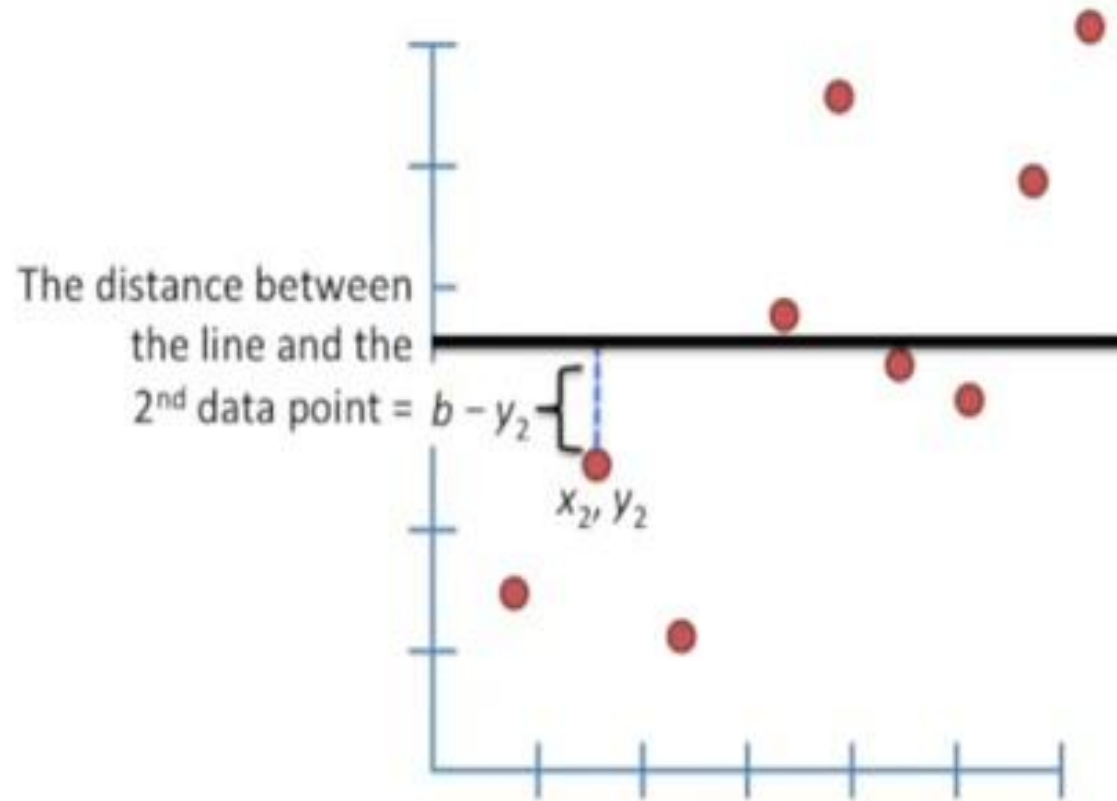




# Measuring Line Error



# Measuring Line Error



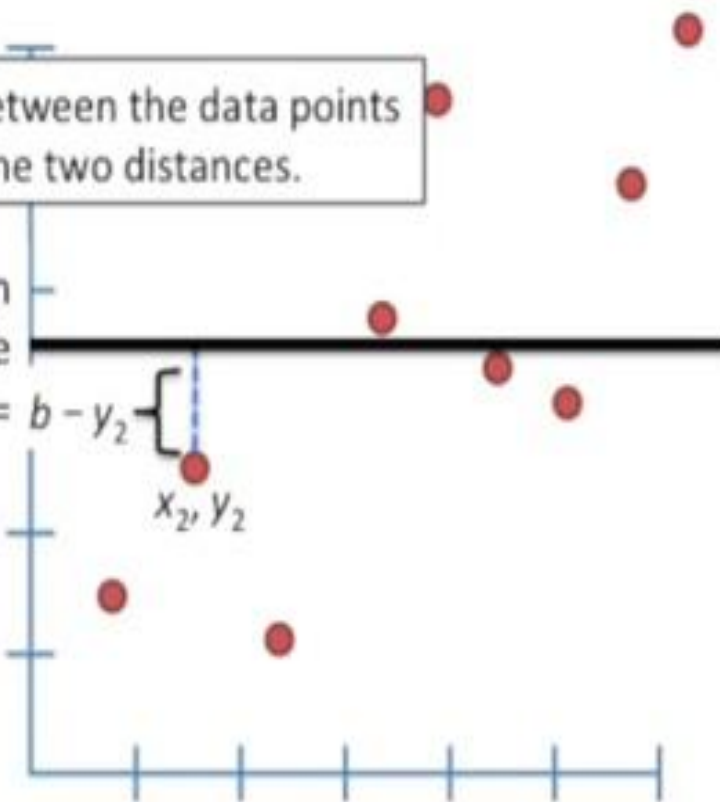
$$(b - y_1) + (b - y_2)$$

So far, the total distance between the data points and the line is the sum of the two distances.

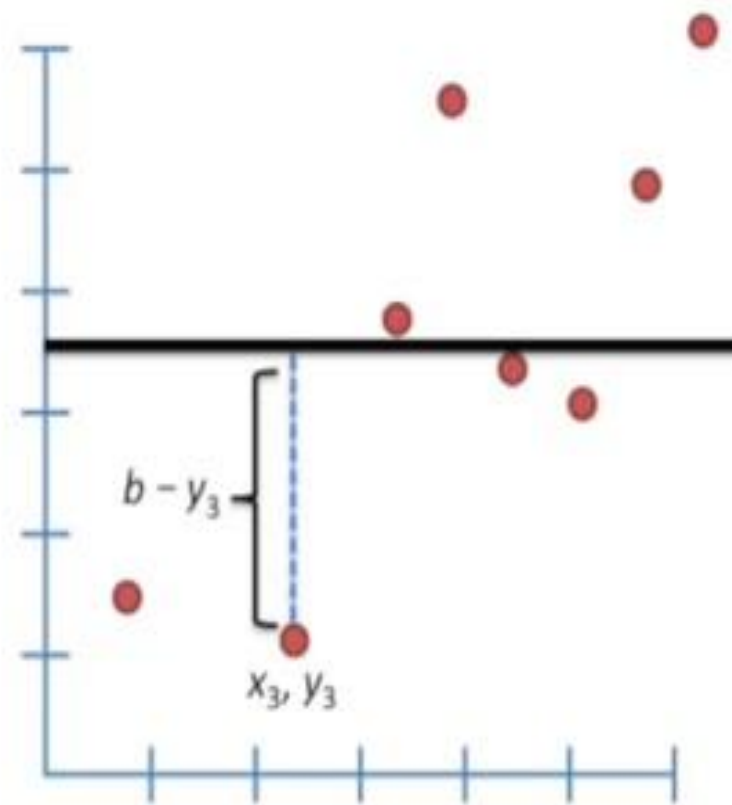
The distance between  
the line and the  
2<sup>nd</sup> data point =

$$b - y_2$$

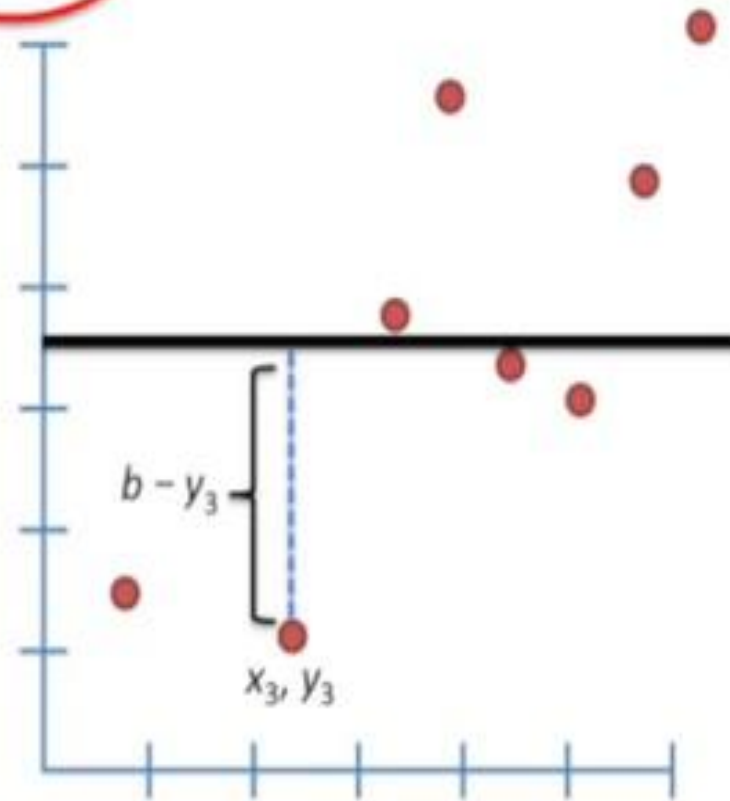
$x_2, y_2$



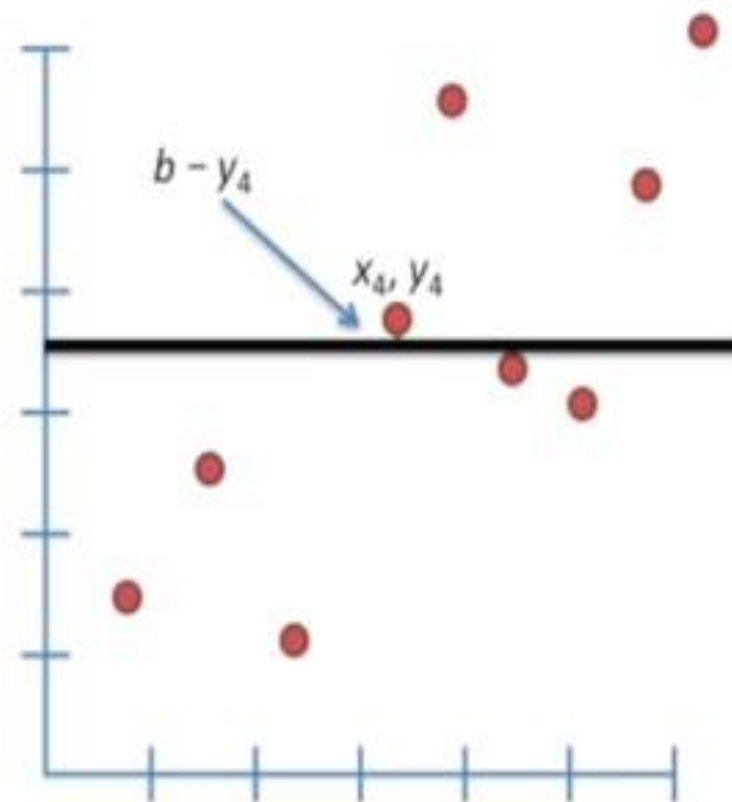
$$(b - y_1) + (b - y_2)$$



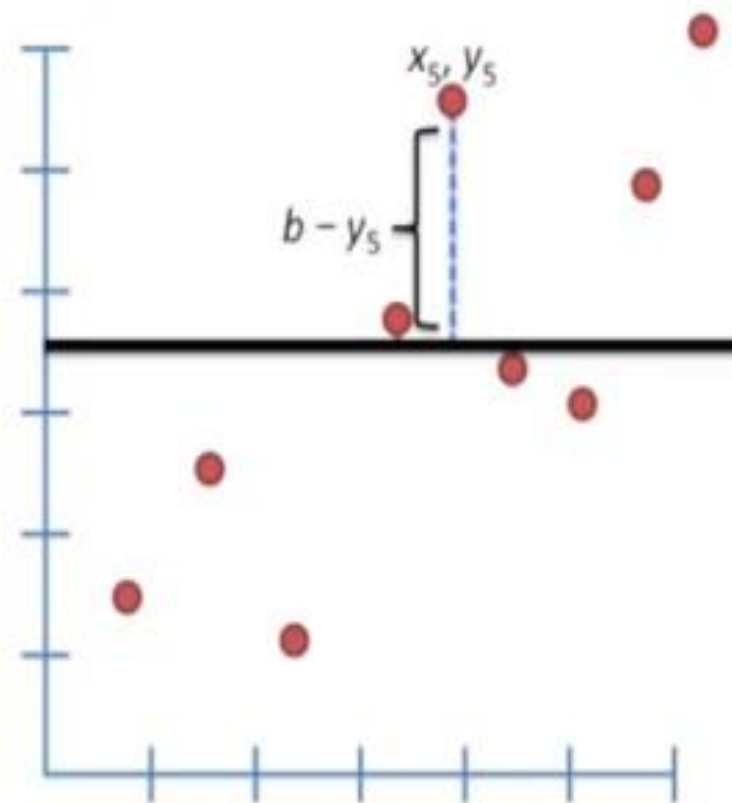
$(b - y_1) + (b - y_2) + (b - y_3)$  Now we've add the 3<sup>rd</sup> distance to our total sum.



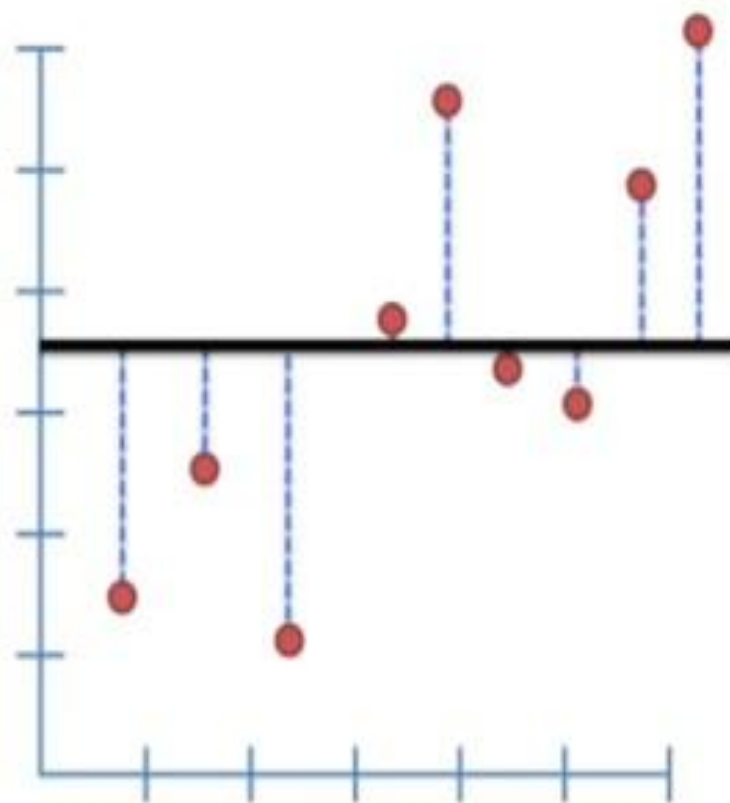
$$(b - y_1) + (b - y_2) + (b - y_3)$$



$$(b - y_1) + (b - y_2) + (b - y_3) + (b - y_4)$$

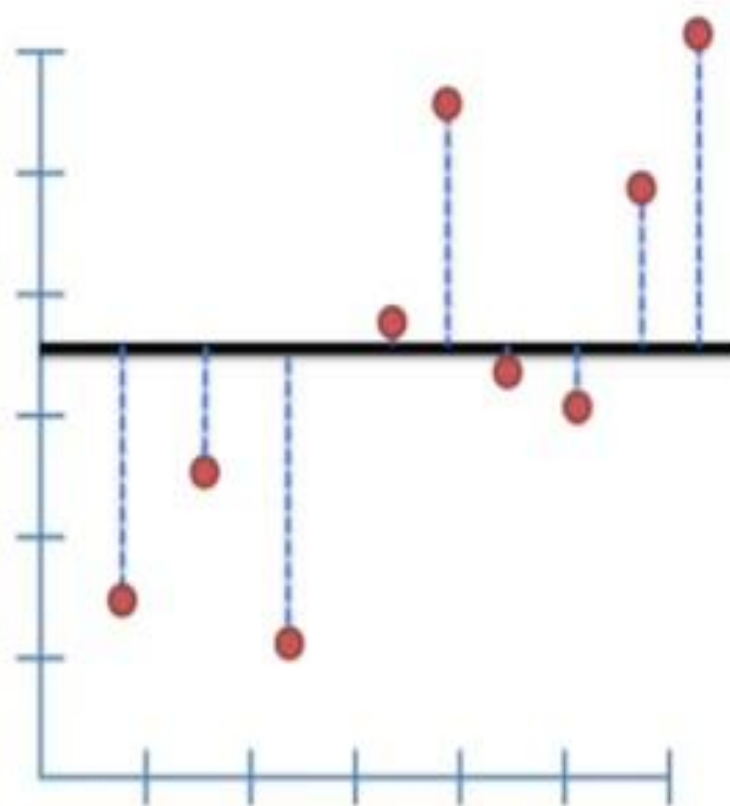


$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$





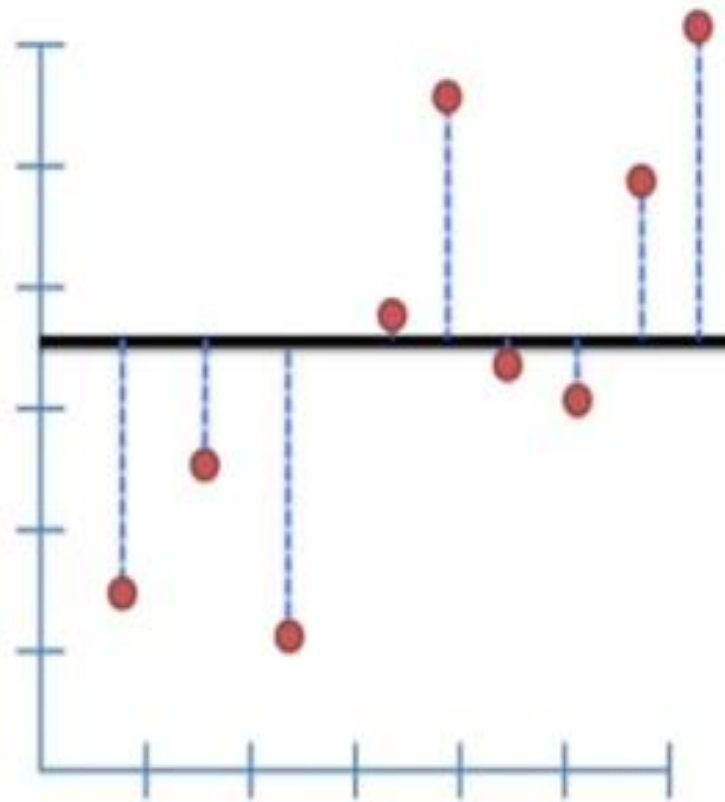
$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$



= 24.62

This is our measure of how well this line fits the data.

$$(b - y_1)^2 + (b - y_2)^2 + (b - y_3)^2 + (b - y_4)^2 + (b - y_5)^2 + (b - y_6)^2 + (b - y_7)^2 + (b - y_8)^2 + (b - y_9)^2$$

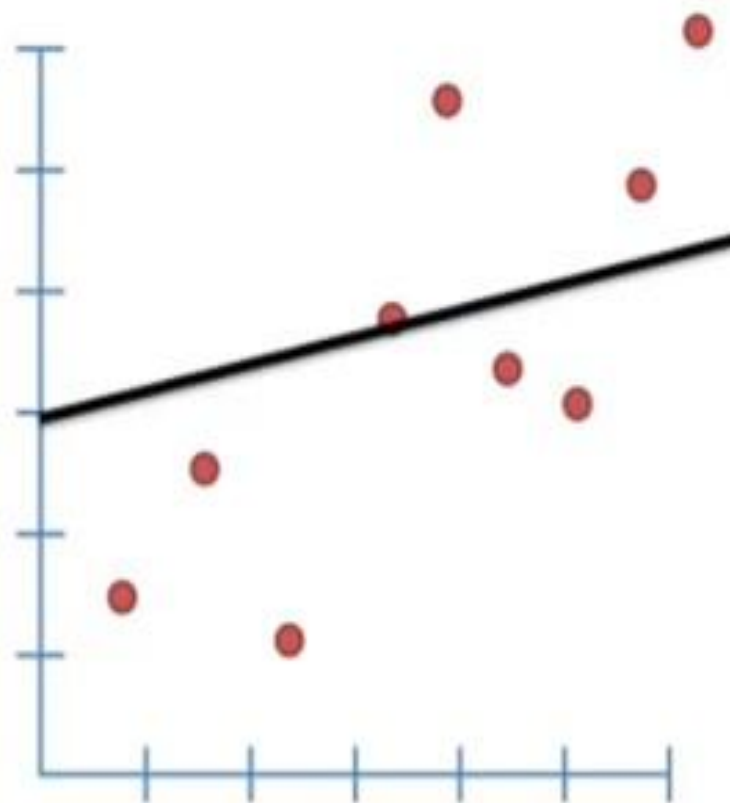


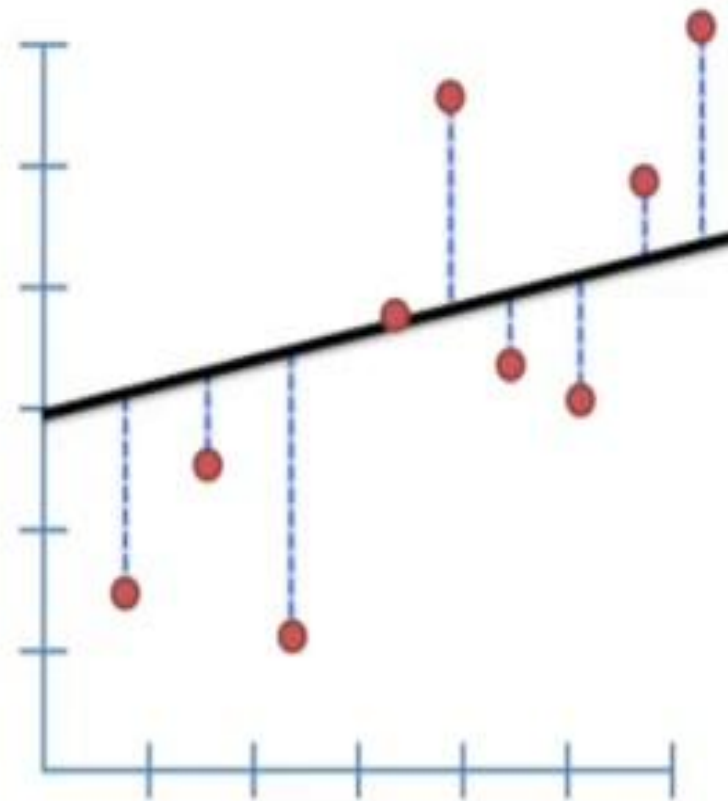
= 24.62

This is our measure of how well this line fits the data.

It's called the "sum of squared residuals," because the residuals are the differences between the real data and the line, and we are summing the square of these values.

Now let's see how good the fit is if we rotate the line a little bit.

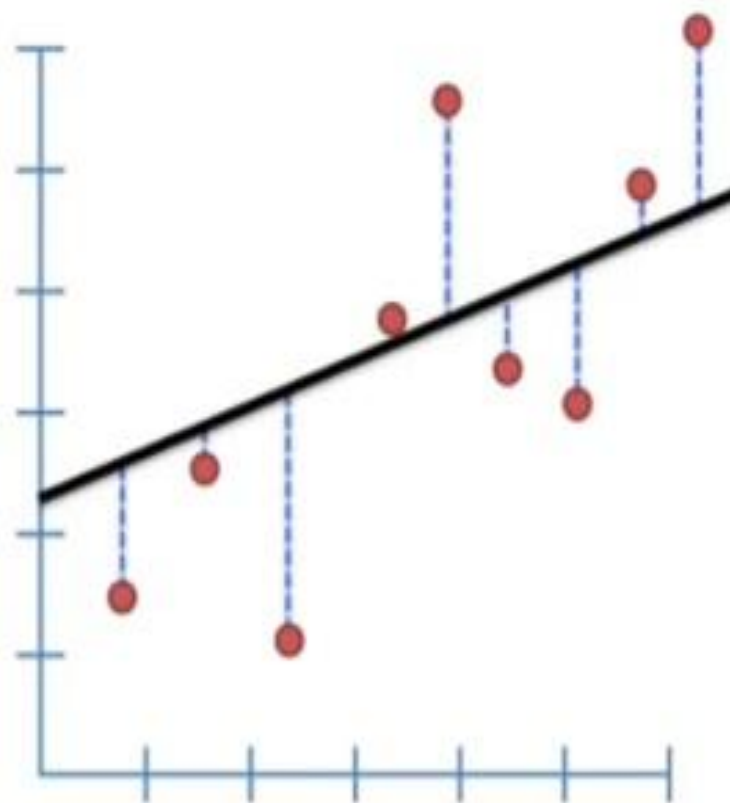




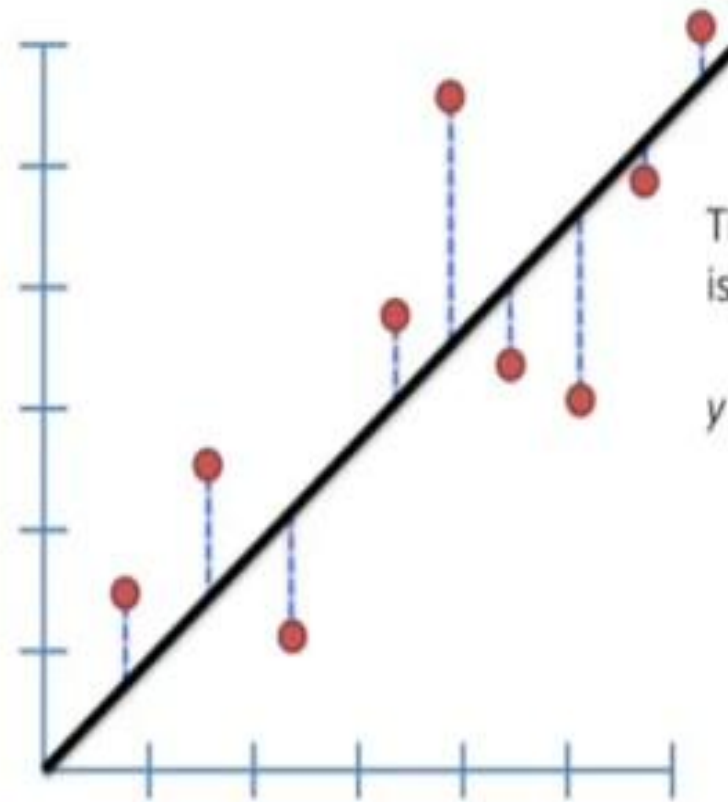
The sum of squared residuals = 18.72

This is better than before.

Does this fit improve if we rotate a little more?



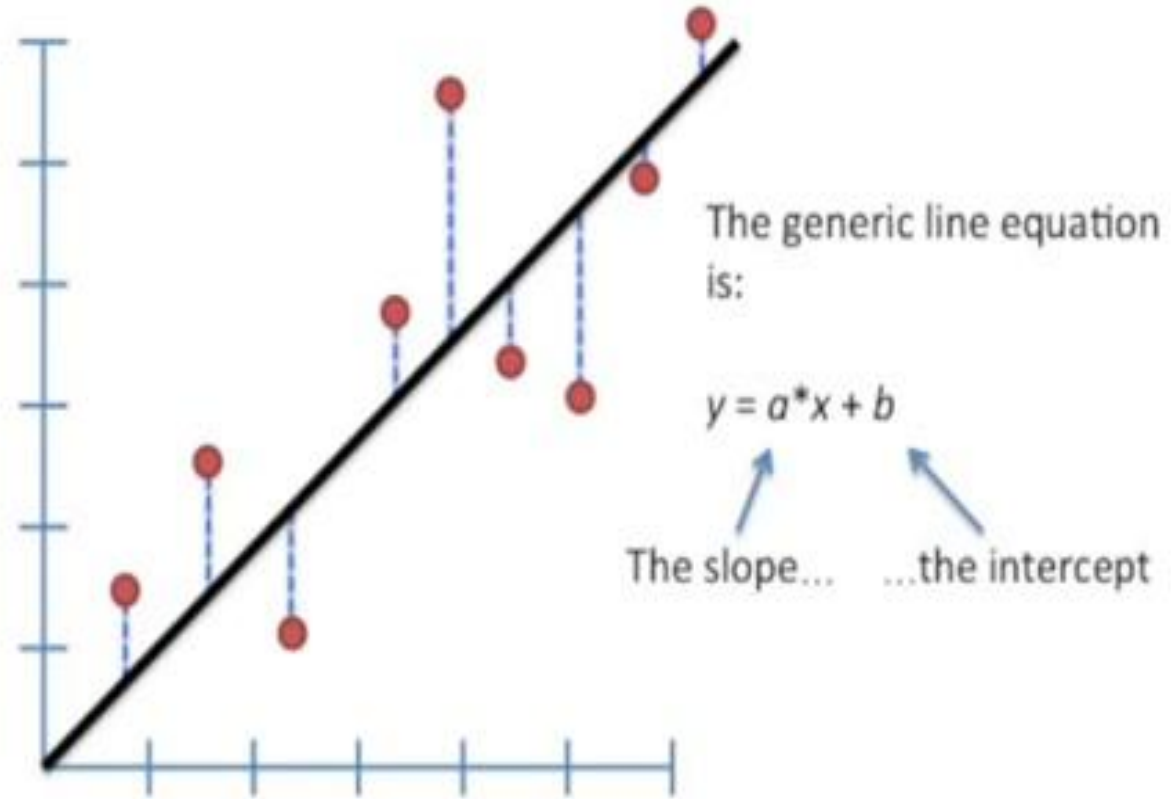
# Generic Line Equation



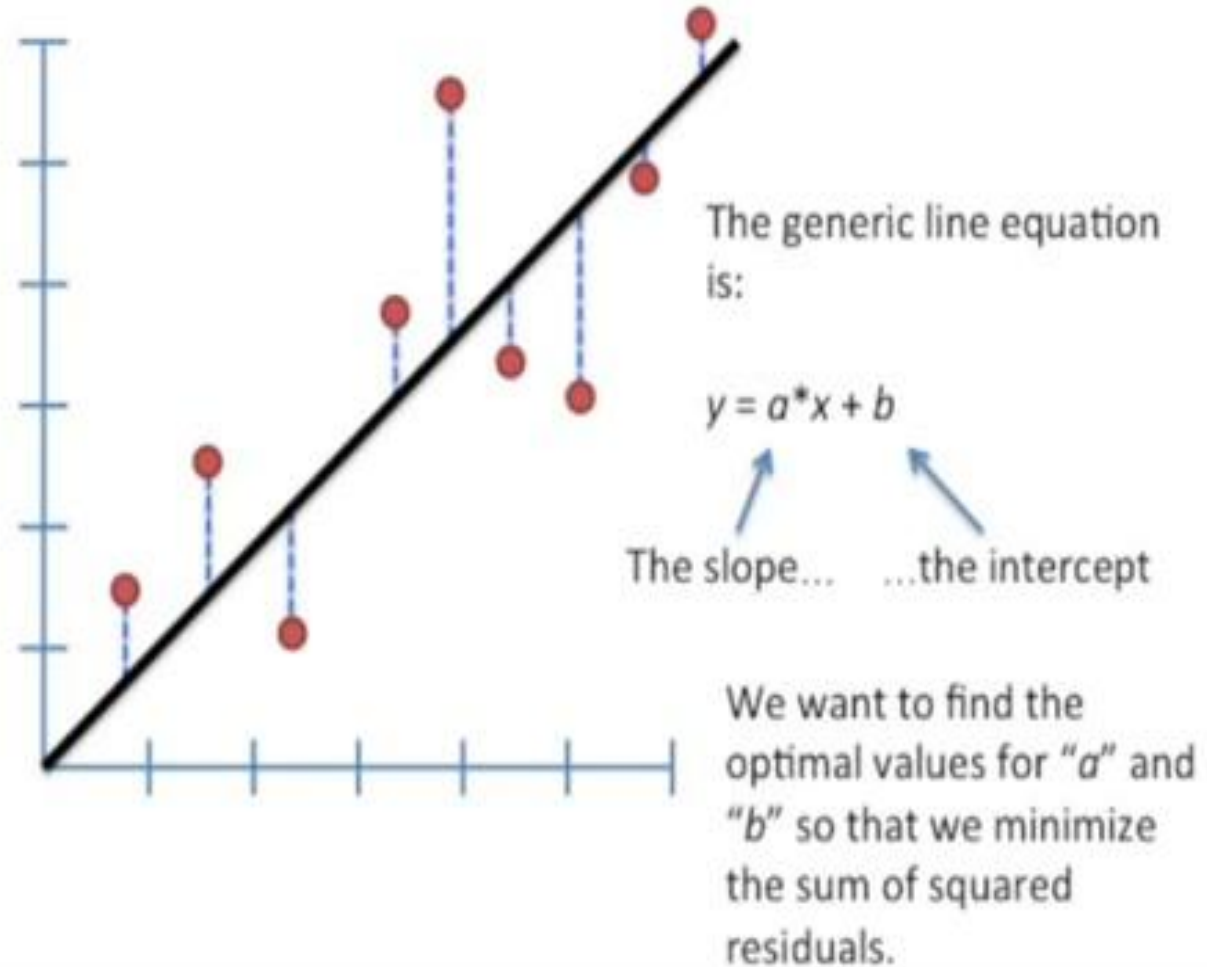
The generic line equation  
is:

$$y = a \cdot x + b$$

# Generic Line Equation



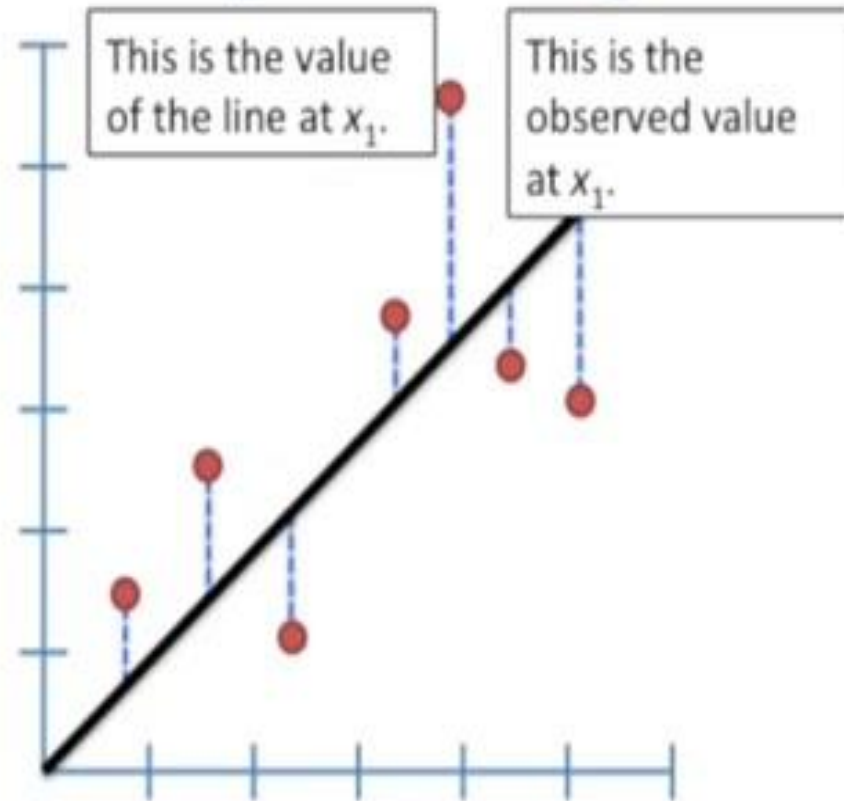
# Generic Line Equation





# Least Square

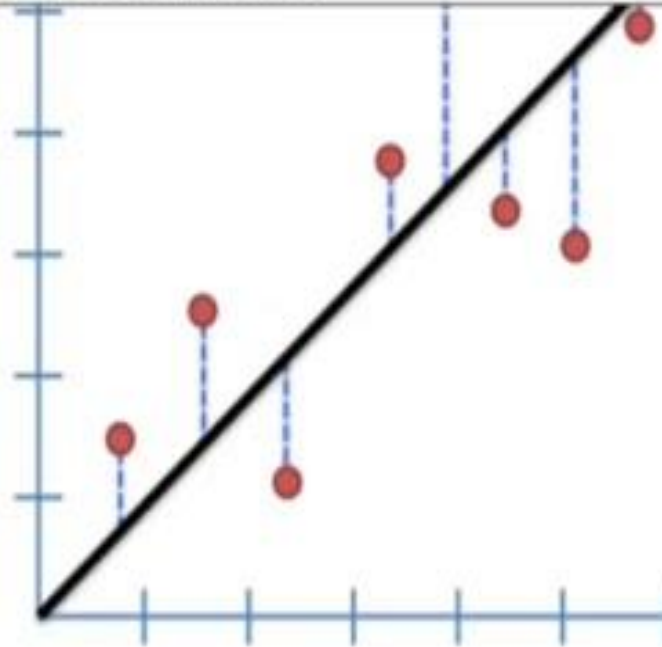
$$\text{Sum of squared residuals} = ((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots$$



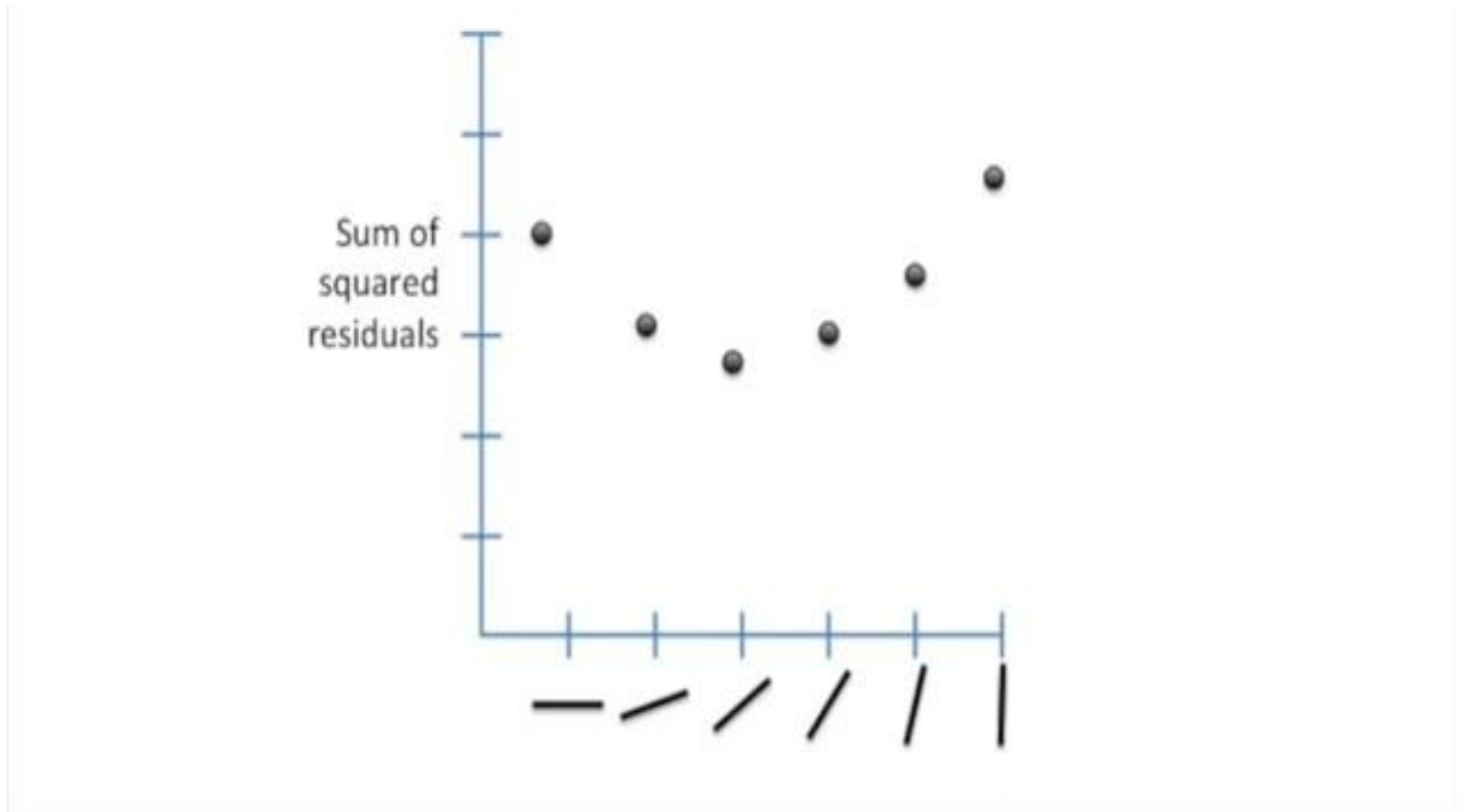
# Least Square

$$\text{Sum of squared residuals} = ((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots$$

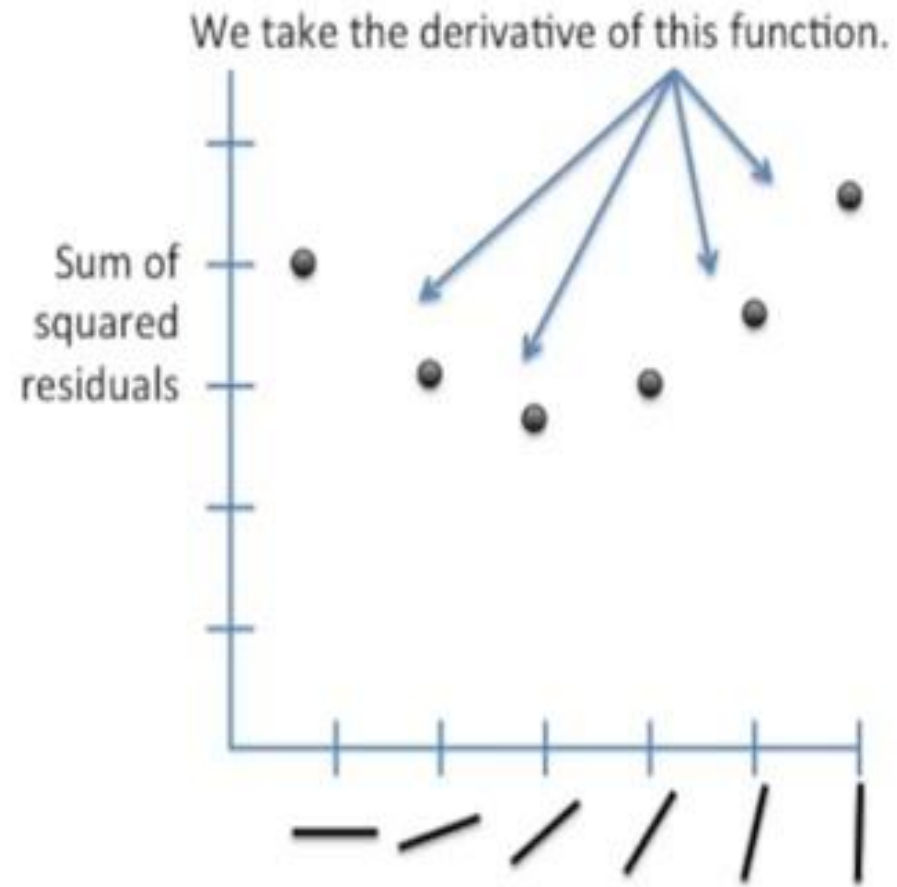
Since we want the line that will give us the smallest sum of squares, this method for finding the best values for " $a$ " and " $b$ " is called "Least Squares".



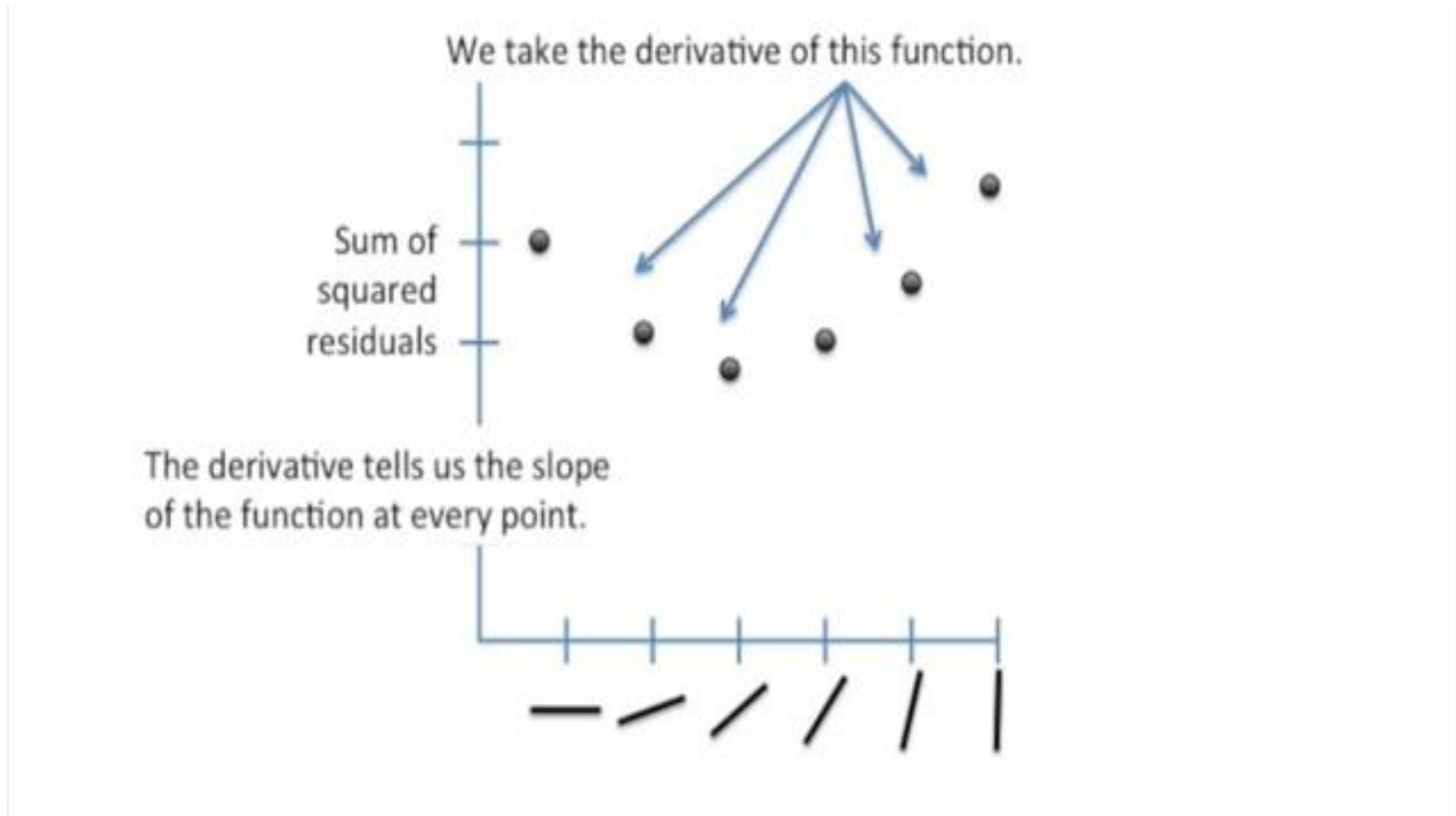
# Sum of Squares Residuals



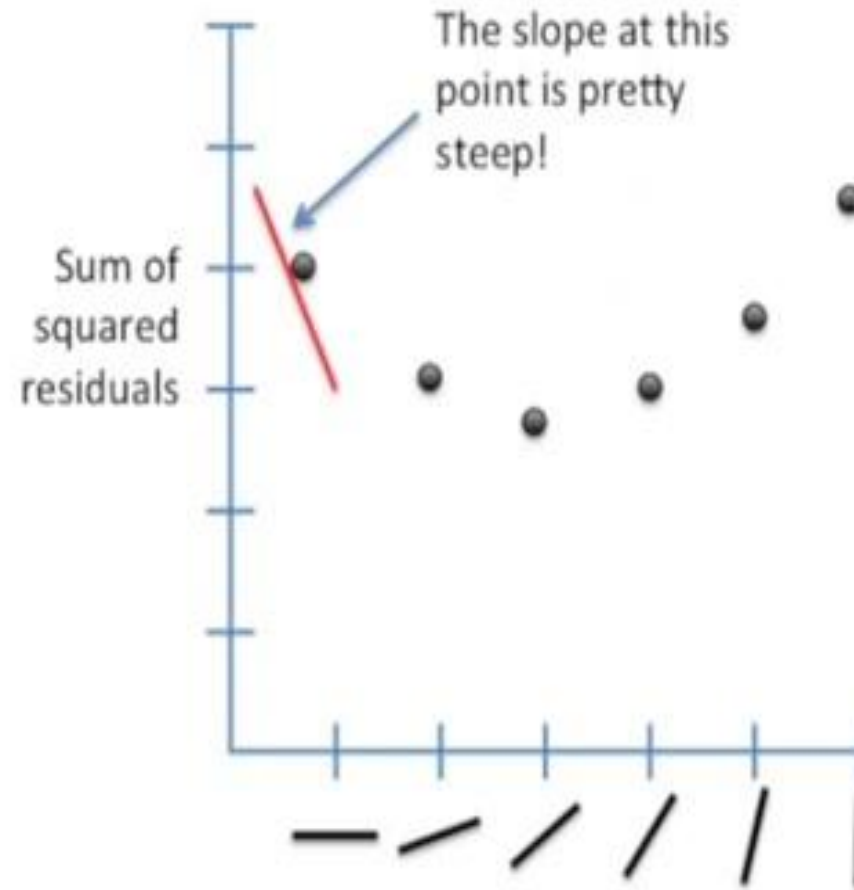
# Finding Best Rotation



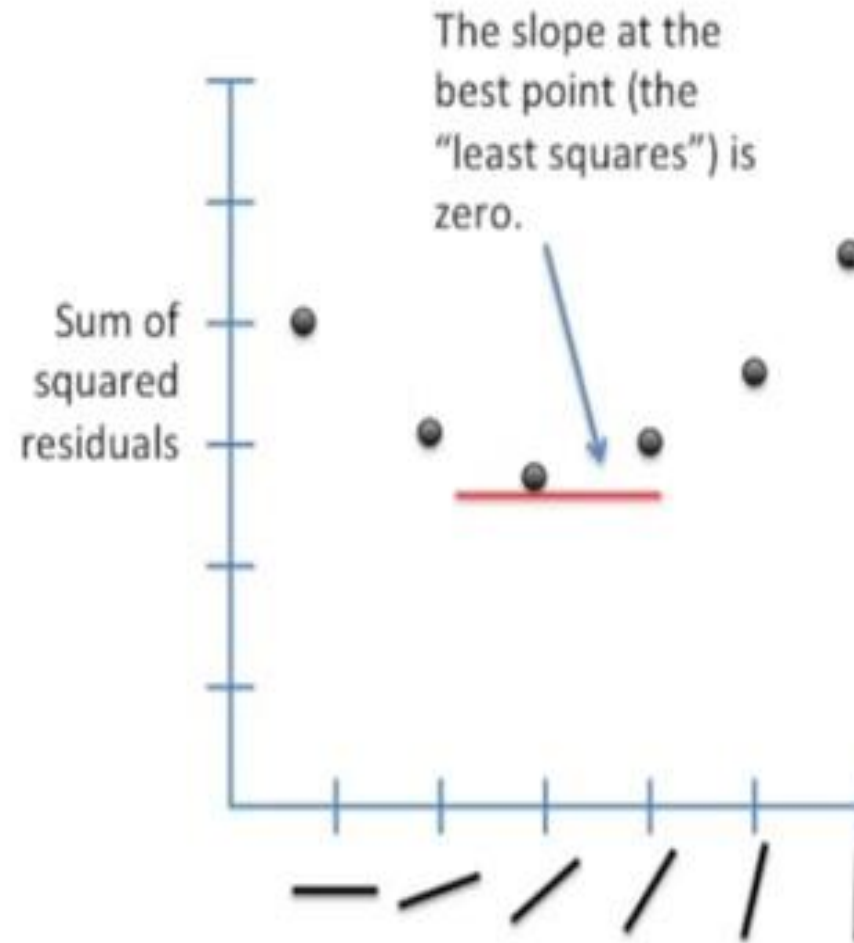
# Finding Best Rotation



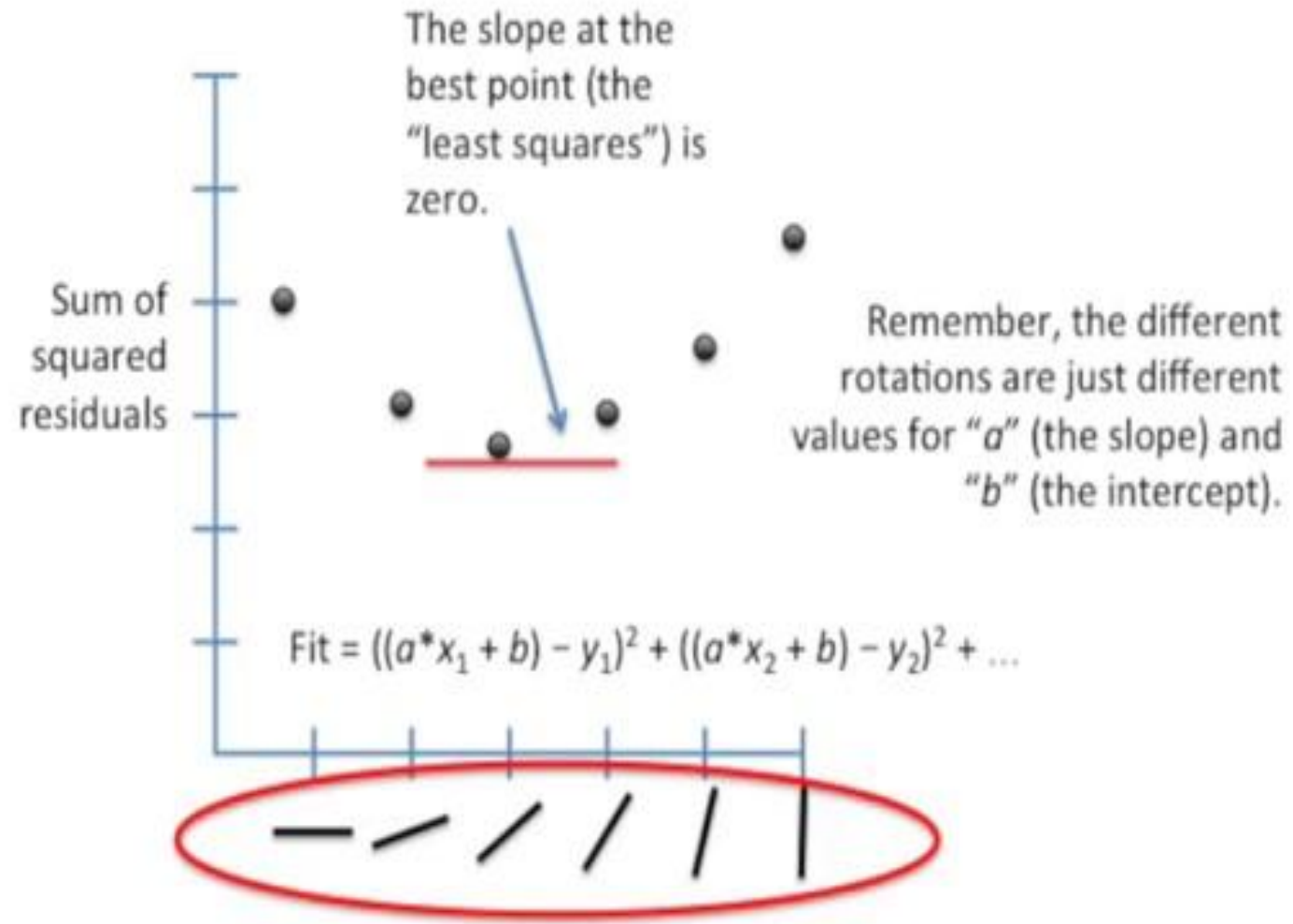
# Finding Best Rotation



# Finding Best Rotation

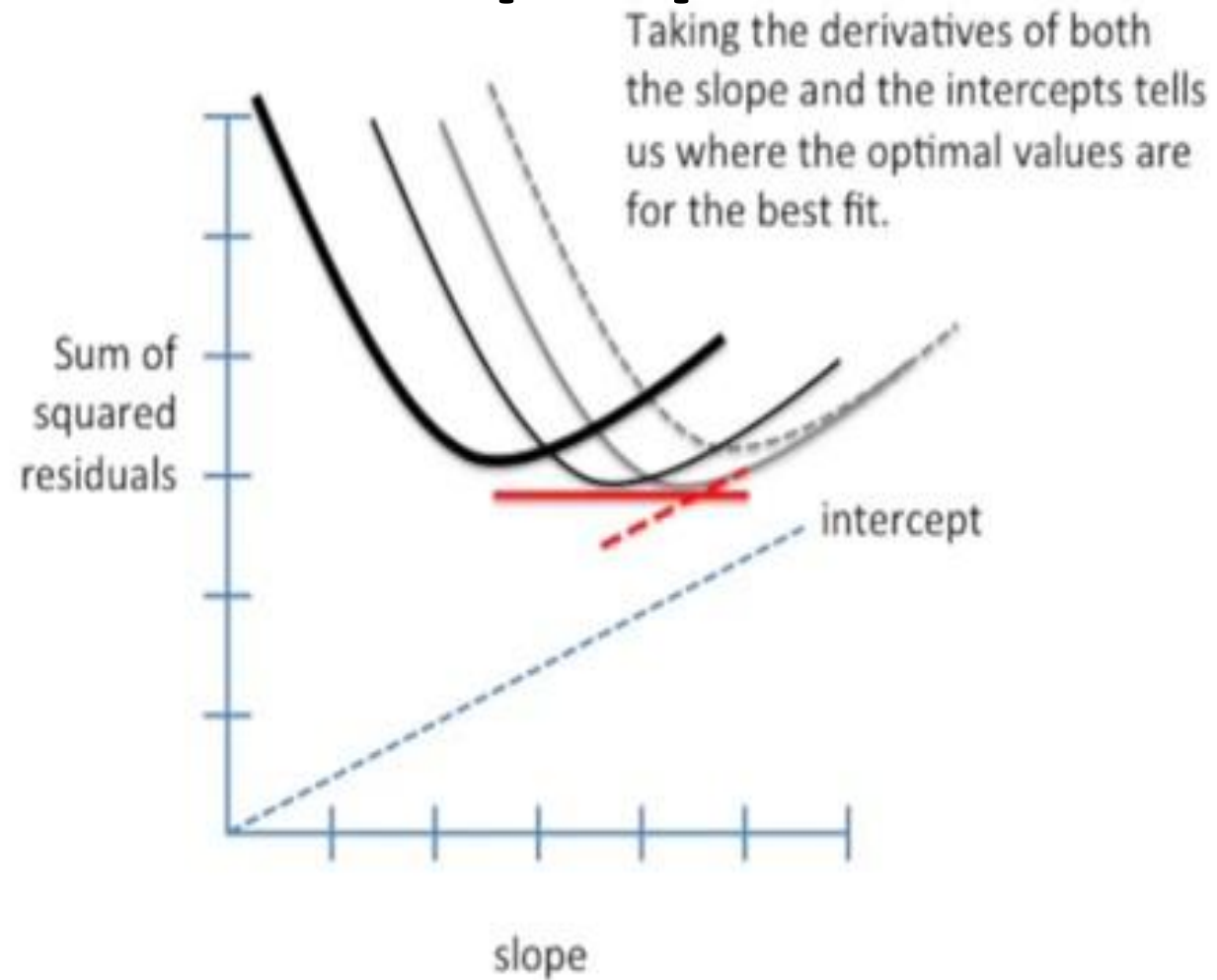


# Finding Best Rotation





# It works also for multiple params



# Simple Linear Regression

---

Linear Model:

|                       |                         |
|-----------------------|-------------------------|
| Dependent<br>Variable | Independent<br>Variable |
| $Y$                   | $X$                     |
| $=$                   | $+$                     |
| $m$                   | $b$                     |
| Slope                 | Intercept (bias)        |

$$Y = mX + b$$
$$Y = \beta_1 X + \beta_0$$

- In general, such a relationship may not hold exactly for the largely unobserved population
- We call the unobserved deviations from  $Y$  the errors.
- The goal is to find estimated values  $m'$  and  $b'$  for the parameters  $m$  and  $b$  which would provide the "best" fit for the data points.

# Least Square Method (LSM)

---

- A standard approach for doing this is to apply the **method of least squares** which attempts to find the parameters  $m, b$  that minimizes the sum of squared error.
- $SSE = \sum_i (y_i - f(x_i))^2 = \sum_i (y_i - mx_i - b)^2$
- also known as the **residual sum of squares**.
- The LSM finds  $m, b$  by setting to zero the first partial derivative of the above function w.r.t.  $m$  and  $b$  which are therefore calculated as follows:
  - $m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$
  - $b = (\sum y - m \sum x) / n$
- LSM can be extended to multiple linear regression.
- An alternative to find  $m, b$ , typically adopted in case of multivariate regression is the Gradient Descent method (see next lectures)

$$m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$$

$$b = (\sum y - m \sum x) / n$$

# LSM - Example

| "x"<br>Hours of<br>Sunshine | "y"<br>Ice Creams<br>Sold |
|-----------------------------|---------------------------|
| 2                           | 4                         |
| 3                           | 5                         |
| 5                           | 7                         |
| 7                           | 10                        |
| 9                           | 15                        |

Let us find the best **m** (slope) and **b** (y-intercept) that suits that data  
 $y = mx + b$

$$m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$$
$$b = (\sum y - m \sum x) / n$$

# LSM - Example

| x | y  | x <sup>2</sup> | xy  |
|---|----|----------------|-----|
| 2 | 4  | 4              | 8   |
| 3 | 5  | 9              | 15  |
| 5 | 7  | 25             | 35  |
| 7 | 10 | 49             | 70  |
| 9 | 15 | 81             | 135 |

Step 1: Calculate x<sup>2</sup> and xy

# LSM - Example

$$m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$$
$$b = (\sum y - m \sum x) / n$$

| x             | y             | x <sup>2</sup>             | xy              |
|---------------|---------------|----------------------------|-----------------|
| 2             | 4             | 4                          | 8               |
| 3             | 5             | 9                          | 15              |
| 5             | 7             | 25                         | 35              |
| 7             | 10            | 49                         | 70              |
| 9             | 15            | 81                         | 135             |
| <b>Σx: 26</b> | <b>Σy: 41</b> | <b>Σx<sup>2</sup>: 168</b> | <b>Σxy: 263</b> |

Step 2: Sum all the columns

# LSM - Example

$$m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$$
$$b = (\sum y - m \sum x) / n$$

| x             | y             | x <sup>2</sup>             | xy              |
|---------------|---------------|----------------------------|-----------------|
| 2             | 4             | 4                          | 8               |
| 3             | 5             | 9                          | 15              |
| 5             | 7             | 25                         | 35              |
| 7             | 10            | 49                         | 70              |
| 9             | 15            | 81                         | 135             |
| <b>Σx: 26</b> | <b>Σy: 41</b> | <b>Σx<sup>2</sup>: 168</b> | <b>Σxy: 263</b> |

Step 3: Calculate the slope and the intercept with N = 5

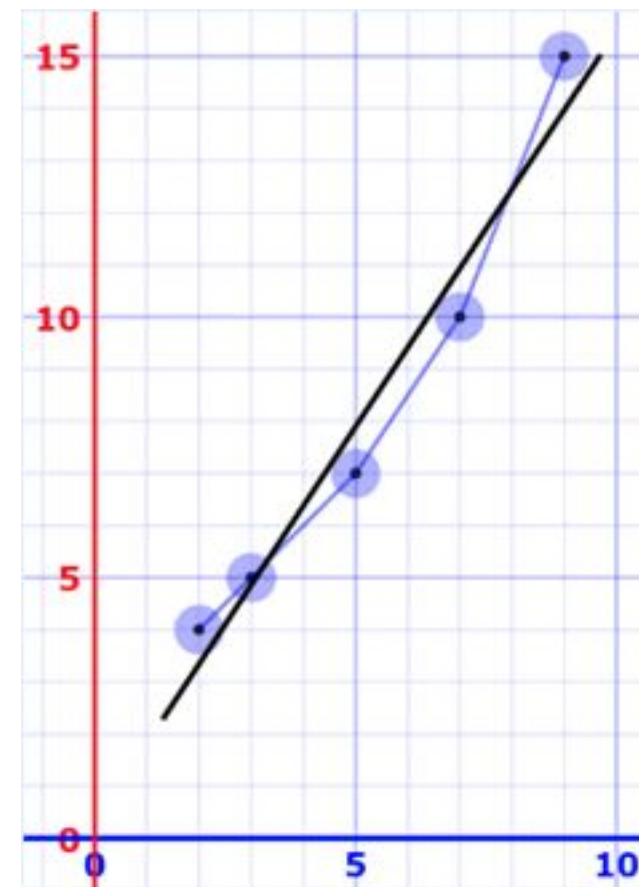
$$m = \frac{N \sum(xy) - \sum x \sum y}{N \sum(x^2) - (\sum x)^2}$$
$$= \frac{5 \times 263 - 26 \times 41}{5 \times 168 - 26^2}$$
$$= \frac{1315 - 1066}{840 - 676}$$
$$= \frac{249}{164} = 1,5183...$$

$$b = \frac{\sum y - m \sum x}{N}$$
$$= \frac{41 - 1,5183 \times 26}{5}$$
$$= 0,3049...$$

# LSM - Example

$$m = (n \sum(xy) - \sum x \sum y) / (n \sum(x^2) - (\sum x)^2)$$
$$b = (\sum y - m \sum x) / n$$

| x | y  | $y = 1,518x + 0,305$ | error |
|---|----|----------------------|-------|
| 2 | 4  | 3,34                 | -0,66 |
| 3 | 5  | 4,86                 | -0,14 |
| 5 | 7  | 7,89                 | 0,89  |
| 7 | 10 | 10,93                | 0,93  |
| 9 | 15 | 13,97                | -1,03 |



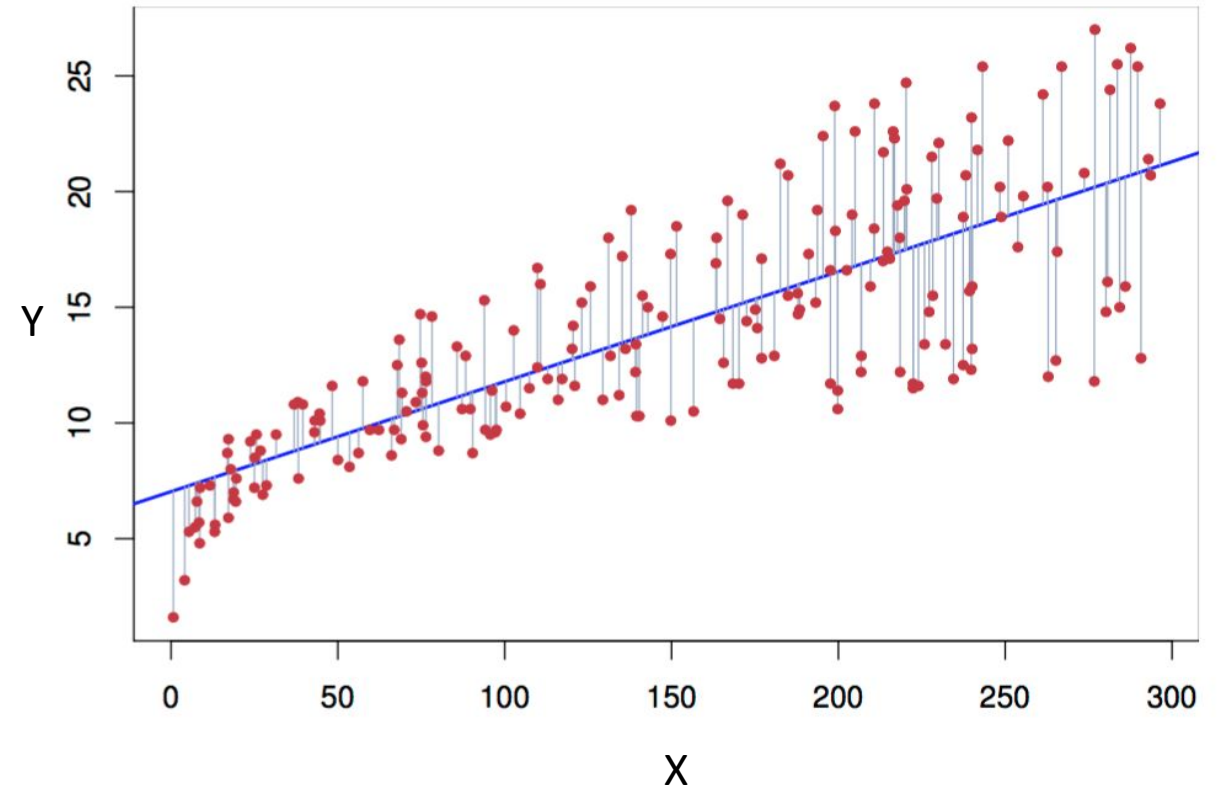
Step 4: test  $y = 1,518x + 0,305$

If  $x = 8$  then we expect to sell 12,45 ice creams

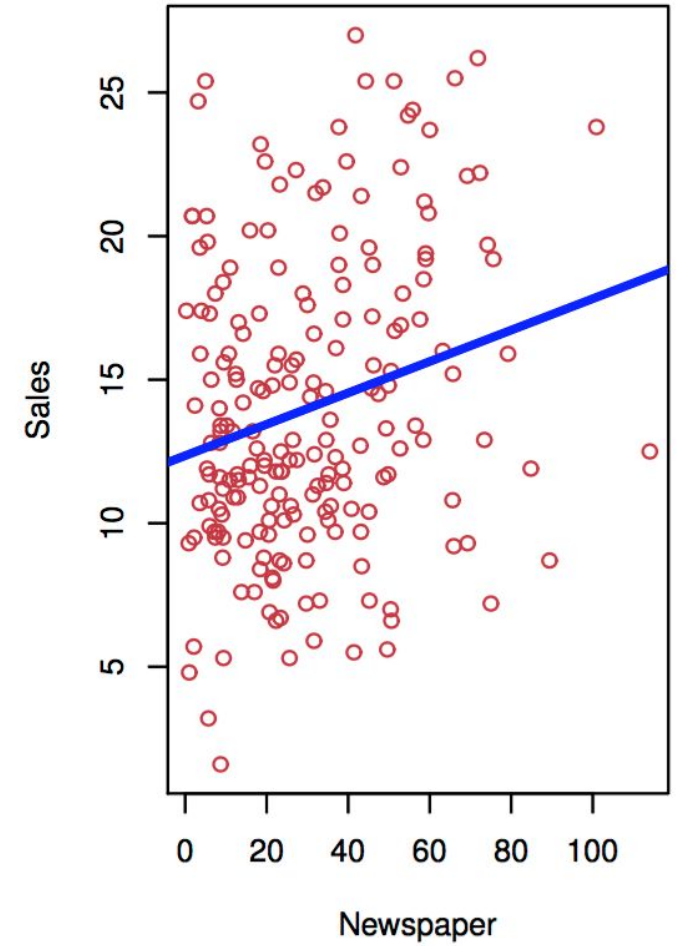
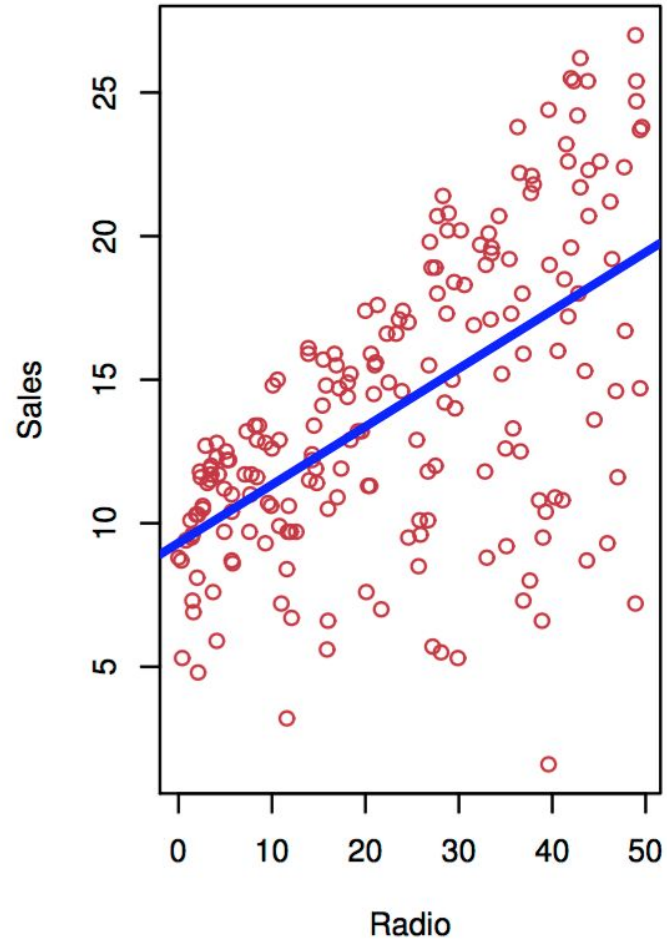
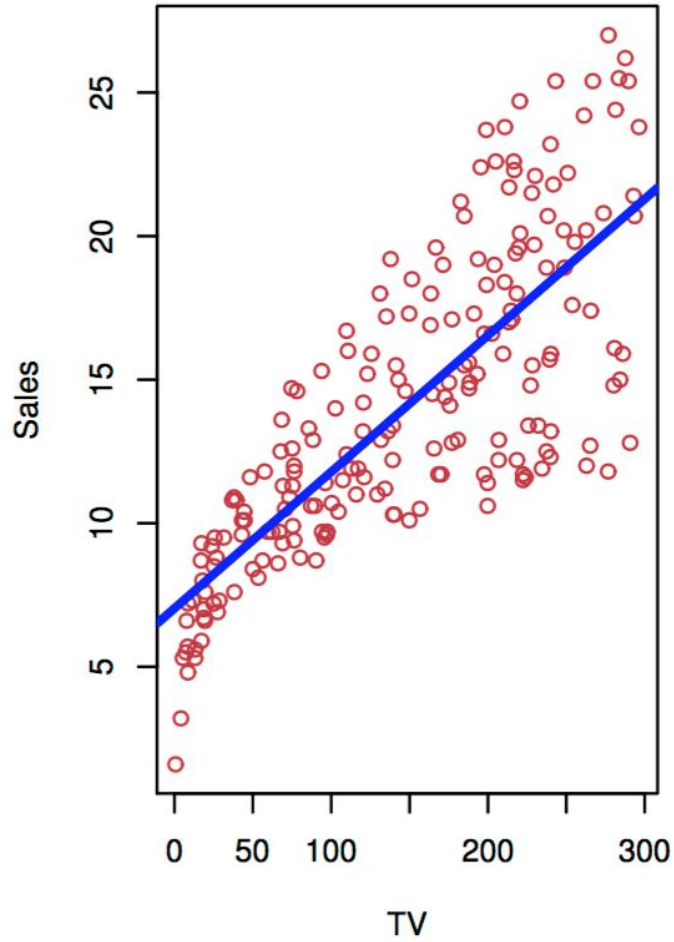


# Least Square Method

- Blue line shows the least square fit. Lines from red points to the regression line illustrate the residuals.
- For any other choice of slope  $m$  or intercept  $b$  the SSE between that line and the observed data would be larger than the SSE of the blue line.



# Examples



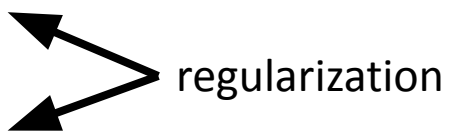
# Alternative Fitting Methods

---

- Linear regressions fitted using gradient descent can benefit from some regularizations.
- However, they can be fitted in other ways, such as by minimizing a penalized version of the least squares cost function as in **ridge regression** (L2-norm penalty) and **lasso** (L1-norm penalty).
- **Tikhonov** regularization, also known as *ridge regression*, is a method of regularization of ill-posed problems particularly useful to mitigate the multicollinearity, which commonly occurs in models with large numbers of parameters.
- **Lasso** (least absolute shrinkage and selection operator) performs both variable selection and regularization in order to enhance the prediction accuracy and interpretability of the statistical model it produces.

Multicollinearity: is a phenomenon in which one predictor variable in a multiple regression model can be linearly predicted from the others with a substantial degree of accuracy. In this situation, the coefficient estimates of the multiple regression may change erratically in response to small changes in the model or the data.

# Linear Regression Models Objective Functions

- Simple  $\beta_0 + \beta_1 x - y$
  - Multiple  $\beta_0 + \sum_i (y_i - \beta_i x_i)^2$
  - Ridge  $\beta_0 + \sum_i (y_i - \beta_i x_i)^2 + \lambda \sum_j \beta_j^2$
  - Lasso  $\beta_0 + \sum_i (y_i - \beta_i x_i)^2 + \lambda \sum_j |\beta_j|$
- 
- A diagram consisting of two arrows pointing from the text 'regularization' to the  $\lambda$  terms in the Ridge and Lasso objective functions. The top arrow points to  $\lambda \sum_j \beta_j^2$  in the Ridge formula, and the bottom arrow points to  $\lambda \sum_j |\beta_j|$  in the Lasso formula.

Ridge: mitigate the problem of multicollinearity

Lasso: variable selection, i.e., minimizes the number of coefficient different from zeros

# Evaluating Regression

- **Coefficient of determination  $R^2$**

- is the proportion of the variance in the dependent variable that is predictable from the independent variable(s)

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

hat means predicted  
 $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$  and  $\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n \epsilon_i^2$

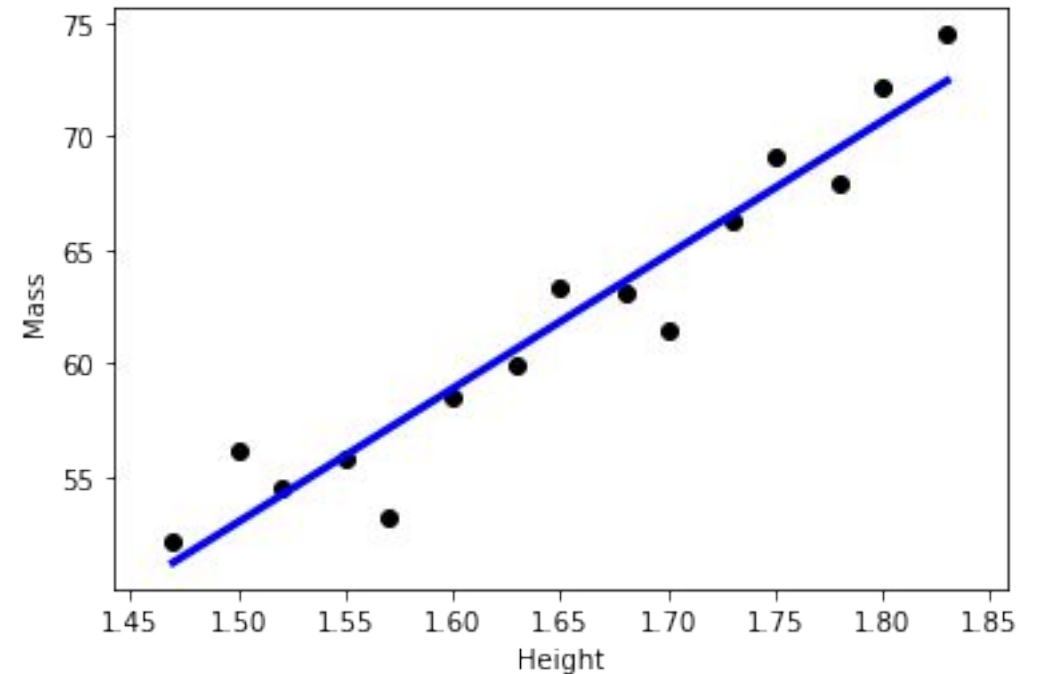
- **Mean Squared/Absolute Error MSE/MAE**

- a risk metric corresponding to the expected value of the squared (quadratic)/absolute error or loss

$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (y_i - \hat{y}_i)^2 \quad \text{MAE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} |y_i - \hat{y}_i|$$

# Example

- Height (m): 1.47, 1.50, 1.52, 1.55, 1.57, 1.60, 1.63, 1.65, 1.68, 1.70, 1.73, 1.75, 1.78, 1.80, 1.83
- Mass (kg): 52.21, 56.12, 54.48, 55.84, 53.20, 58.57, 59.93, 63.29, 63.11, 61.47, 66.28, 69.10, 67.92, 72.19, 74.46
- Intercept: -35.30454824113264
- Coefficient: 58.87472632
- $R^2$ : 0.93
- MSE: 3.40
- MAE: 1.43



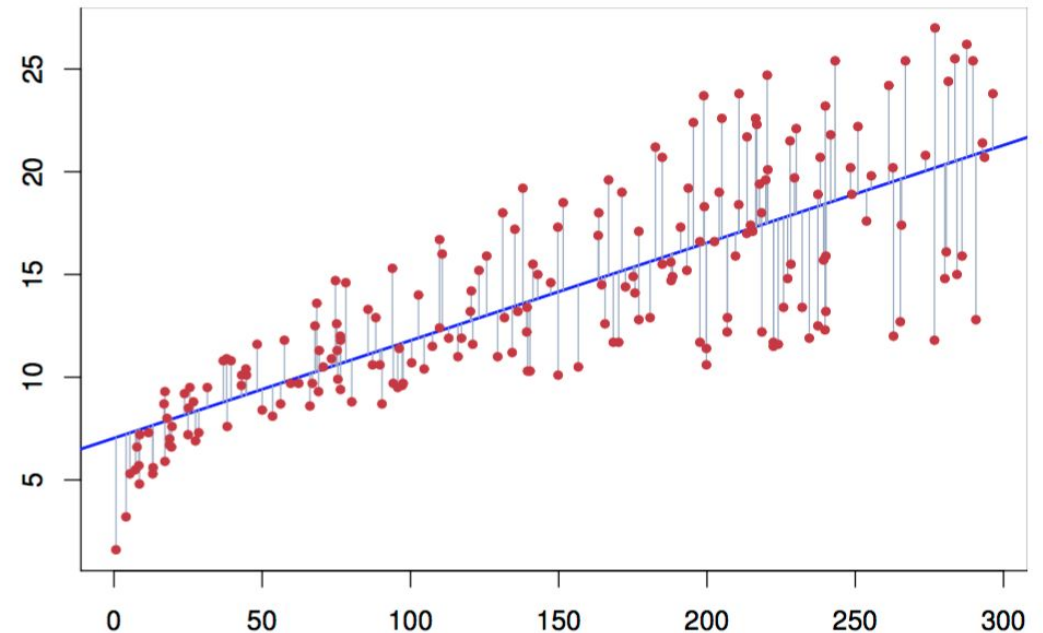
# Nonlinear Regression

---

# Linear Regression Recap

- Linear regression is used to fit a linear model to data where the dependent variable is continuous.
- Given a set of points  $(X_i, Y_i)$ , we wish to find a linear function (or line in 2 dimensions) that “goes through” these points.
- In general, the points are not exactly aligned.
- The objective is to find the line that best fits the points.

$$Y = \beta_1 X + \beta_0$$

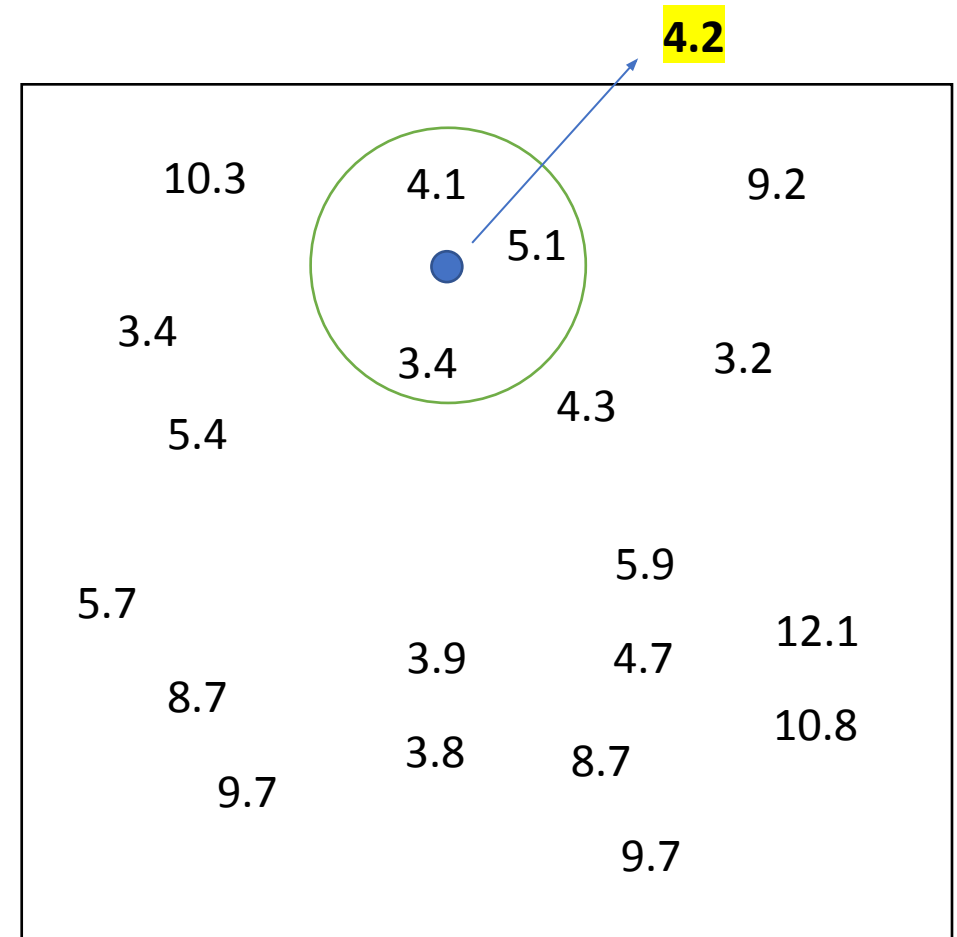




# k-NN for Regression

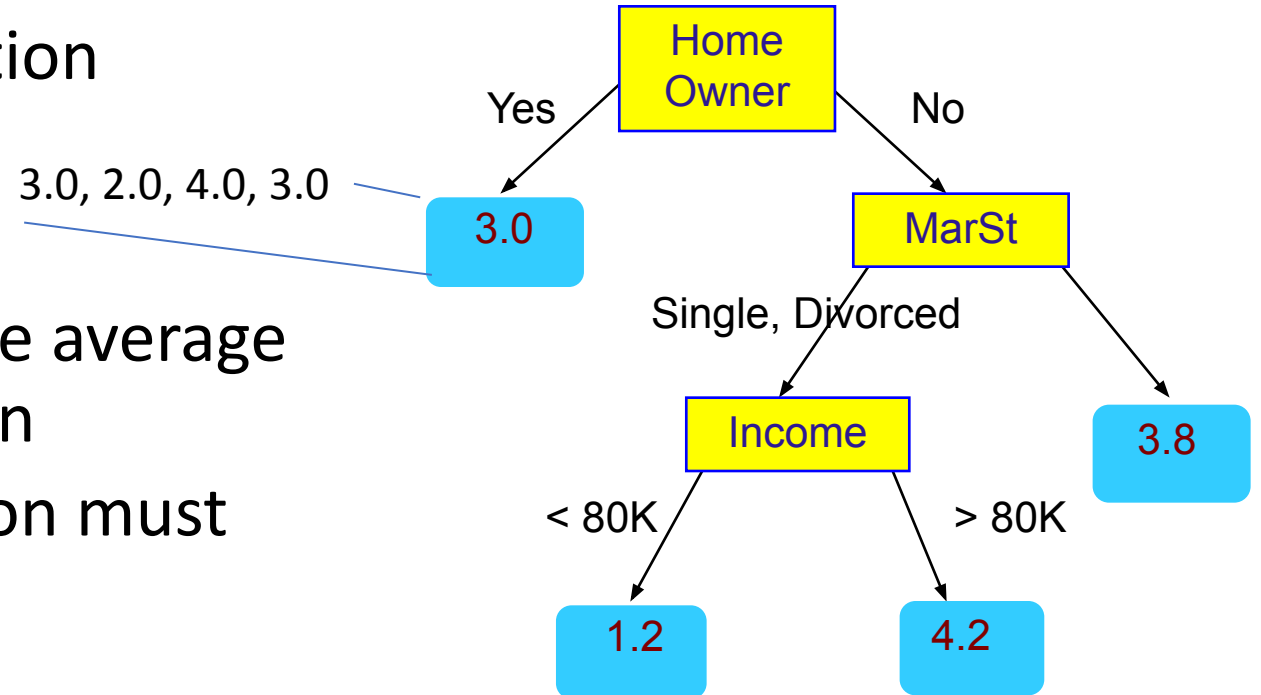
Given a set of training records (memory), and a test record:

1. **Compute the distances** from the records in the training to the test.
2. **Identify the  $k$  “nearest” records.**
3. Use target value of nearest neighbors to **determine the value** of unknown record (e.g., by averaging the values).



# Decision Trees for Regression

- The same induction and application procedures can be used.
- The only differences are:
  - When leaves are not pure, the average value is returned as prediction
  - Different optimization criterion must be used such as
    - MSE
    - MAE

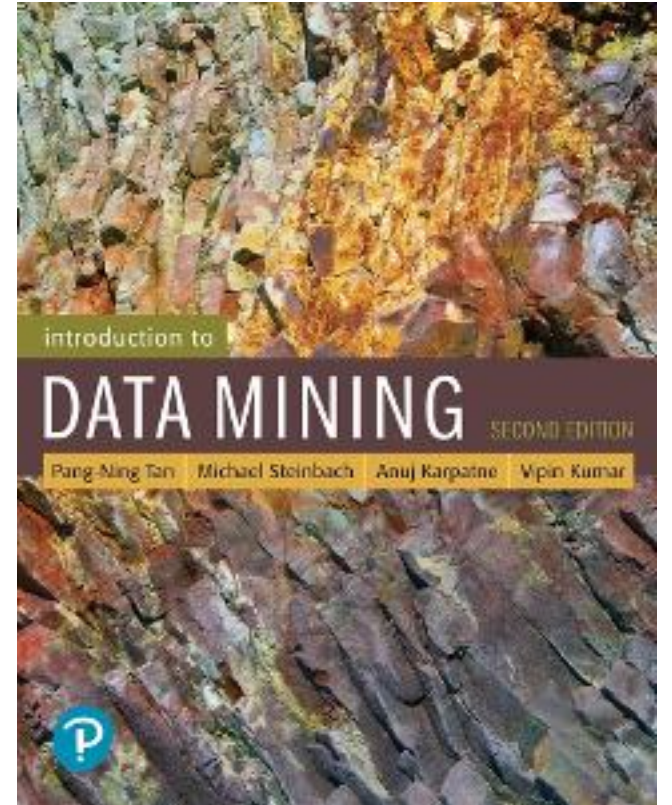


$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (y_i - \hat{y}_i)^2 \quad \text{MAE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} |y_i - \hat{y}_i|$$

# References

---

- Regression. Appendix D. Introduction to Data Mining.



# DATA MINING 1

## Pattern Mining & Association Rule Mining

---

Dino Pedreschi, Riccardo Guidotti

*Revisited slides from Lecture Notes for Chapter 5 “Introduction to Data Mining”, 2nd Edition by Tan, Steinbach, Karpatne, Kumar*



UNIVERSITÀ DI PISA

# Association Rules - Module Outline

---

- What are association rules (AR) and what are they used for:
  - The paradigmatic application: Market Basket Analysis
  - The single dimensional AR (intra-attribute)

# Market Basket Analysis: The Context

- Analyzing customer purchasing habits by finding associations and correlations between the different items that customers place in their “shopping basket”



Customer1



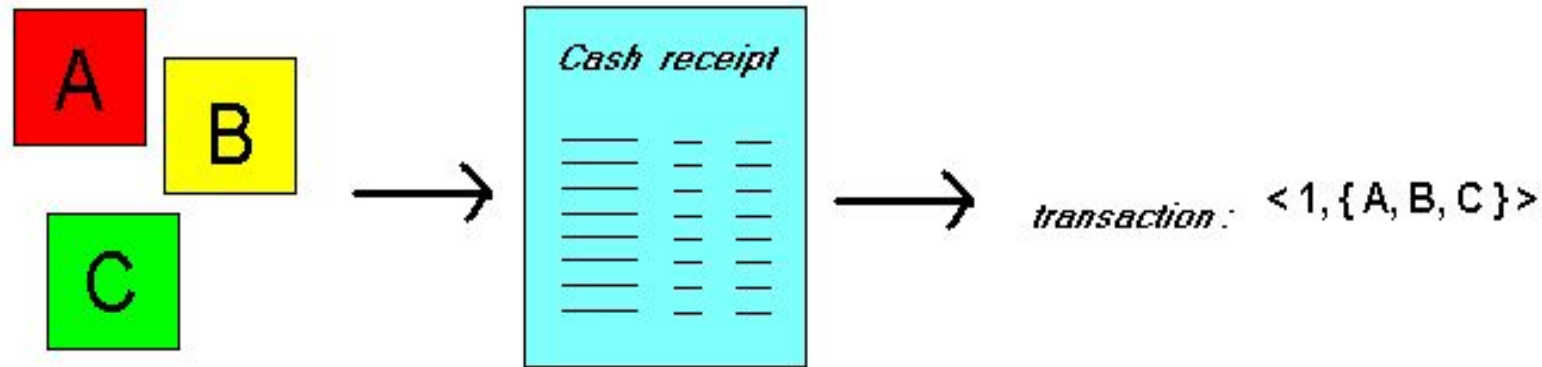
Customer2



Customer3

# Market Basket Analysis: The Context

- Given: a database of customer **transactions**, where each transaction is a **set of items**.
- Goal: Find groups of items which are **frequently purchased together**.



# Goal of MBA

---

- Extract information on purchasing behavior
- Actionable information: can suggest
  - new store layouts
  - new product assortments
  - which products to put on promotion
- MBA applicable whenever a customer purchases multiple things in proximity
  - credit cards
  - services of telecommunication companies
  - banking services
  - medical treatments



# MBA: applicable to many other contexts

---

Telecommunication:

Each customer is a transaction containing the set of customer's phone calls

Atmospheric phenomena:

Each time interval (e.g. a day) is a transaction containing the set of observed event (rains, wind, etc.)

Etc.

# Association Rules

---

- Express how product/services relate to each other, and tend to group together
- “if a customer purchases three-way calling, then will also purchase call-waiting”
- simple to understand
- actionable information: bundle three-way calling and call-waiting in a single package
- Examples.
  - Rule form: “Body  $\rightarrow$  Head [support, confidence]”.
  - $\text{buys}(x, \text{“diapers”}) \rightarrow \text{buys}(x, \text{“beers”}) [0.5\%, 60\%]$
  - $\text{major}(x, \text{“CS”}) \text{ and } \text{takes}(x, \text{“DB”}) \rightarrow \text{grade}(x, \text{“A”}) [1\%, 75\%]$

Body/Head/Antecedent       $X \rightarrow Y$       Head/Tail/Consequent

# Useful, trivial, unexplicable

---

- **Useful**: “On Thursdays, grocery store consumers often purchase diapers and beer together”.
- **Trivial**: “Customers who purchase maintenance agreements are very likely to purchase large appliances”.
- **Unexplicable**: “When a new hardware store opens, one of the most sold items is toilet rings.”

# Apriori

---

# Association Rule Mining

- Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction

## Market-Basket transactions

| <i><b>TID</b></i> | <i><b>Items</b></i>              |
|-------------------|----------------------------------|
| <b>1</b>          | <b>Bread, Milk</b>               |
| <b>2</b>          | <b>Bread, Diaper, Beer, Eggs</b> |
| <b>3</b>          | <b>Milk, Diaper, Beer, Coke</b>  |
| <b>4</b>          | <b>Bread, Milk, Diaper, Beer</b> |
| <b>5</b>          | <b>Bread, Milk, Diaper, Coke</b> |

## Example of Association Rules

$\{\text{Diaper}\} \rightarrow \{\text{Beer}\},$   
 $\{\text{Milk, Bread}\} \rightarrow \{\text{Eggs, Coke}\},$   
 $\{\text{Beer, Bread}\} \rightarrow \{\text{Milk}\},$

Implication means co-occurrence,  
not causality!

Find groups of items which are frequently purchased together

# Definition: Frequent Itemset

- **Itemset**

- A collection of one or more items
  - Example: {Milk, Bread, Diaper}
- k-itemset
  - An itemset that contains k items

- **Support count ( $\sigma$ )**

- Frequency of occurrence of an itemset
- E.g.  $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$

- **Support**

- Fraction of transactions that contain an itemset
- E.g.  $s(\{\text{Milk, Bread, Diaper}\}) = 2/5$

- **Frequent Itemset**

- An itemset whose support is greater than or equal to a *minsup* threshold

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Bread, Diaper, Beer, Eggs |
| 3          | Milk, Diaper, Beer, Coke  |
| 4          | Bread, Milk, Diaper, Beer |
| 5          | Bread, Milk, Diaper, Coke |

# Definition: Association Rule

- Association Rule
  - An implication expression of the form  $X \rightarrow Y$ , where X and Y are itemsets
  - Example:  
 $\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Bread, Diaper, Beer, Eggs |
| 3          | Milk, Diaper, Beer, Coke  |
| 4          | Bread, Milk, Diaper, Beer |
| 5          | Bread, Milk, Diaper, Coke |

- Rule Evaluation Metrics
  - Support (s)
    - ♦ Fraction of transactions that contain both X and Y
  - Confidence (c)
    - ♦ Measures how often items in Y appear in transactions that contain X

Example:

$\{\text{Milk, Diaper}\} \Rightarrow \{\text{Beer}\}$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

# Association rules - module outline

---

- How to compute AR
  - Basic Apriori Algorithm and its optimizations
  - Multi-Dimension AR (inter-attribute)
  - Quantitative AR



# Association Rule Mining Task

---

- Given a set of transactions  $T$ , the goal of association rule mining is to find all rules having
    - support  $\geq \textit{minsup}$  threshold
    - confidence  $\geq \textit{minconf}$  threshold
  - Brute-force approach:
    - List all possible association rules
    - Compute the support and confidence for each rule
    - Prune rules that fail the *minsup* and *minconf* thresholds
- ⇒ **Computationally prohibitive!**

# Mining Association Rules

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Bread, Diaper, Beer, Eggs |
| 3          | Milk, Diaper, Beer, Coke  |
| 4          | Bread, Milk, Diaper, Beer |
| 5          | Bread, Milk, Diaper, Coke |

## Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$  ( $s=0.4$ ,  $c=0.67$ )

$\{\text{Milk, Beer}\} \rightarrow \{\text{Diaper}\}$  ( $s=0.4$ ,  $c=1.0$ )

$\{\text{Diaper, Beer}\} \rightarrow \{\text{Milk}\}$  ( $s=0.4$ ,  $c=0.67$ )

$\{\text{Beer}\} \rightarrow \{\text{Milk, Diaper}\}$  ( $s=0.4$ ,  $c=0.67$ )

$\{\text{Diaper}\} \rightarrow \{\text{Milk, Beer}\}$  ( $s=0.4$ ,  $c=0.5$ )

$\{\text{Milk}\} \rightarrow \{\text{Diaper, Beer}\}$  ( $s=0.4$ ,  $c=0.5$ )

## Observations:

- All the above rules are binary partitions of the same itemset:  
 $\{\text{Milk, Diaper, Beer}\}$
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

# Mining Association Rules

---

- Two-step approach:
  1. Frequent Itemset Generation
    - Generate all itemsets whose support  $\geq$  minsup
  2. Rule Generation
    - Generate high confidence rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset
- Frequent itemset generation is still computationally expensive

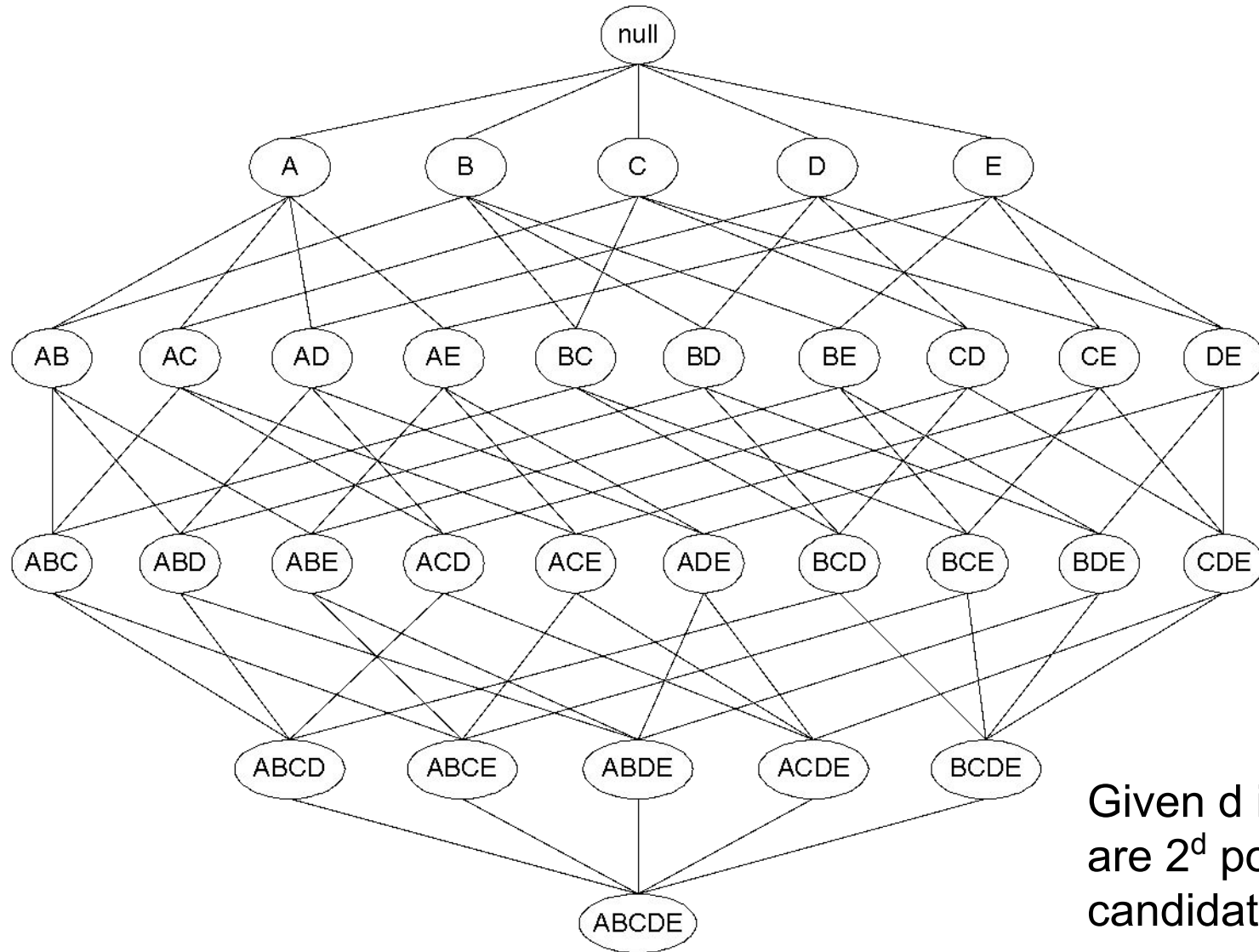
# Basic Apriori Algorithm

---

## Problem Decomposition

1. Find the *frequent itemsets*: the sets of items that satisfy the support constraint
  - A subset of a frequent itemset is also a frequent itemset, i.e., if  $\{A, B\}$  is a frequent itemset, both  $\{A\}$  and  $\{B\}$  should be a frequent itemset
  - Iteratively find frequent itemsets with cardinality from 1 to  $k$  ( $k$ -itemset)
2. Use the frequent itemsets to generate association rules.

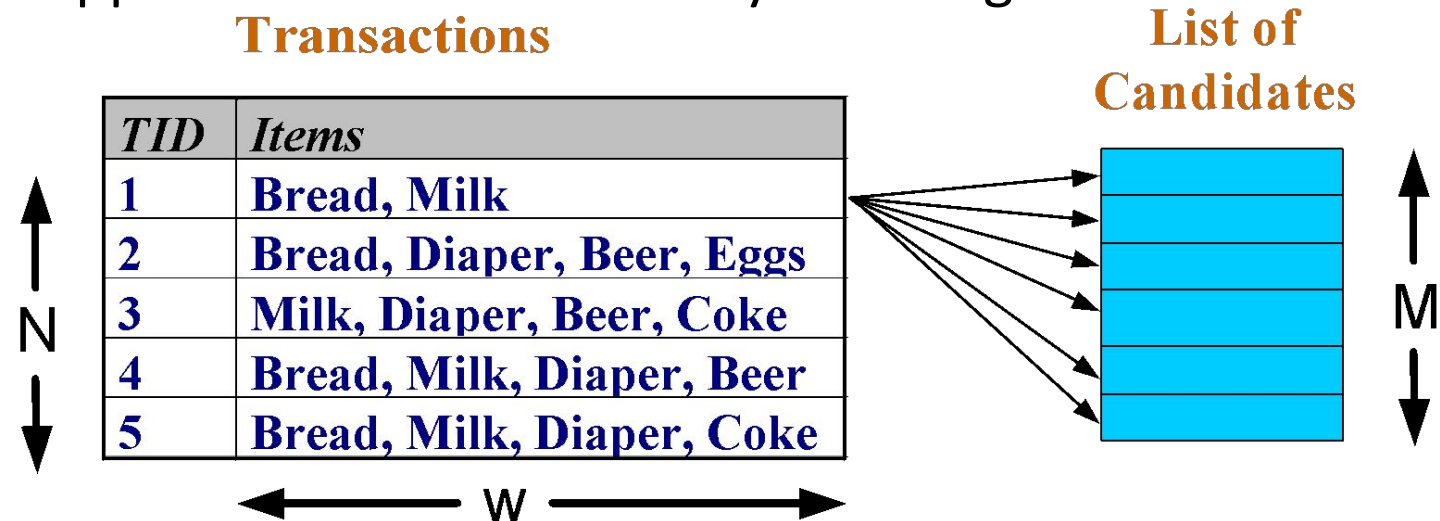
# Frequent Itemset Generation



Given  $d$  items, there are  $2^d$  possible candidate itemsets

# Frequent Itemset Generation

- Brute-force approach:
  - Each itemset in the lattice is a **candidate** frequent itemset
  - Count the support of each candidate by scanning the database



- Match each transaction against every candidate
- Complexity  $\sim O(NMw) \Rightarrow$  **Expensive since  $M = 2^d$  !!!**

# Frequent Itemset Generation Strategies

---

- Reduce the **number of candidates** (M)
  - Complete search:  $M=2^d$
  - Use pruning techniques to reduce M
- Reduce the **number of transactions** (N)
  - Reduce size of N as the size of itemset increases
- Reduce the **number of comparisons** (NM)
  - Use efficient data structures to store the candidates or transactions
  - No need to match every candidate against every transaction

# Reducing Number of Candidates

---

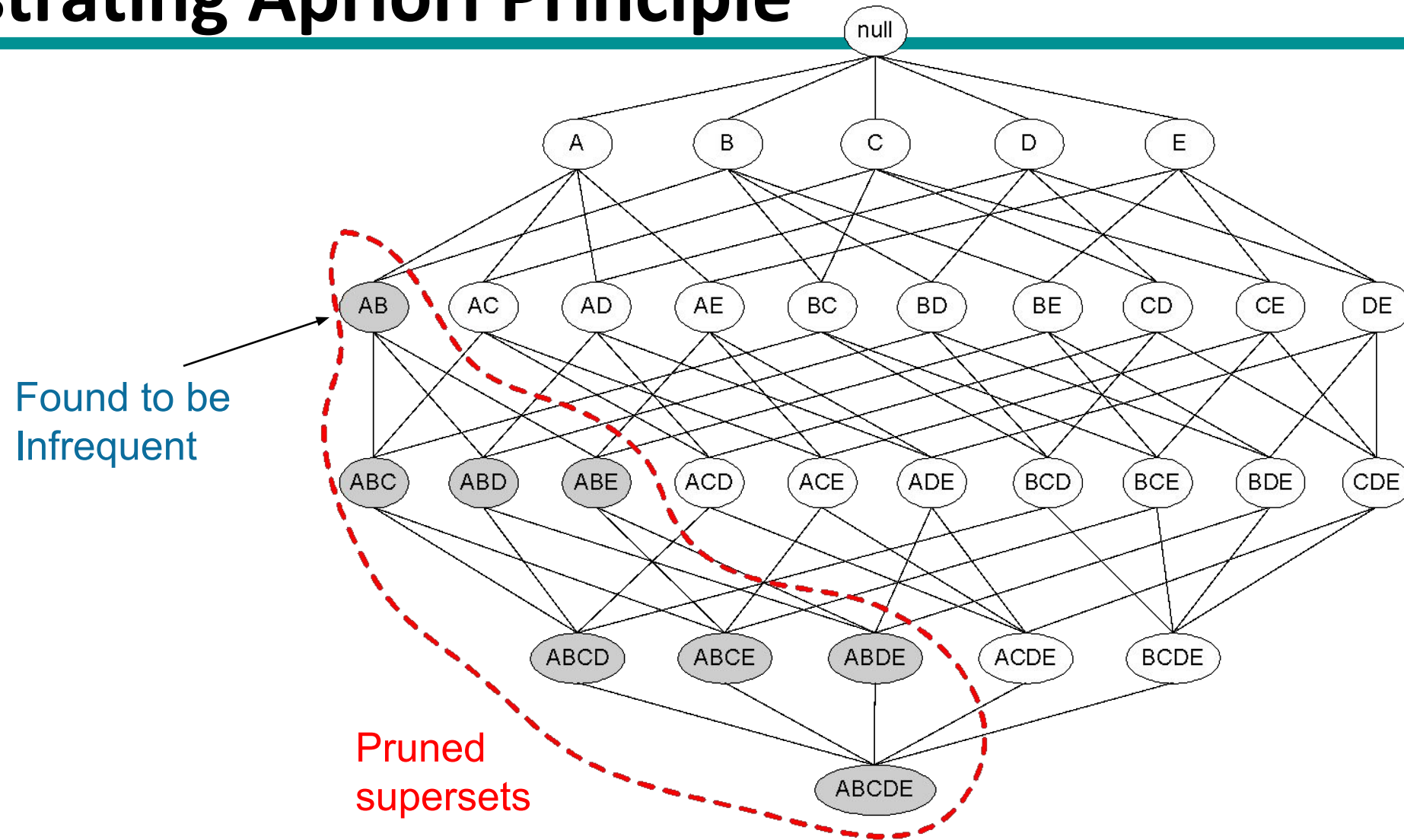
- **Apriori principle:**
  - If an itemset is frequent, then all of its subsets must also be frequent
- Apriori principle holds due to the following property of the support measure:

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- Support of an itemset never exceeds the support of its subsets
- This is known as the **anti-monotone** property of support



# Illustrating Apriori Principle



# Illustrating Apriori Principle

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Beer, Bread, Diaper, Eggs |
| 3          | Beer, Coke, Diaper, Milk  |
| 4          | Beer, Bread, Diaper, Milk |
| 5          | Bread, Coke, Diaper, Milk |



Items (1-itemsets)

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Minimum Support = 3

# Illustrating Apriori Principle

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Beer, Bread, Diaper, Eggs |
| 3          | Beer, Coke, Diaper, Milk  |
| 4          | Beer, Bread, Diaper, Milk |
| 5          | Bread, Coke, Diaper, Milk |



Items (1-itemsets)

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Minimum Support = 3

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        |
|----------------|
| {Bread,Milk}   |
| {Bread, Beer } |
| {Bread,Diaper} |
| {Beer, Milk}   |
| {Diaper, Milk} |
| {Beer,Diaper}  |

Minimum Support = 3

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        | Count |
|----------------|-------|
| {Bread,Milk}   | 3     |
| {Beer, Bread}  | 2     |
| {Bread,Diaper} | 3     |
| {Beer,Milk}    | 2     |
| {Diaper,Milk}  | 3     |
| {Beer,Diaper}  | 3     |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        | Count |
|----------------|-------|
| {Bread,Milk}   | 3     |
| {Beer, Bread}  | 2     |
| {Bread,Diaper} | 3     |
| {Beer,Milk}    | 2     |
| {Diaper,Milk}  | 3     |
| {Beer,Diaper}  | 3     |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)



| Itemset                    |
|----------------------------|
| { Beer, Diaper, Milk}      |
| { Beer,Bread,Diaper}       |
| <b>{Bread,Diaper,Milk}</b> |
| { Beer, Bread, Milk}       |

Triplets (3-itemsets)

(No need to generate candidates involving Bread, Beer or Milk, Beer)

Minimum Support = 3

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        | Count |
|----------------|-------|
| {Bread,Milk}   | 3     |
| {Beer, Bread}  | 2     |
| {Bread,Diaper} | 3     |
| {Beer,Milk}    | 2     |
| {Diaper,Milk}  | 3     |
| {Beer,Diaper}  | 3     |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3



Triplets (3-itemsets)

| Itemset                      | Count    |
|------------------------------|----------|
| { Beer, Diaper, Milk}        | 2        |
| { Beer,Bread, Diaper}        | 2        |
| <b>{Bread, Diaper, Milk}</b> | <b>2</b> |
| {Beer, Bread, Milk}          | 1        |

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        | Count |
|----------------|-------|
| {Bread,Milk}   | 3     |
| {Beer, Bread}  | 2     |
| {Bread,Diaper} | 3     |
| {Beer,Milk}    | 2     |
| {Diaper,Milk}  | 3     |
| {Beer,Diaper}  | 3     |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3



Triplets (3-itemsets)

| Itemset               | Count |
|-----------------------|-------|
| { Beer, Diaper, Milk} | 2     |
| { Beer,Bread, Diaper} | 2     |
| {Bread, Diaper, Milk} | 2     |
| {Beer, Bread, Milk}   | 1     |

| TID | Items                     |
|-----|---------------------------|
| 1   | Bread, Milk               |
| 2   | Beer, Bread, Diaper, Eggs |
| 3   | Beer, Coke, Diaper, Milk  |
| 4   | Beer, Bread, Diaper, Milk |
| 5   | Bread, Coke, Diaper, Milk |



# Apriori Algorithm

---

- $F_k$ : frequent k-itemsets
- $L_k$ : candidate k-itemsets
- Algorithm
  - Let  $k=1$
  - Generate  $F_1 = \{\text{frequent 1-itemsets}\}$
  - Repeat until  $F_k$  is empty
    - **Candidate Generation:** Generate  $L_{k+1}$  from  $F_k$
    - **Candidate Pruning:** Prune candidate itemsets in  $L_{k+1}$  containing subsets of length  $k$  that are infrequent
    - **Support Counting:** Count the support of each candidate in  $L_{k+1}$  by scanning the DB
    - **Candidate Elimination:** Eliminate candidates in  $L_{k+1}$  that are infrequent, leaving only those that are frequent  $\Rightarrow F_{k+1}$

# Candidate Generation: $F_{k-1} \times F_{k-1}$ Method

---

- Merge two frequent  $(k-1)$ -itemsets if they have a common element (if  $k = 2$ )
- Merge two frequent  $(k-1)$ -itemsets if their first  $(k-2)$  items are identical (if  $k > 2$ )
- $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$ 
  - Merge(ABC, ABD) = ABCD
  - Merge(ABC, ABE) = ABCE
  - Merge(ABD, ABE) = ABDE
  - Do not merge(ABD, ACD) because they share only prefix of length 1 instead of length 2

# Candidate Pruning

---

- Let  $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$  be the set of frequent 3-itemsets
- $L_4 = \{ABCD, ABCE, ABDE\}$  is the set of candidate 4-itemsets generated (from previous slide)
- Candidate pruning
  - Prune ABCE because ACE and BCE are infrequent
  - Prune ABDE because ADE is infrequent
- After candidate pruning:  $L_4 = \{ABCD\}$

# Illustrating Apriori Principle

| Item   | Count |
|--------|-------|
| Bread  | 4     |
| Coke   | 2     |
| Milk   | 4     |
| Beer   | 3     |
| Diaper | 4     |
| Eggs   | 1     |

Items (1-itemsets)



| Itemset        | Count |
|----------------|-------|
| {Bread,Milk}   | 3     |
| {Bread,Beer}   | 2     |
| {Bread,Diaper} | 3     |
| {Milk,Beer}    | 2     |
| {Milk,Diaper}  | 3     |
| {Beer,Diaper}  | 3     |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3



Triplets (3-itemsets)

| Itemset               | Count |
|-----------------------|-------|
| {Bread, Diaper, Milk} | 2     |

Use of  $F_{k-1} \times F_{k-1}$  method for candidate generation results in only one 3-itemset. This is eliminated after the support counting step.

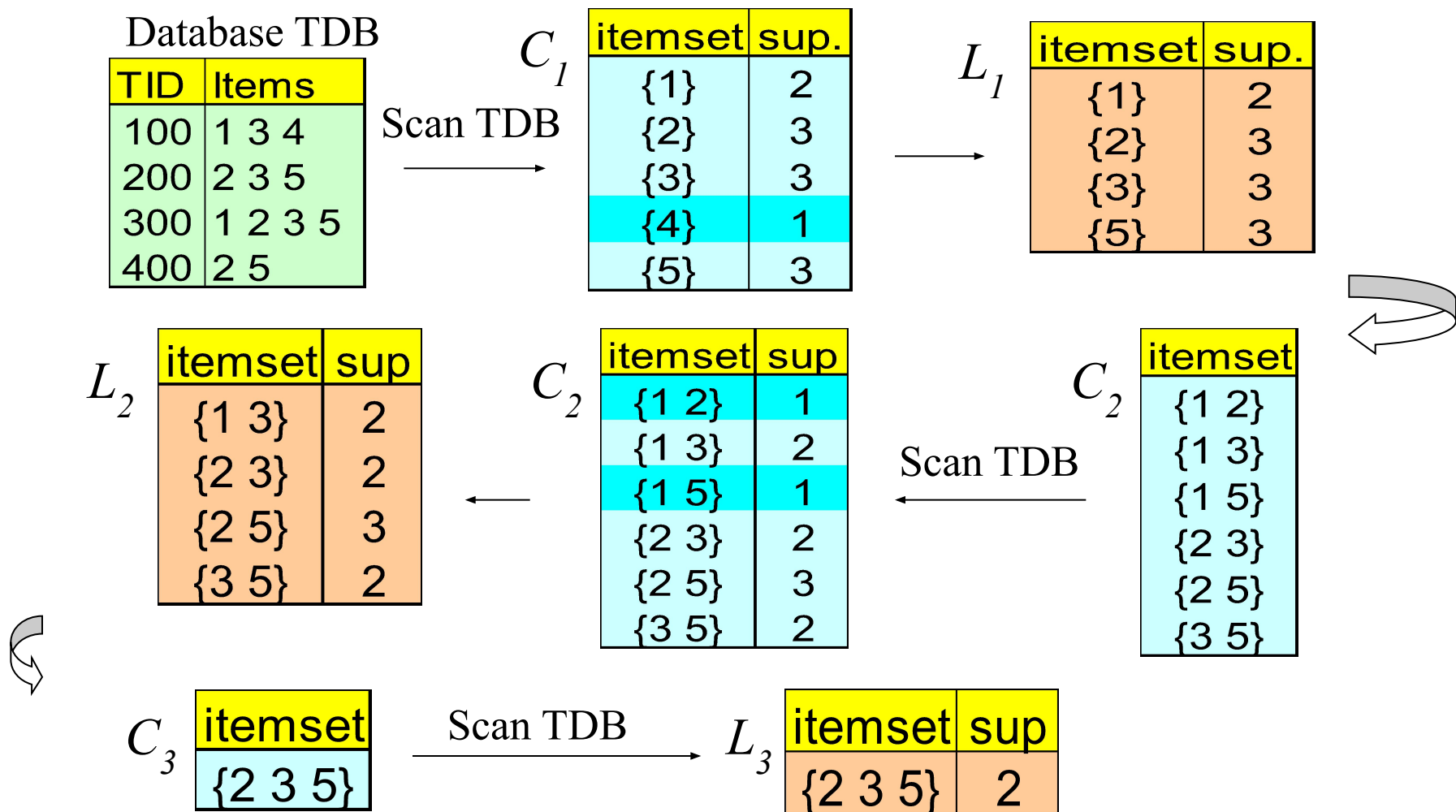
# Support Counting of Candidate Itemsets

- Scan the database of transactions to determine the support of each candidate itemset
- Must match every candidate itemset against every transaction, which is an expensive operation

| <i>TID</i> | <i>Items</i>              |
|------------|---------------------------|
| 1          | Bread, Milk               |
| 2          | Beer, Bread, Diaper, Eggs |
| 3          | Beer, Coke, Diaper, Milk  |
| 4          | Beer, Bread, Diaper, Milk |
| 5          | Bread, Coke, Diaper, Milk |

| Itemset                      |
|------------------------------|
| { Beer, Diaper, Milk}        |
| { Beer,Bread,Diaper}         |
| <b>{Bread, Diaper, Milk}</b> |
| { Beer, Bread, Milk}         |

# Apriori Execution Example ( $\text{min\_sup} = 2$ )



# Rule Generation

---

- Given a frequent itemset  $L$ , find all non-empty subsets  $f \subset L$  such that  $f \rightarrow L - f$  satisfies the minimum confidence requirement
  - If  $\{A,B,C,D\}$  is a frequent itemset, candidate rules:  
ABC  $\rightarrow$  D,    ABD  $\rightarrow$  C,    ACD  $\rightarrow$  B,    BCD  $\rightarrow$  A,  
A  $\rightarrow$  BCD,    B  $\rightarrow$  ACD,    C  $\rightarrow$  ABD,    D  $\rightarrow$  ABC  
AB  $\rightarrow$  CD,    AC  $\rightarrow$  BD,    AD  $\rightarrow$  BC,    BC  $\rightarrow$  AD,  
BD  $\rightarrow$  AC,    CD  $\rightarrow$  AB,
- If  $|L| = k$ , then there are  $2^k - 2$  candidate association rules (ignoring  $L \rightarrow \emptyset$  and  $\emptyset \rightarrow L$ )

# Rule Generation

$$\{\text{Milk, Diaper}\} \Rightarrow \{\text{Beer}\} \quad c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

- In general, confidence does not have an anti-monotone property  
 $c(ABC \rightarrow D)$  can be larger or smaller than  $c(AB \rightarrow D)$

- But confidence of rules generated from the same itemset has an anti-monotone property

- E.g., Suppose  $\{A, B, C, D\}$  is a frequent 4-itemset:

$$c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$$

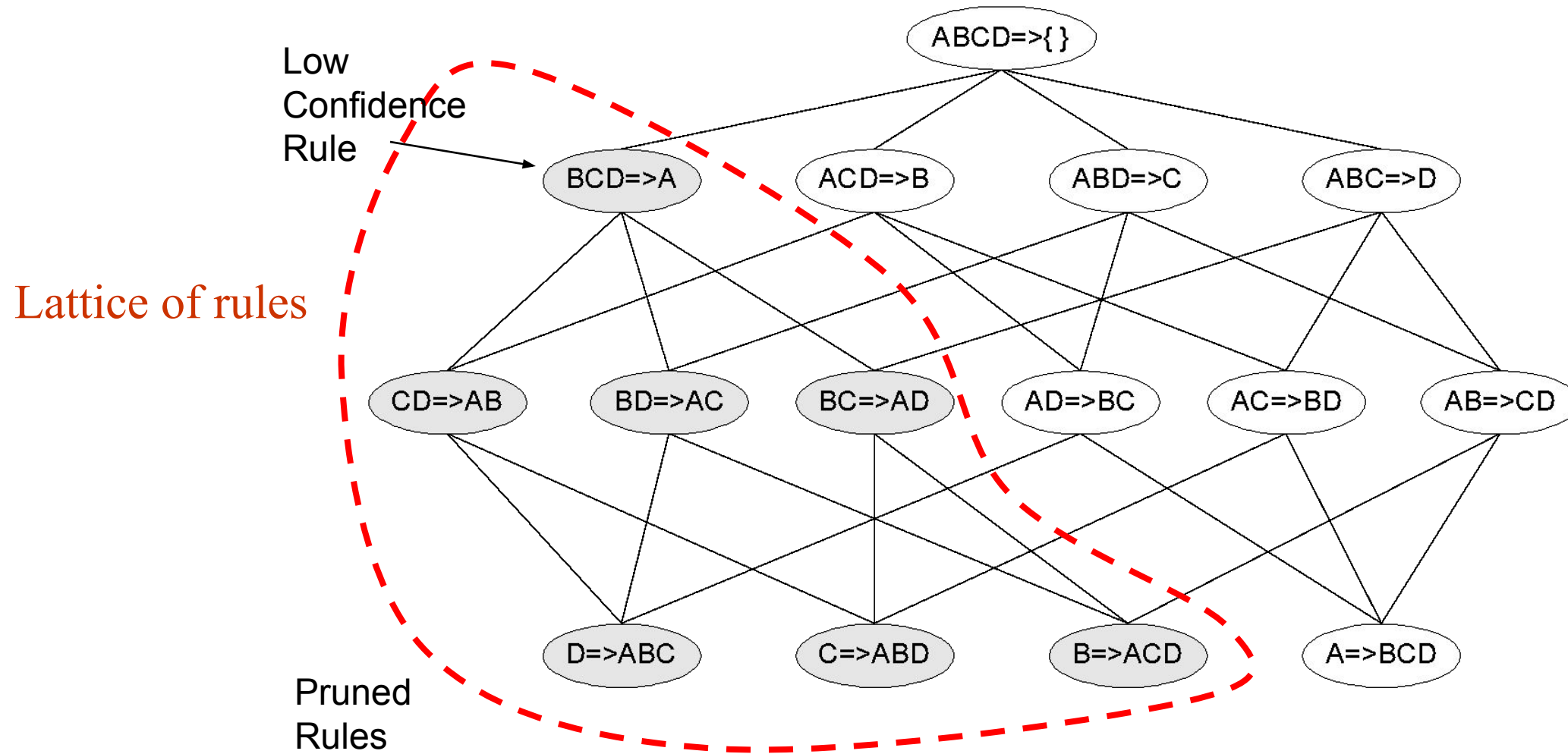
$$s(A, B, C, D) / s(A, B, C) \geq s(A, B, C, D) / s(A, B) \geq s(A, B, C, D) / s(A)$$

$$s(A, B, C) \leq s(A, B) \leq s(A)$$

- Confidence is anti-monotone w.r.t. number of items on the RHS of the rule
  - If  $c(ABC \rightarrow D) < \text{min\_conf}$  it is useless to calculate  $c(AB \rightarrow CD)$

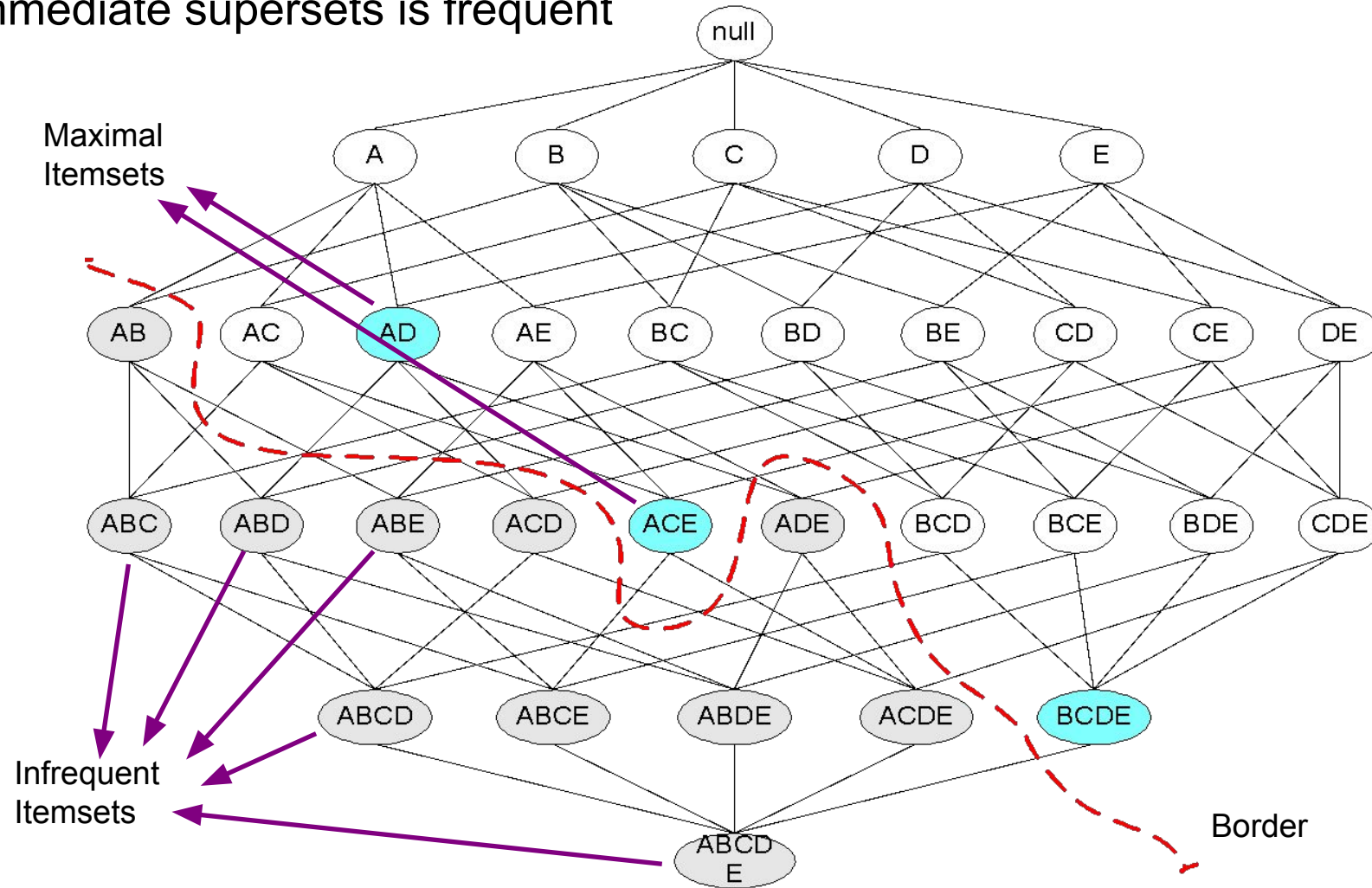


# Rule Generation for Apriori Algorithm



# Maximal Frequent Itemset

An itemset is maximal frequent if it is frequent and none of its immediate supersets is frequent



# Closed Itemset

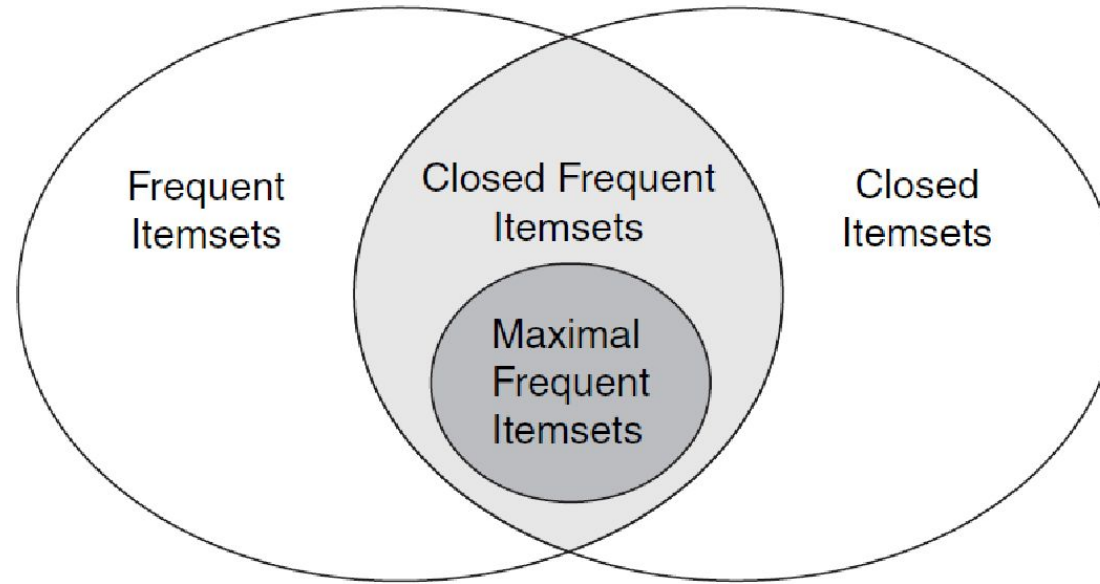
- An itemset X is closed if none of its immediate supersets has the same support as the itemset X.
- X is not closed if at least one of its immediate supersets has support count as X.

| TID | Items     |
|-----|-----------|
| 1   | {A,B}     |
| 2   | {B,C,D}   |
| 3   | {A,B,C,D} |
| 4   | {A,B,D}   |
| 5   | {A,B,C,D} |

| Itemset | Support |
|---------|---------|
| {A}     | 4       |
| {B}     | 5       |
| {C}     | 3       |
| {D}     | 4       |
| {A,B}   | 4       |
| {A,C}   | 2       |
| {A,D}   | 3       |
| {B,C}   | 3       |
| {B,D}   | 4       |
| {C,D}   | 3       |

| Itemset   | Support |
|-----------|---------|
| {A,B,C}   | 2       |
| {A,B,D}   | 3       |
| {A,C,D}   | 2       |
| {B,C,D}   | 2       |
| {A,B,C,D} | 2       |

# Maximal vs Closed Itemsets



**Figure 5.18.** Relationships among frequent, closed, closed frequent, and maximal frequent itemsets.

# Pattern Evaluation

---

- Association rule algorithms can produce large number of rules
- Interestingness measures can be used to prune/rank the patterns
- In the original formulation, support & confidence are the only measures used

# Computing Interestingness Measure

- Given  $X \rightarrow Y$  or  $\{X, Y\}$ , information needed to compute interestingness can be obtained from a contingency table

Contingency table

|           | Y        | $\bar{Y}$ |          |
|-----------|----------|-----------|----------|
| X         | $f_{11}$ | $f_{10}$  | $f_{1+}$ |
| $\bar{X}$ | $f_{01}$ | $f_{00}$  | $f_{0+}$ |
|           | $f_{+1}$ | $f_{+0}$  | N        |

$f_{11}$ : support of X and Y  
 $f_{10}$ : support of X and  $\bar{Y}$   
 $f_{01}$ : support of  $\bar{X}$  and Y  
 $f_{00}$ : support of  $\bar{X}$  and  $\bar{Y}$

Used to define various measures

- support, confidence, Gini, entropy, etc.

# Drawback of Confidence

| Custo<br>mers | Tea | Coffee | ... |
|---------------|-----|--------|-----|
| C1            | 0   | 1      | ... |
| C2            | 1   | 0      | ... |
| C3            | 1   | 1      | ... |
| C4            | 1   | 0      | ... |
| ...           |     |        |     |

|                  | <i>Coffee</i> | $\overline{Coffee}$ |      |
|------------------|---------------|---------------------|------|
| <i>Tea</i>       | 150           | 50                  | 200  |
| $\overline{Tea}$ | 650           | 150                 | 800  |
|                  | 800           | 200                 | 1000 |

Association Rule: Tea  $\rightarrow$  Coffee

Confidence  $\cong P(\text{Coffee}|\text{Tea}) = 150/200 = 0.75$

Confidence  $> 50\%$ , meaning people who drink tea are more likely to drink coffee than not drink coffee

So, this rule seems reasonable

# Drawback of Confidence

|            | Coffee | <u>Coffee</u> |      |
|------------|--------|---------------|------|
| Tea        | 150    | 50            | 200  |
| <u>Tea</u> | 650    | 150           | 800  |
|            | 800    | 200           | 1000 |

Association Rule: Tea  $\rightarrow$  Coffee

Confidence =  $P(\text{Coffee}|\text{Tea}) = 150/200 = 0.75$

but  $P(\text{Coffee}) = 0.8$ , which means knowing that a person drinks tea reduces the probability that the person drinks coffee!

$\Rightarrow$  Note that  $P(\text{Coffee} | \overline{\text{Tea}}) = 650/800 = 0.8125$



# Measure for Association Rules

---

- So, what kind of rules do we really want?
  - Confidence( $X \rightarrow Y$ ) should be sufficiently high
    - To ensure that people who buy X will more likely buy Y than not buy Y
- Confidence( $X \rightarrow Y$ ) > support(Y)
  - Otherwise, rule will be misleading because having item X actually reduces the chance of having item Y in the same transaction
  - Is there any measure that capture this constraint?
    - Answer: Yes. There are many of them.

# Statistical Relationship between X and Y

---

- The criterion  
 $\text{confidence}(X \rightarrow Y) = \text{support}(Y)$

is equivalent to:

- $P(Y|X) = P(Y)$
- $P(X,Y) = P(X) \times P(Y)$  (X and Y are independent)

If  $P(X,Y) > P(X) \times P(Y)$  : X & Y are positively correlated

If  $P(X,Y) < P(X) \times P(Y)$  : X & Y are negatively correlated

# Measures that take into account statistical dependence

$$Lift = \frac{P(Y | X)}{P(Y)}$$

$$Interest = \frac{P(X, Y)}{P(X)P(Y)}$$

lift is used for rules while  
interest is used both for rules  
and itemsets

$$PS = P(X, Y) - P(X)P(Y)$$

$$\phi - coefficient = \frac{P(X, Y) - P(X)P(Y)}{\sqrt{P(X)[1 - P(X)]P(Y)[1 - P(Y)]}}$$

# Example: Lift/Interest

|            | Coffee | <u>Coffee</u> |      |
|------------|--------|---------------|------|
| Tea        | 150    | 50            | 200  |
| <u>Tea</u> | 650    | 150           | 800  |
|            | 800    | 200           | 1000 |

Association Rule: Tea  $\rightarrow$  Coffee

Confidence =  $P(\text{Coffee}|\text{Tea}) = 0.75$

but  $P(\text{Coffee}) = 0.8$

$\Rightarrow$  Interest =  $0.15 / (0.2 \times 0.8) = 0.9375$  ( $< 1$ , therefore is negatively associated)

# Continuous and Categorical Attributes

How to apply association analysis to non-symmetric binary variables?

| Gender | ... | Age | Annual Income | No of hours spent online per week | No of email accounts | Privacy Concern |
|--------|-----|-----|---------------|-----------------------------------|----------------------|-----------------|
| Female | ... | 26  | 90K           | 20                                | 4                    | Yes             |
| Male   | ... | 51  | 135K          | 10                                | 2                    | No              |
| Male   | ... | 29  | 80K           | 10                                | 3                    | Yes             |
| Female | ... | 45  | 120K          | 15                                | 3                    | Yes             |
| Female | ... | 31  | 95K           | 20                                | 5                    | Yes             |
| Male   | ... | 25  | 55K           | 25                                | 5                    | Yes             |
| Male   | ... | 37  | 100K          | 10                                | 1                    | No              |
| Male   | ... | 41  | 65K           | 8                                 | 2                    | No              |
| Female | ... | 26  | 85K           | 12                                | 1                    | No              |
| ...    | ... | ... | ...           | ...                               | ...                  | ...             |

Example of Association Rule:

$\{\text{Gender}=\text{Male}, \text{Age} \in [21,30)\} \rightarrow \{\text{No of hours online} \geq 10\}$

# Handling Categorical Attributes

- Example: Internet Usage Data

| Gender | Level of Education | State      | Computer at Home | Online Auction | Chat Online | Online Banking | Privacy Concerns |
|--------|--------------------|------------|------------------|----------------|-------------|----------------|------------------|
| Female | Graduate           | Illinois   | Yes              | Yes            | Daily       | Yes            | Yes              |
| Male   | College            | California | No               | No             | Never       | No             | No               |
| Male   | Graduate           | Michigan   | Yes              | Yes            | Monthly     | Yes            | Yes              |
| Female | College            | Virginia   | No               | Yes            | Never       | Yes            | Yes              |
| Female | Graduate           | California | Yes              | No             | Never       | No             | Yes              |
| Male   | College            | Minnesota  | Yes              | Yes            | Weekly      | Yes            | Yes              |
| Male   | College            | Alaska     | Yes              | Yes            | Daily       | Yes            | No               |
| Male   | High School        | Oregon     | Yes              | No             | Never       | No             | No               |
| Female | Graduate           | Texas      | No               | No             | Monthly     | No             | No               |
| ...    | ...                | ...        | ...              | ...            | ...         | ...            | ...              |

{Level of Education=Graduate, Online Banking=Yes}  
→ {Privacy Concerns = Yes}

# Handling Categorical Attributes

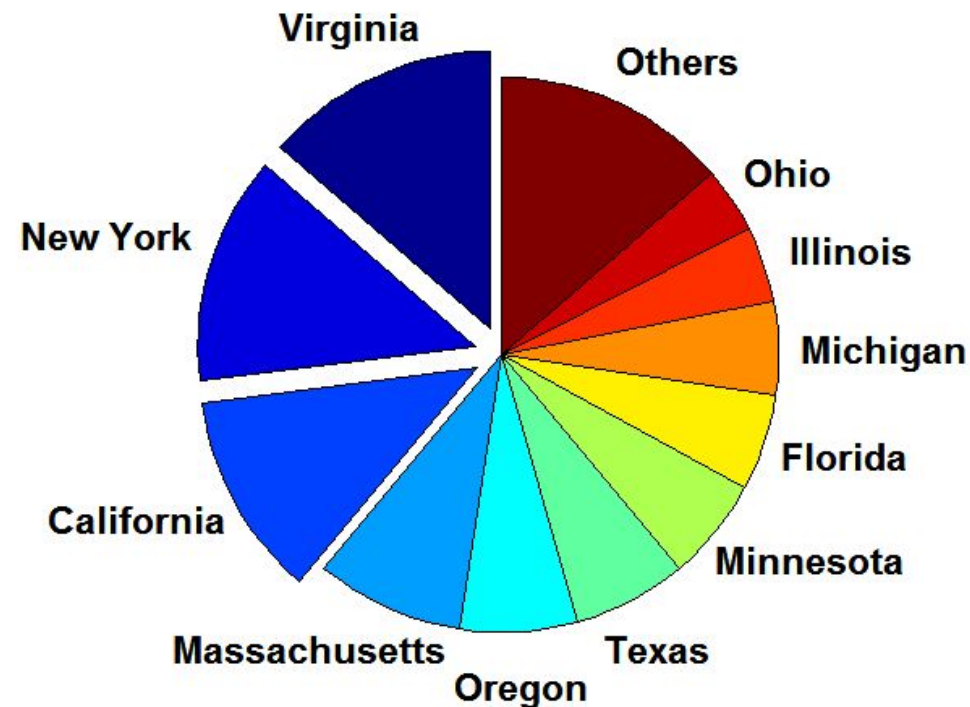
- Introduce a new “item” for each distinct attribute-value pair

| Male | Female | Education<br>= Graduate | Education<br>= College | Education<br>= High School | ... | Privacy<br>= Yes | Privacy<br>= No |
|------|--------|-------------------------|------------------------|----------------------------|-----|------------------|-----------------|
| 0    | 1      | 1                       | 0                      | 0                          | ... | 1                | 0               |
| 1    | 0      | 0                       | 1                      | 0                          | ... | 0                | 1               |
| 1    | 0      | 1                       | 0                      | 0                          | ... | 1                | 0               |
| 0    | 1      | 0                       | 1                      | 0                          | ... | 1                | 0               |
| 0    | 1      | 1                       | 0                      | 0                          | ... | 1                | 0               |
| 1    | 0      | 0                       | 1                      | 0                          | ... | 1                | 0               |
| 1    | 0      | 0                       | 0                      | 0                          | ... | 0                | 1               |
| 1    | 0      | 0                       | 0                      | 1                          | ... | 0                | 1               |
| 0    | 1      | 1                       | 0                      | 0                          | ... | 0                | 1               |
| ...  | ...    | ...                     | ...                    | ...                        | ... | ...              | ...             |

# Handling Categorical Attributes

---

- Some attributes can have many possible values
  - Many of their attribute values have very low support
    - Potential solution: Aggregate the low-support attribute values





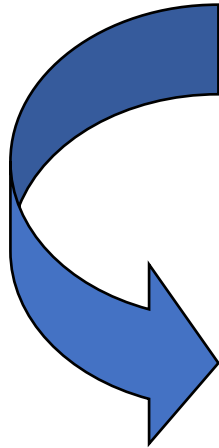
# Handling Continuous Attributes

---

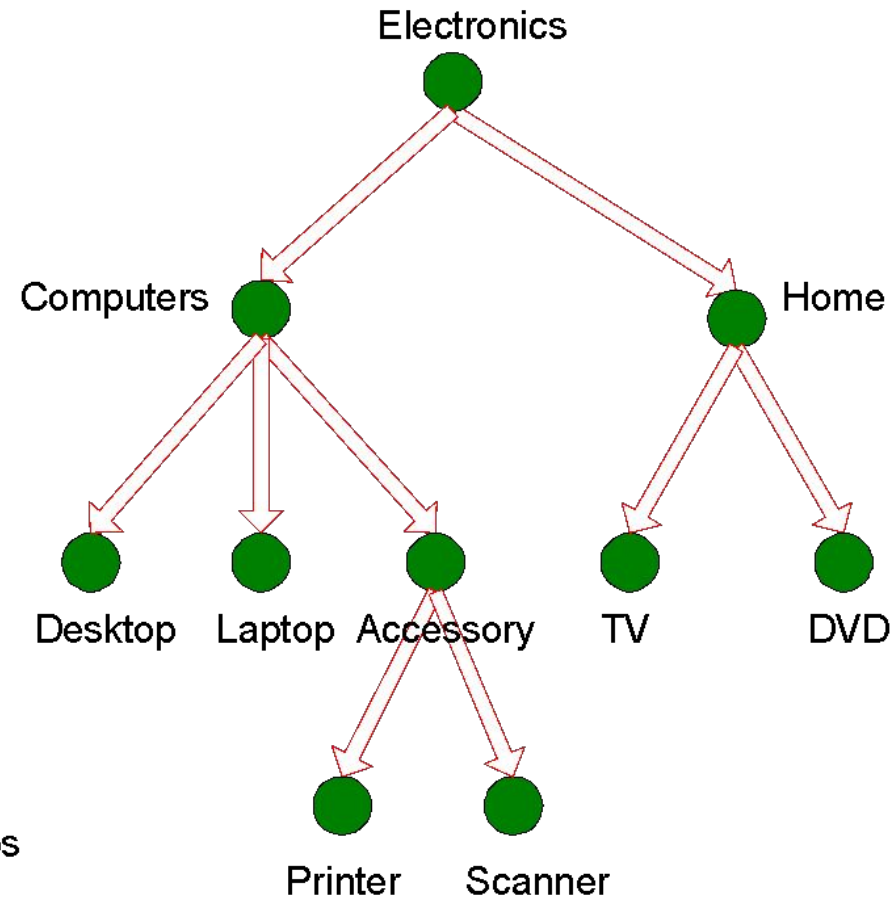
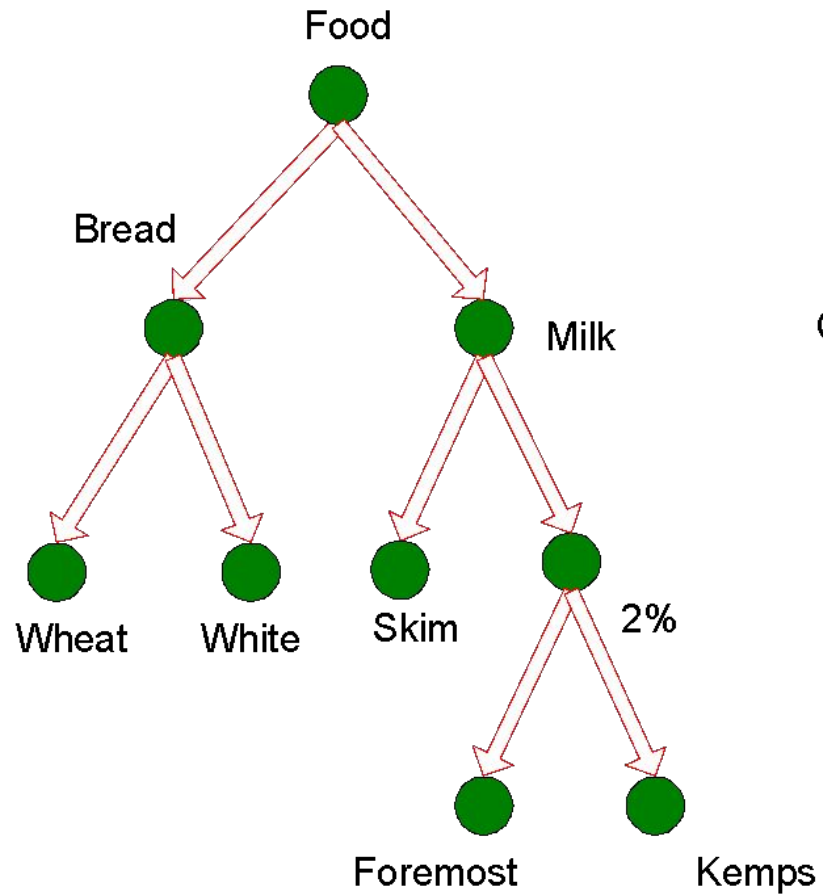
- Different methods:
  - Discretization-based
  - Statistics-based
  - Non-discretization based
    - minApriori
- Different kinds of rules can be produced:
  - $\{\text{Age} \in [21, 30), \text{No of hours online} \in [10, 20)\} \rightarrow \{\text{Chat Online} = \text{Yes}\}$
  - $\{\text{Age} \in [21, 30), \text{Chat Online} = \text{Yes}\} \rightarrow \text{No of hours online: } \mu=14, \sigma=4$

# Discretization-based Methods

| Gender | ... | Age | Annual Income | No of hours spent online per week | No of email accounts | Privacy Concern |
|--------|-----|-----|---------------|-----------------------------------|----------------------|-----------------|
| Female | ... | 26  | 90K           | 20                                | 4                    | Yes             |
| Male   | ... | 51  | 135K          | 10                                | 2                    | No              |
| Male   | ... | 29  | 80K           | 10                                | 3                    | Yes             |
| Female | ... | 45  | 120K          | 15                                | 3                    | Yes             |
| Female | ... | 31  | 95K           | 20                                | 5                    | Yes             |
| Male   | ... | 25  | 55K           | 25                                | 5                    | Yes             |
| Male   | ... | 37  | 100K          | 10                                | 1                    | No              |
| Male   | ... | 41  | 65K           | 8                                 | 2                    | No              |
| Female | ... | 26  | 85K           | 12                                | 1                    | No              |
| ...    | ... | ... | ...           | ...                               | ...                  | ...             |

[illegible]

# Concept Hierarchies



# Multi-level Association Rules

---

- Why should we incorporate concept hierarchy?
  - Rules at lower levels may not have enough support to appear in any frequent itemsets
  - Rules at lower levels of the hierarchy are overly specific
    - e.g., skim milk → white bread, 2% milk → wheat bread, skim milk → wheat bread, etc.are indicative of association between milk and bread
- Rules at higher level of hierarchy may be too generic

# Multi-level Association Rules

---

- Approach 1: Extend current association rule formulation by augmenting each transaction with higher level items

Original Transaction: {skim milk, wheat bread}

Augmented Transaction: {skim milk, wheat bread, milk, bread, food}

- Issues:
  - Items that reside at higher levels have much higher support counts
  - if support threshold is low, too many frequent patterns involving items from the higher levels
  - Increased dimensionality of the data

# Multi-level Association Rules

---

- Approach 2:
  - Generate frequent patterns at highest level first
  - Then, generate frequent patterns at the next highest level, and so on
- Issues:
  - I/O requirements will increase dramatically because we need to perform more passes over the data
  - May miss some potentially interesting cross-level association patterns

# Is Apriori Fast Enough?

---

- The core of the Apriori algorithm:
  - Use frequent  $(k - 1)$ -itemsets to generate candidate frequent  $k$ -itemsets
  - Use database scan and pattern matching to collect counts for the candidate itemsets
- The bottleneck of Apriori: candidate generation
  - Huge candidate sets:
    - $10^4$  frequent 1-itemset will generate  $10^7$  candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g.,  $\{a_1, a_2, \dots, a_{100}\}$ , one needs to generate  $2^{100} \approx 10^{30}$  candidates.
  - Multiple scans of database:
    - Needs  $(n + 1)$  scans,  $n$  is the length of the longest pattern

# FP-Growth

---



# Mining Frequent Patterns Without Candidate Generation

---

- Compress a large database into a compact, Frequent-Pattern tree (FP-tree) structure
  - highly condensed, but complete for frequent pattern mining
  - avoid costly database scans
- Develop an efficient, FP-tree-based frequent pattern mining method
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only!

# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>               |
|------------|-----------------------------------|
| 100        | { <i>f, a, c, d, g, i, m, p</i> } |
| 200        | { <i>a, b, c, f, l, m, o</i> }    |
| 300        | { <i>b, f, h, j, o</i> }          |
| 400        | { <i>b, c, k, s, p</i> }          |
| 500        | { <i>a, f, c, e, l, p, m, n</i> } |

*min\_support* = 3

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree

**Header Table**

| <u><i>Item frequency head</i></u> |   |
|-----------------------------------|---|
| <i>f</i>                          | 4 |
| <i>c</i>                          | 4 |
| <i>a</i>                          | 3 |
| <i>b</i>                          | 3 |
| <i>m</i>                          | 3 |
| <i>p</i>                          | 3 |

# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> | <i>min_support = 3</i> |
|------------|--------------------------|---------------------------------|------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |                        |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |                        |
| 300        | {b, f, h, j, o}          | {f, b}                          |                        |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |                        |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |                        |

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree

**Header Table**

| <i>Item</i> | <i>frequency</i> | <i>head</i> |
|-------------|------------------|-------------|
| <i>f</i>    | 4                |             |
| <i>c</i>    | 4                |             |
| <i>a</i>    | 3                |             |
| <i>b</i>    | 3                |             |
| <i>m</i>    | 3                |             |
| <i>p</i>    | 3                |             |

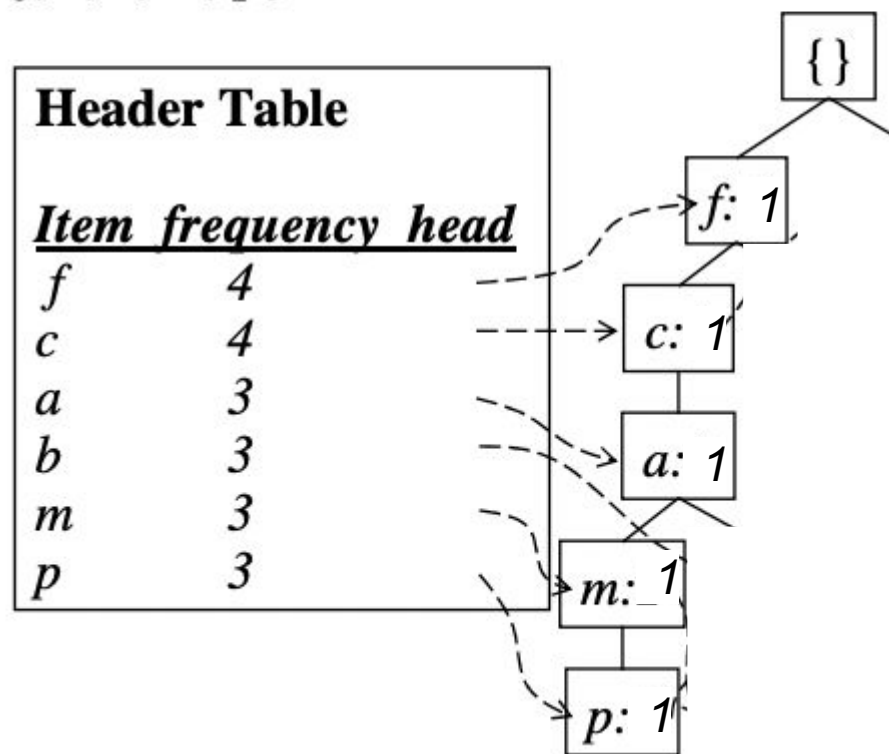
# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 3*

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree



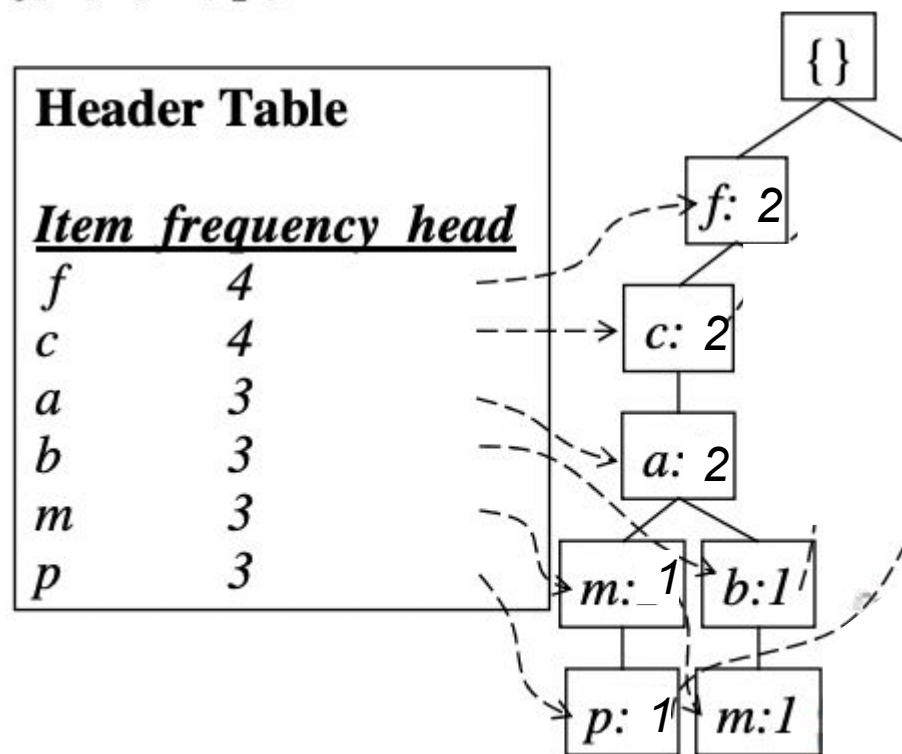
# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 3*

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree



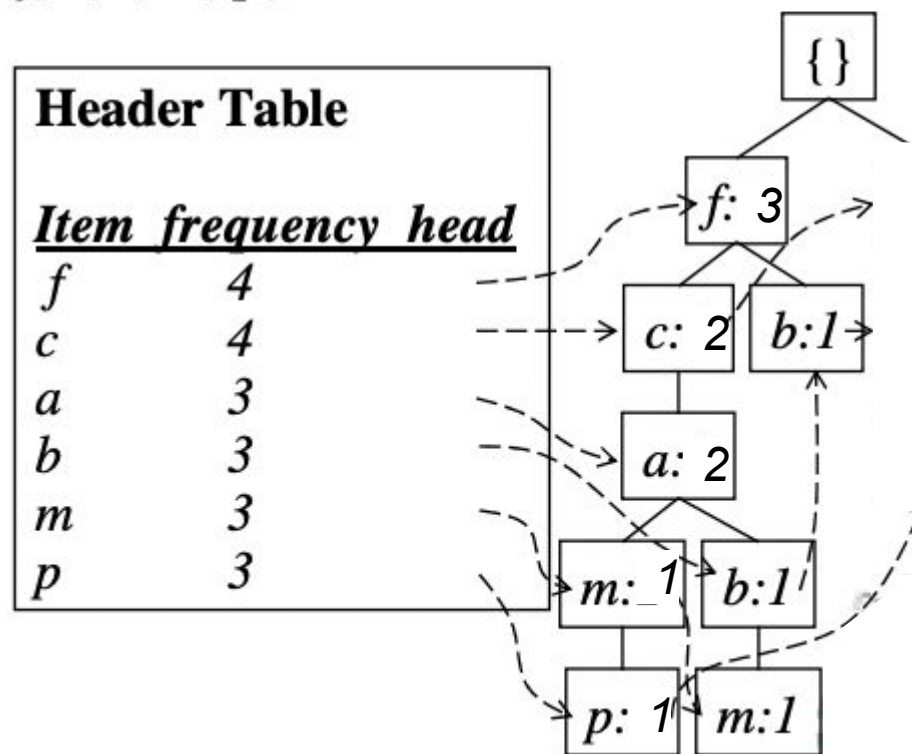
# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 3*

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree





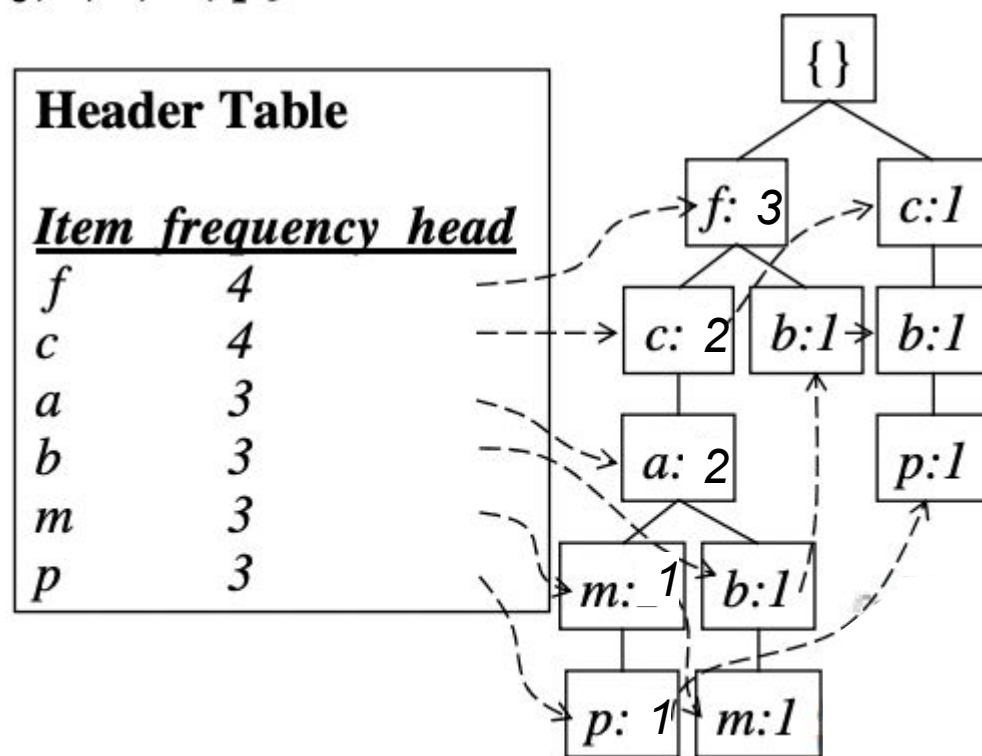
# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 3*

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree



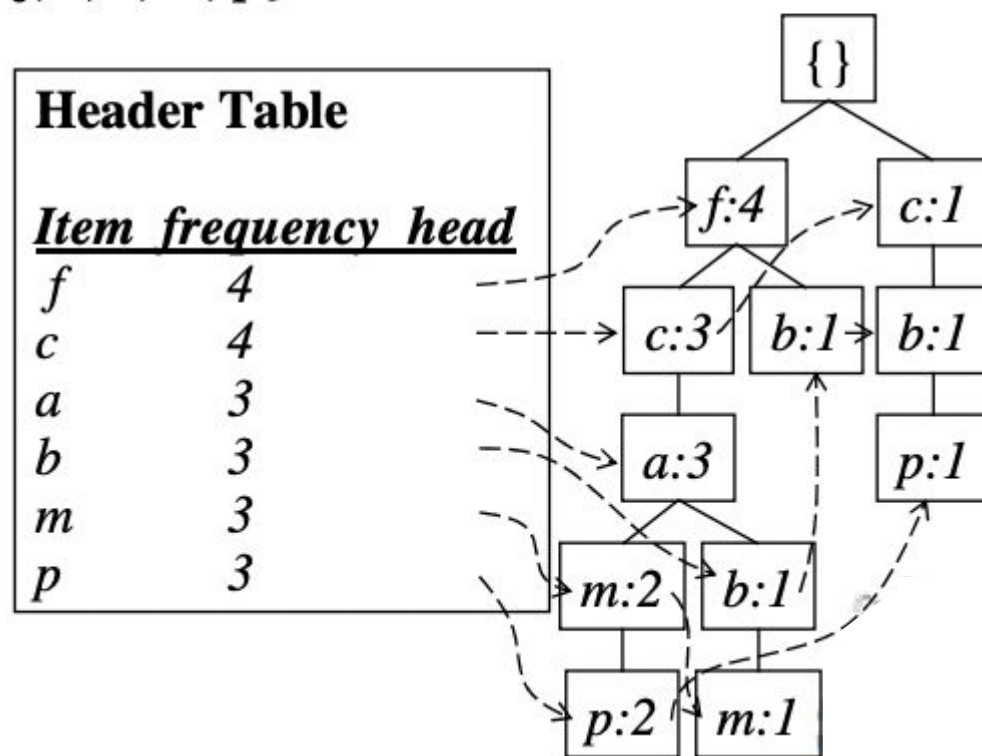
# How to construct a FP-tree

| <i>TID</i> | <i>Items bought</i>      | <i>(ordered) frequent items</i> |
|------------|--------------------------|---------------------------------|
| 100        | {f, a, c, d, g, i, m, p} | {f, c, a, m, p}                 |
| 200        | {a, b, c, f, l, m, o}    | {f, c, a, b, m}                 |
| 300        | {b, f, h, j, o}          | {f, b}                          |
| 400        | {b, c, k, s, p}          | {c, b, p}                       |
| 500        | {a, f, c, e, l, p, m, n} | {f, c, a, m, p}                 |

*min\_support = 3*

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree





# Benefits of the FP-tree Structure

---

- Completeness:
  - never breaks a long pattern of any transaction
  - preserves complete information for frequent pattern mining
- Compactness
  - reduce irrelevant information—infrequent items are gone
  - frequency descending ordering: more frequent items are more likely to be shared
  - never be larger than the original database (if not count node-links and counts)

# Mining Frequent Patterns Using FP-tree

---

- General idea (divide-and-conquer)
  - Recursively grow frequent pattern path using the FP-tree
- Method
  - For each item, construct its *conditional pattern-base*, and then its *conditional FP-tree*
  - Repeat the process on each newly created conditional FP-tree
  - Until the resulting FP-tree is *empty*, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

# Major Steps to Mine FP-tree

---

1. Construct conditional pattern base for each item in the FP-tree
2. Construct conditional FP-tree from each conditional pattern-base
3. Recursively mine conditional FP-trees and grow frequent patterns obtained so far
4. If the conditional FP-tree contains a single path, simply enumerate all the patterns

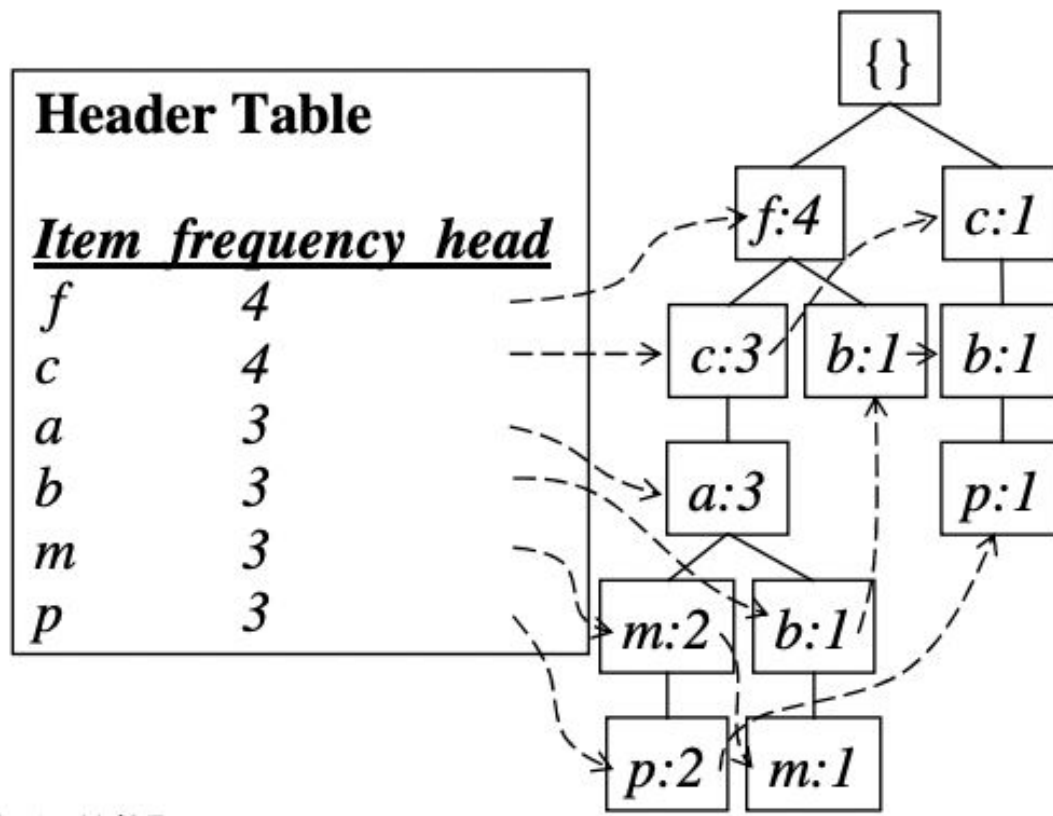
# Properties of FP-tree for Conditional Pattern Base Construction

---

- Node-link property: For any frequent item  $a_i$ , all the possible frequent patterns that contain  $a_i$  can be obtained by following  $a_i$ 's node-links, starting from  $a_i$ 's head in the FP-tree header
- Prefix path property: To calculate the frequent patterns for a node  $a_i$  in a path  $P$ , only the prefix sub-path of  $a_i$  in  $P$  need to be accumulated, and its frequency count should carry the same count as node  $a_i$ .

# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base

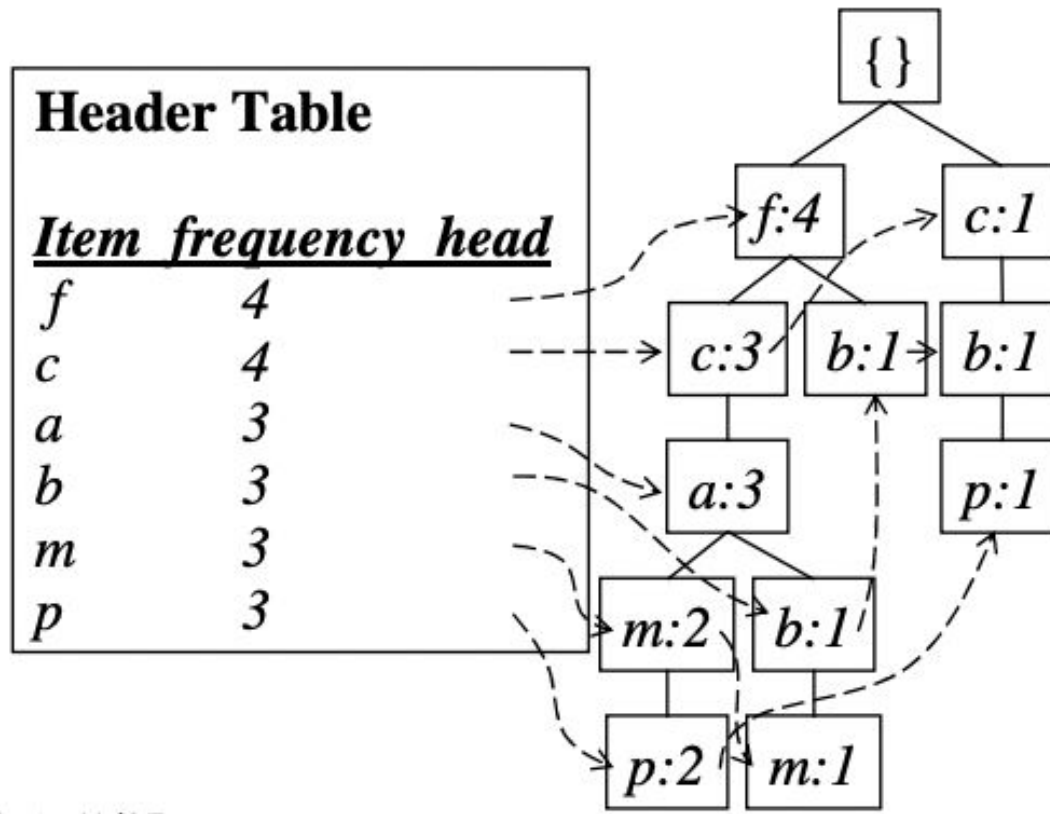


***Conditional pattern bases***

***item***      ***cond. pattern base***

# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



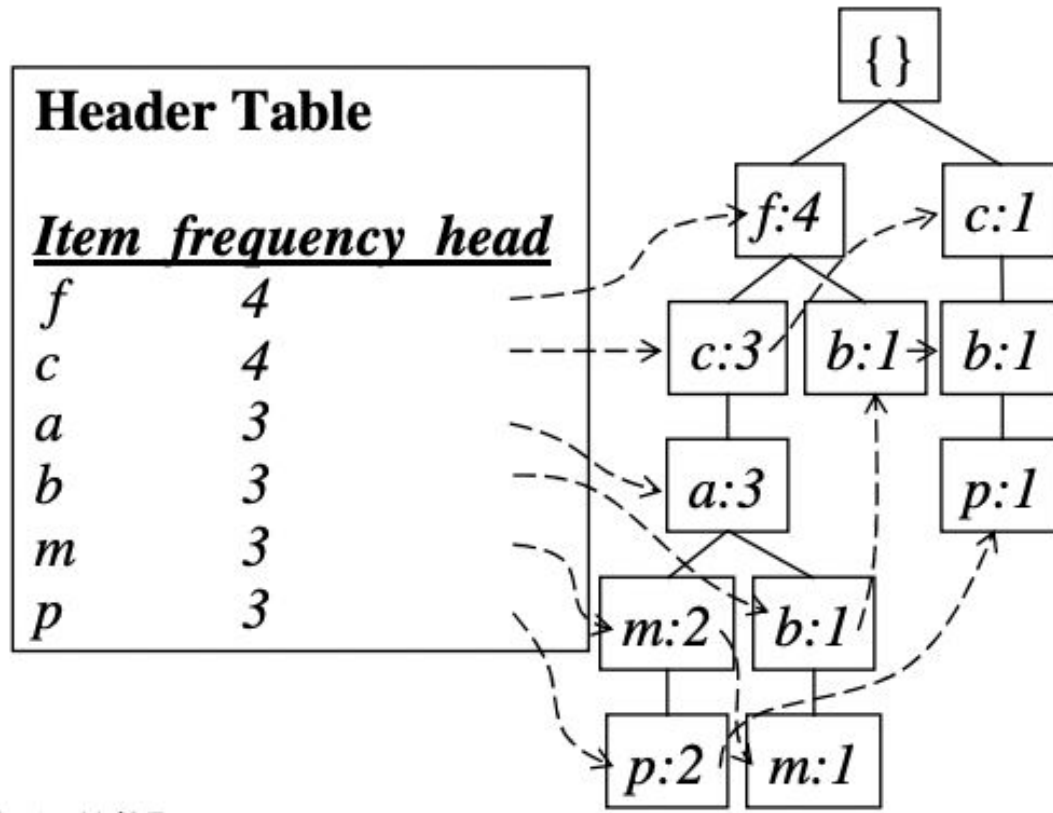
**Conditional pattern bases**

item    cond. pattern base

*p*    *fcam:2, cb:1*

# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



**Conditional pattern bases**

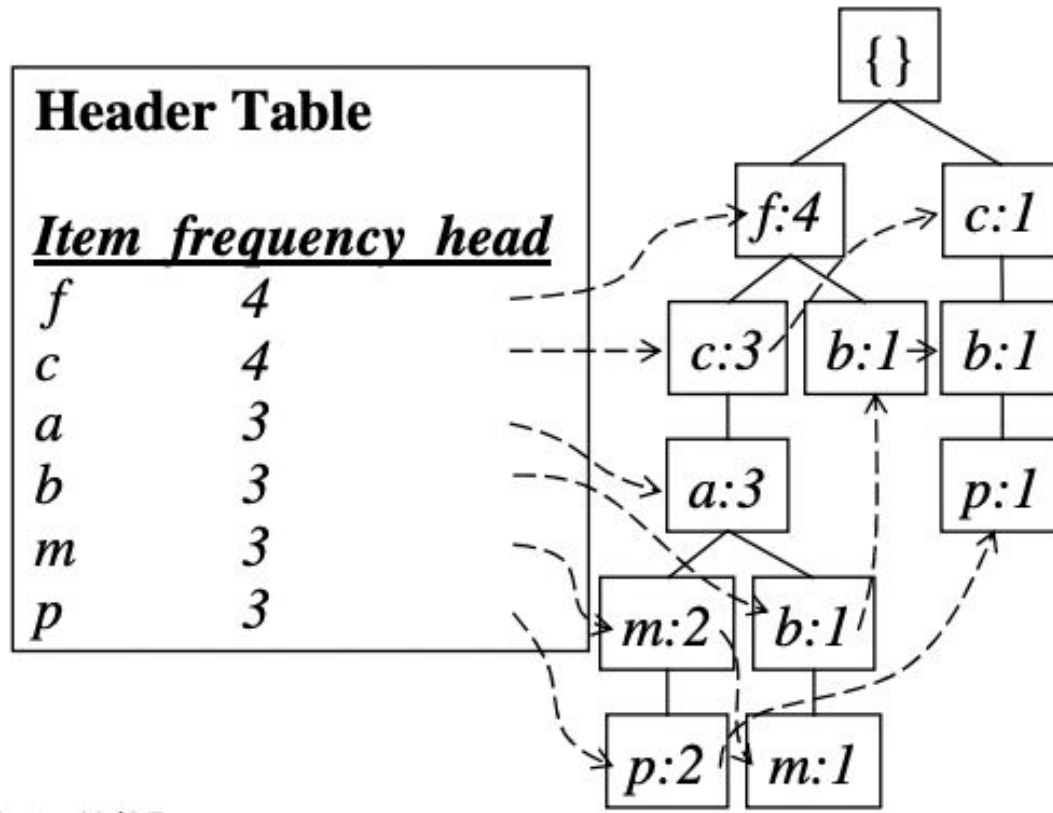
item      cond. pattern base

*m*      *fca:2, fcab:1*

*p*      *fcam:2, cb:1*

# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



**Conditional pattern bases**

item      cond. pattern base

*b*      *fca*:1, *f*:1, *c*:1

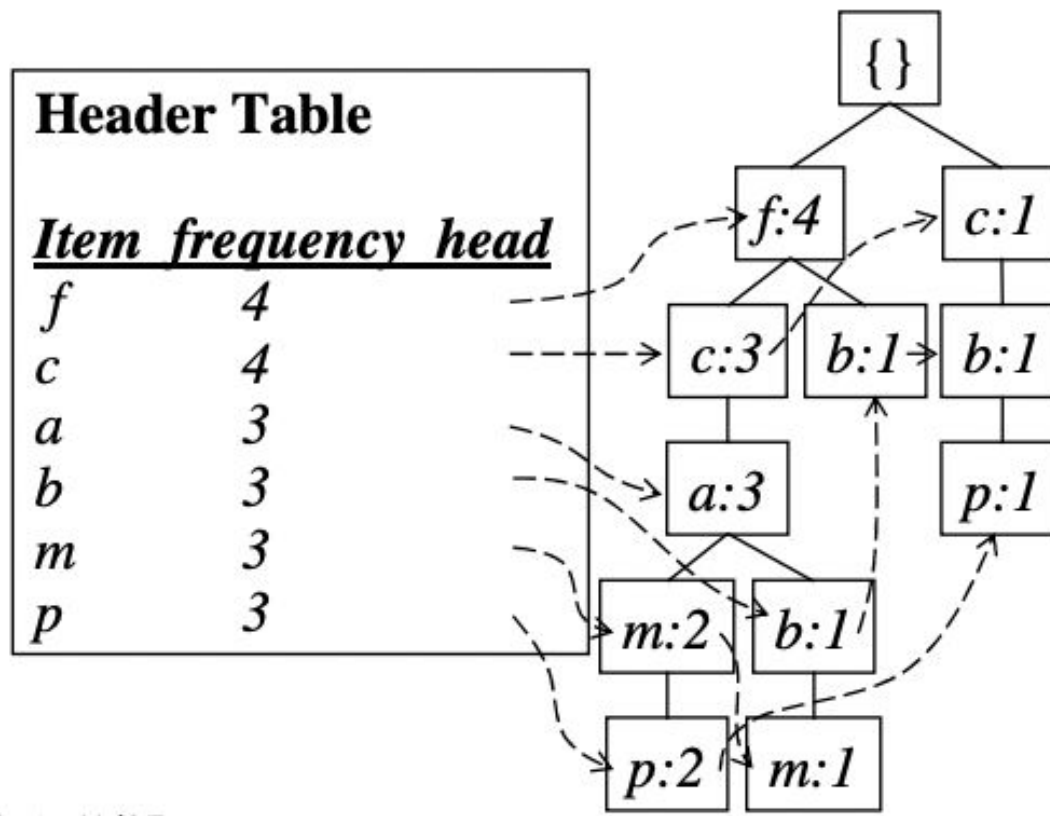
*m*      *fca*:2, *fcab*:1

*p*      *fcam*:2, *cb*:1



# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



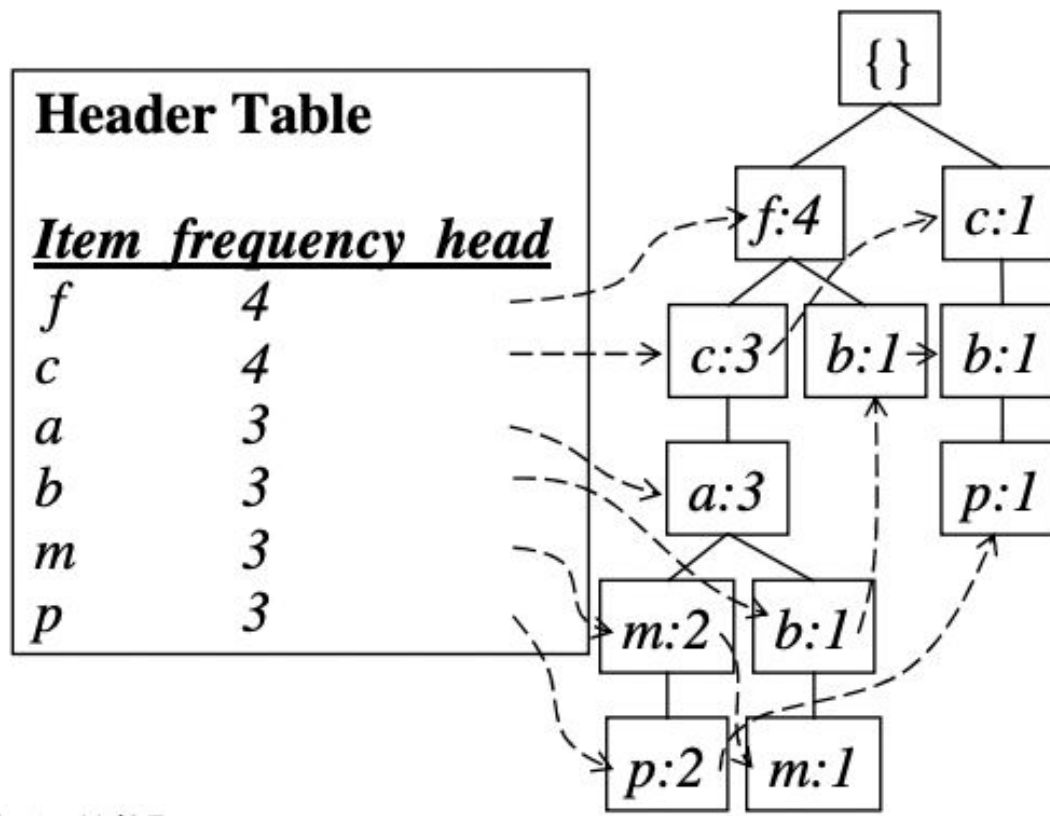
## *Conditional pattern bases*

item      cond. pattern base

*a*      *fc:3*  
*b*      *fca:1, f:1, c:1*  
*m*      *fca:2, fcab:1*  
*p*      *fcam:2, cb:1*

# Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



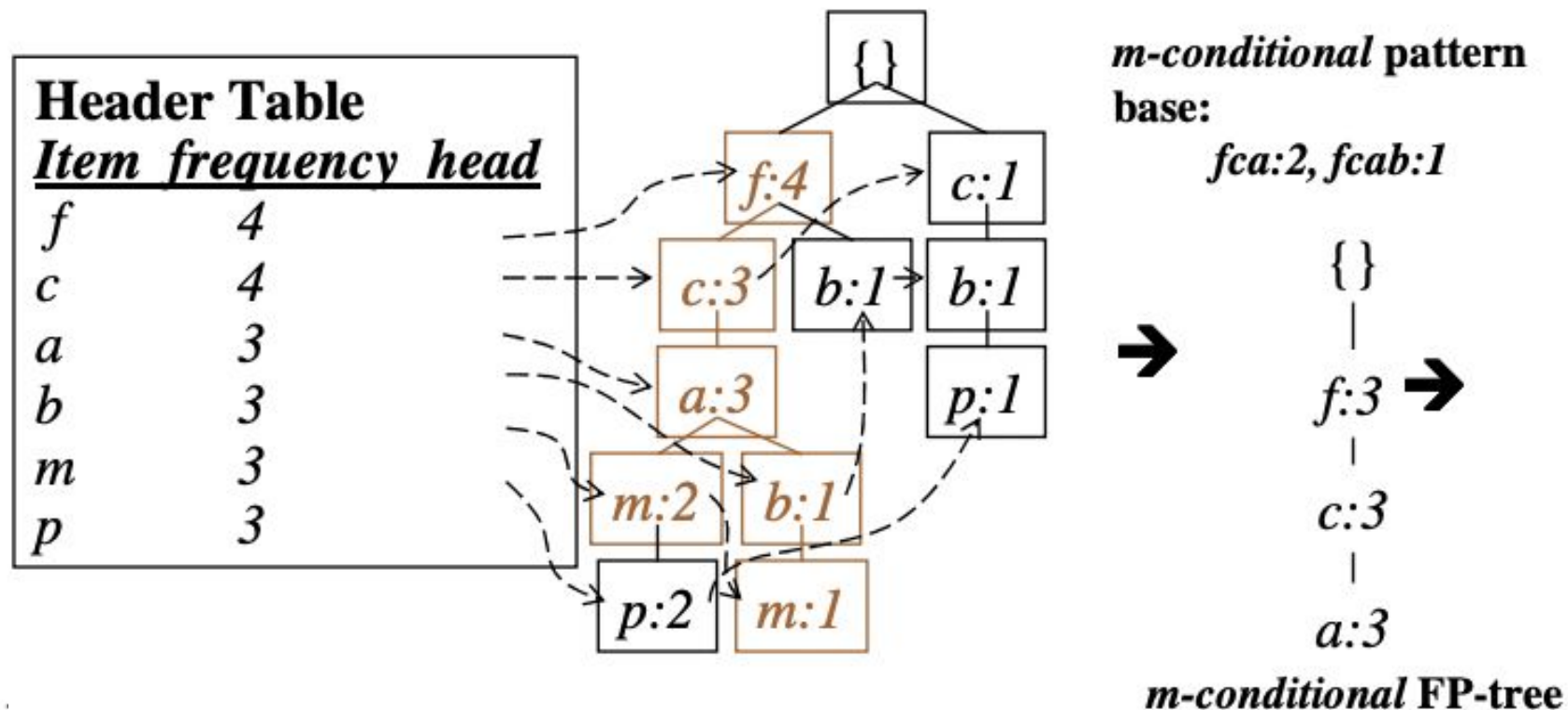
## *Conditional pattern bases*

item      cond. pattern base

*c*      *f*:3  
*a*      *fc*:3  
*b*      *fca*:1, *f*:1, *c*:1  
*m*      *fca*:2, *fcab*:1  
*p*      *fcam*:2, *cb*:1

## Step 2: Construct Conditional FP-tree

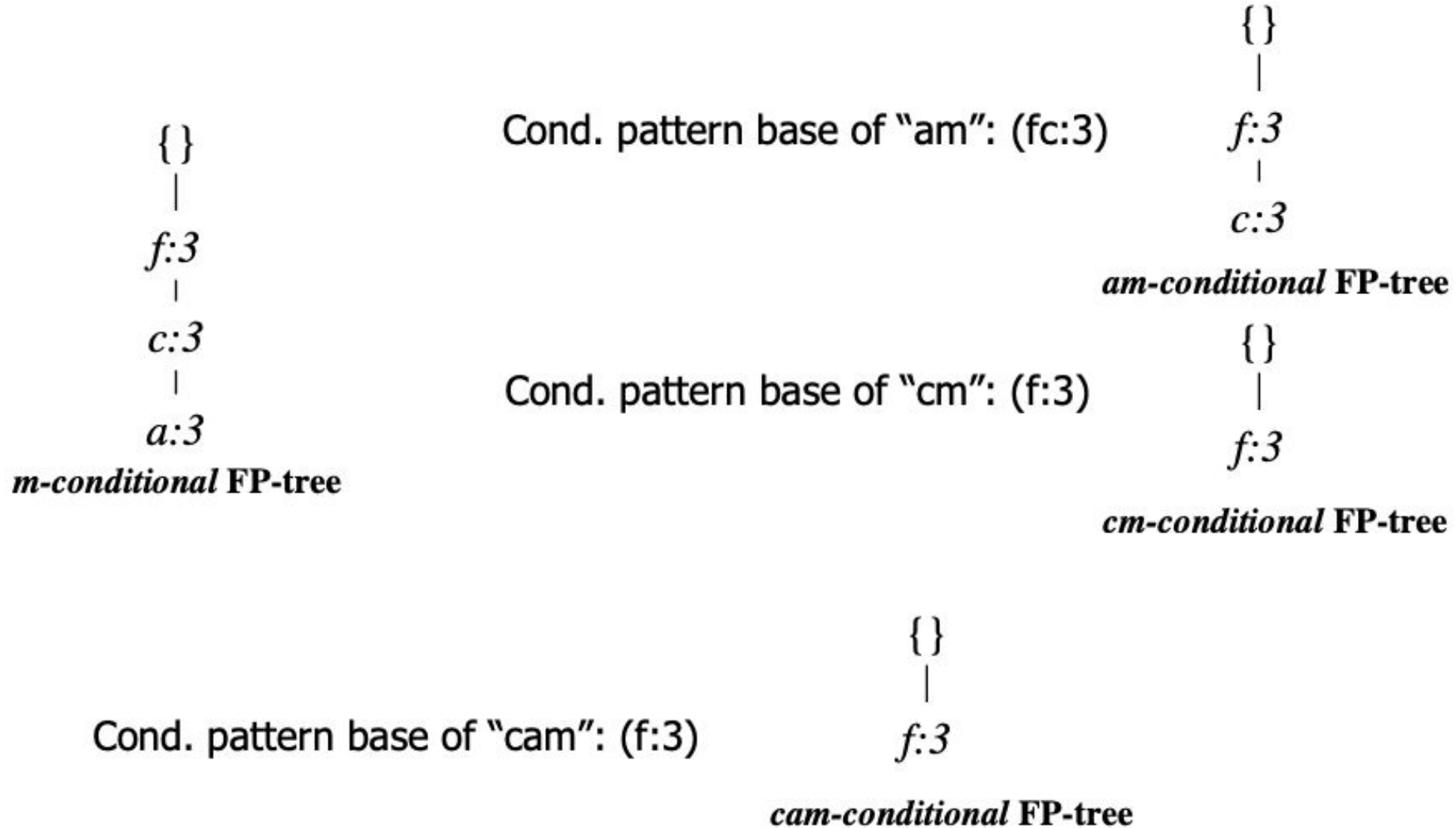
- For each pattern-base
  - Accumulate the count for each item in the base
  - Construct the FP-tree for the frequent items of the pattern base



# Mining Frequent Patterns by Creating Conditional Pattern Bases

| Item | Conditional pattern-base | Conditional FP-tree |
|------|--------------------------|---------------------|
| p    | {(fcam:2), (cb:1)}       | {(c:3)} p           |
| m    | {(fca:2), (fcab:1)}      | {(f:3, c:3, a:3)} m |
| b    | {(fca:1), (f:1), (c:1)}  | Empty               |
| a    | {(fc:3)}                 | {(f:3, c:3)} a      |
| c    | {(f:3)}                  | {(f:3)} c           |
| f    | Empty                    | Empty               |

# Step 3: recursively mine the conditional FP-tree



# Single FP-tree Path Generation

- Suppose an FP-tree  $T$  has a single path  $P$
- The complete set of frequent pattern of  $T$  can be generated by enumeration of all the combinations of the sub-paths of  $P$

$\{\}$   
|  
 $f:3$   
|  
 $c:3$   
|  
 $a:3$



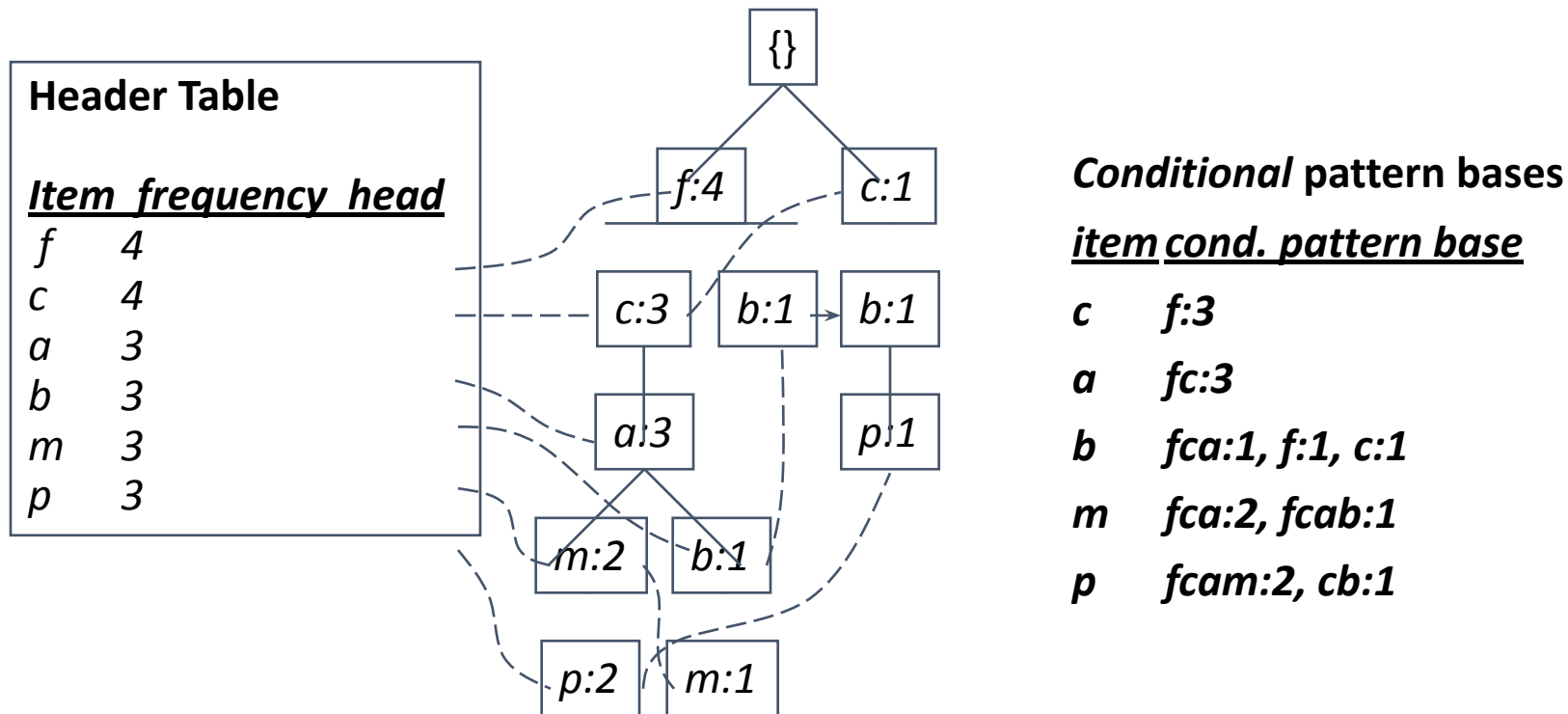
**All frequent patterns  
concerning  $m$**

$m$ ,  
 $fm$ ,  $cm$ ,  $am$ ,  
 $fcm$ ,  $fam$ ,  $cam$ ,  
 $fcam$

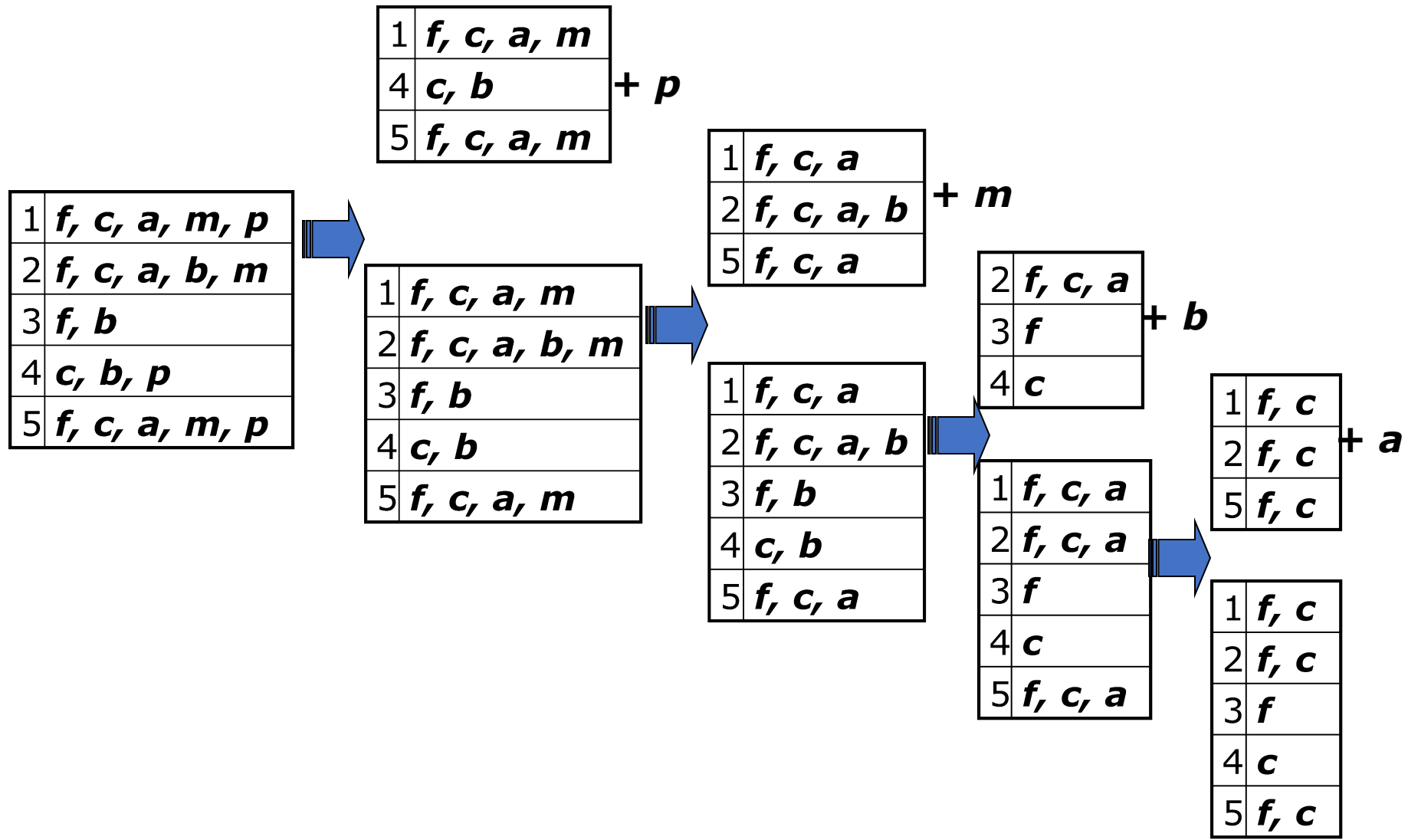
***m-conditional FP-tree***

# Find Patterns Having p From P-conditional Database

- Starting at the frequent item header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item  $p$
- Accumulate all of *transformed prefix paths* of item  $p$  to form  $p$ 's conditional pattern base

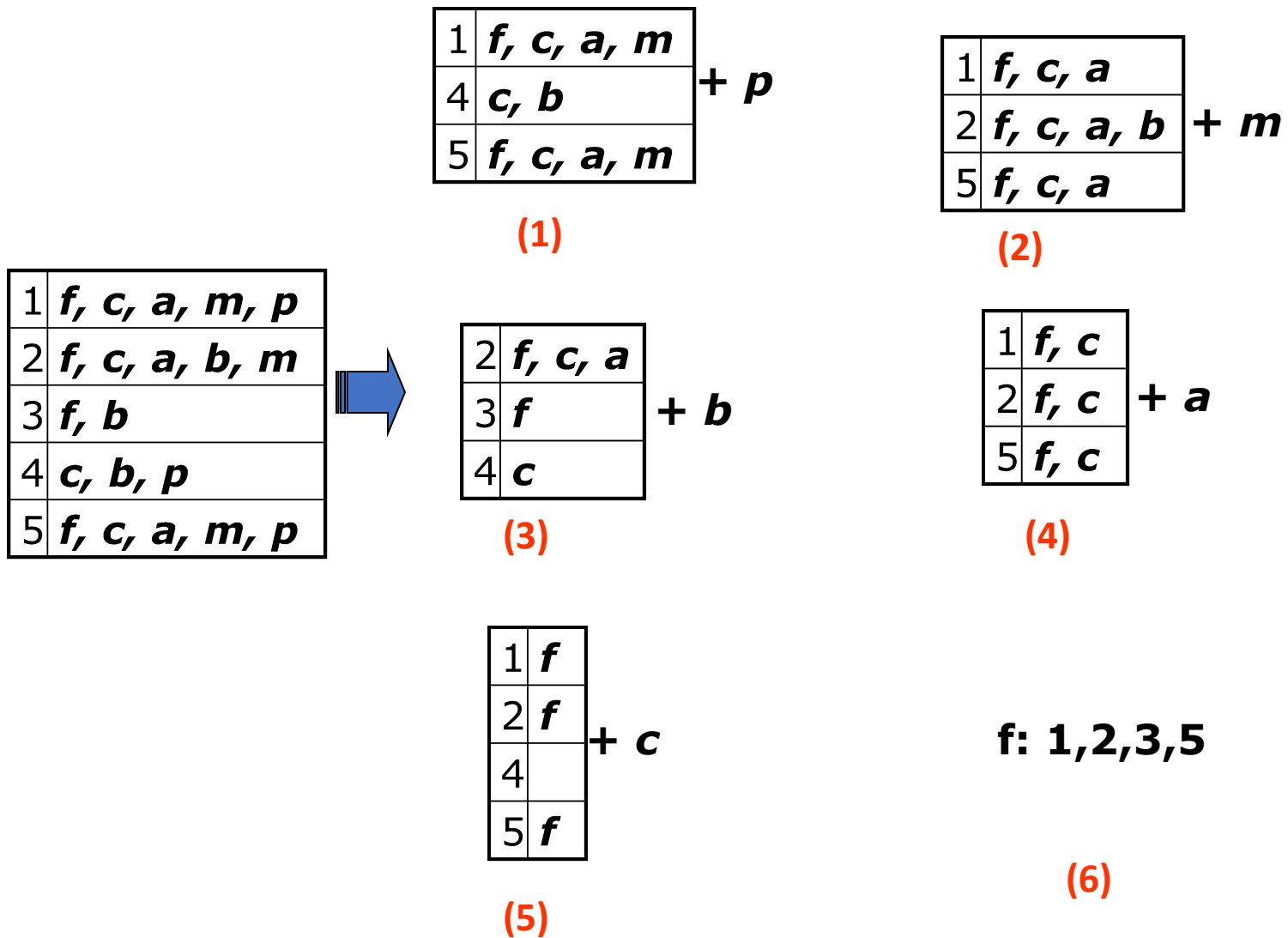


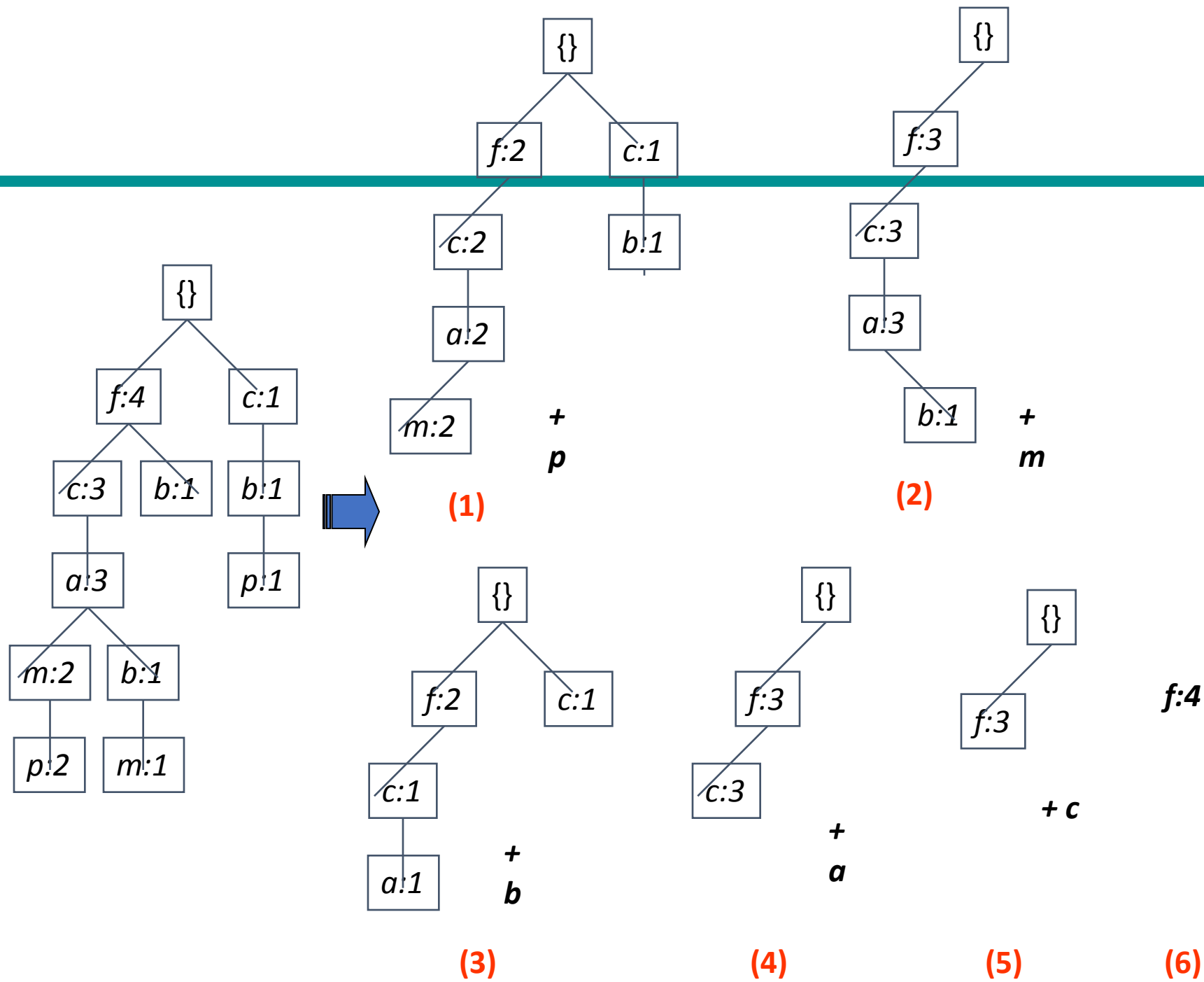
# FP-Growth

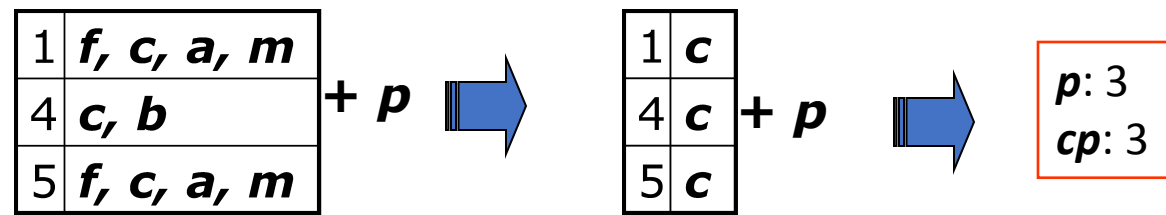




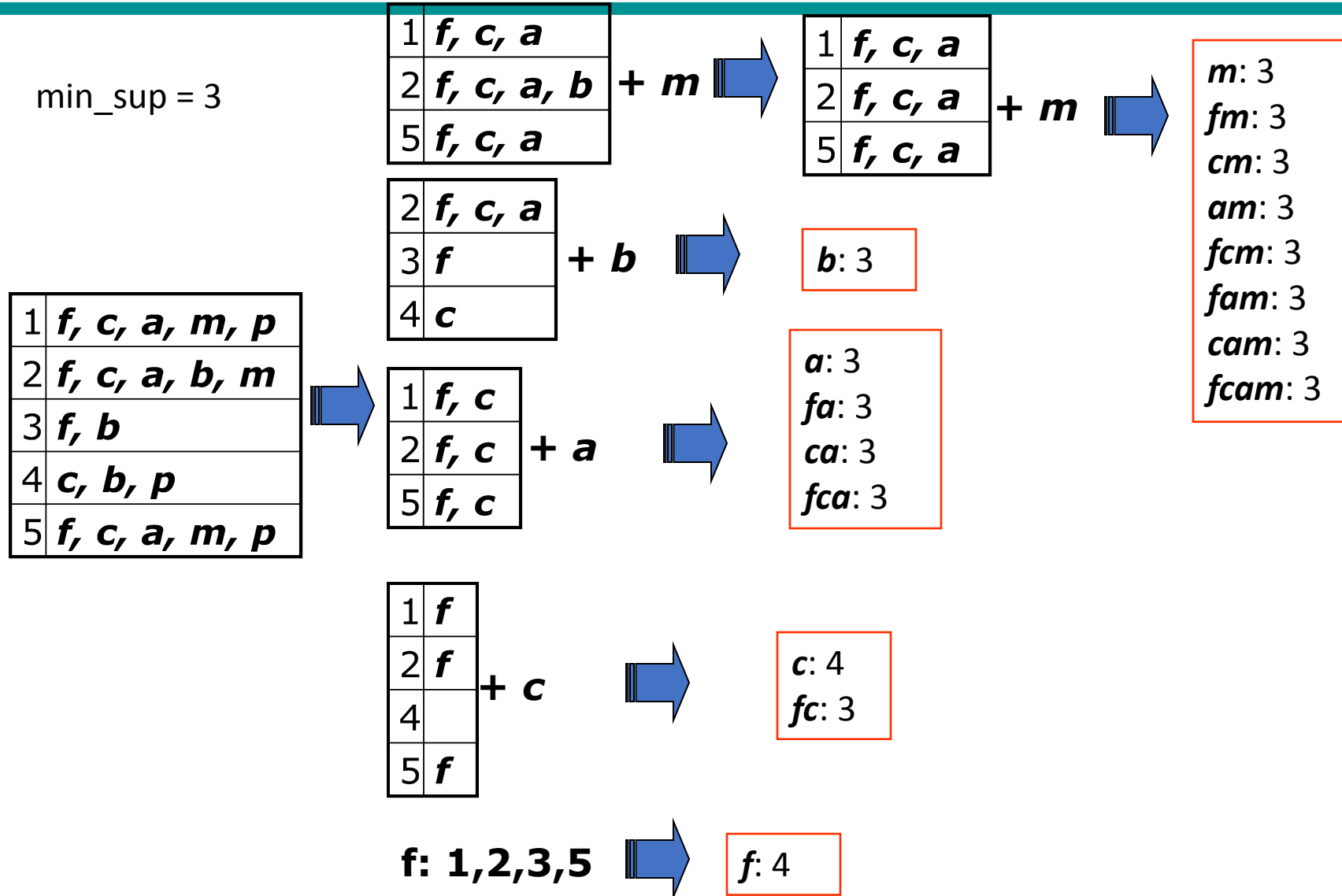
# FP-Growth





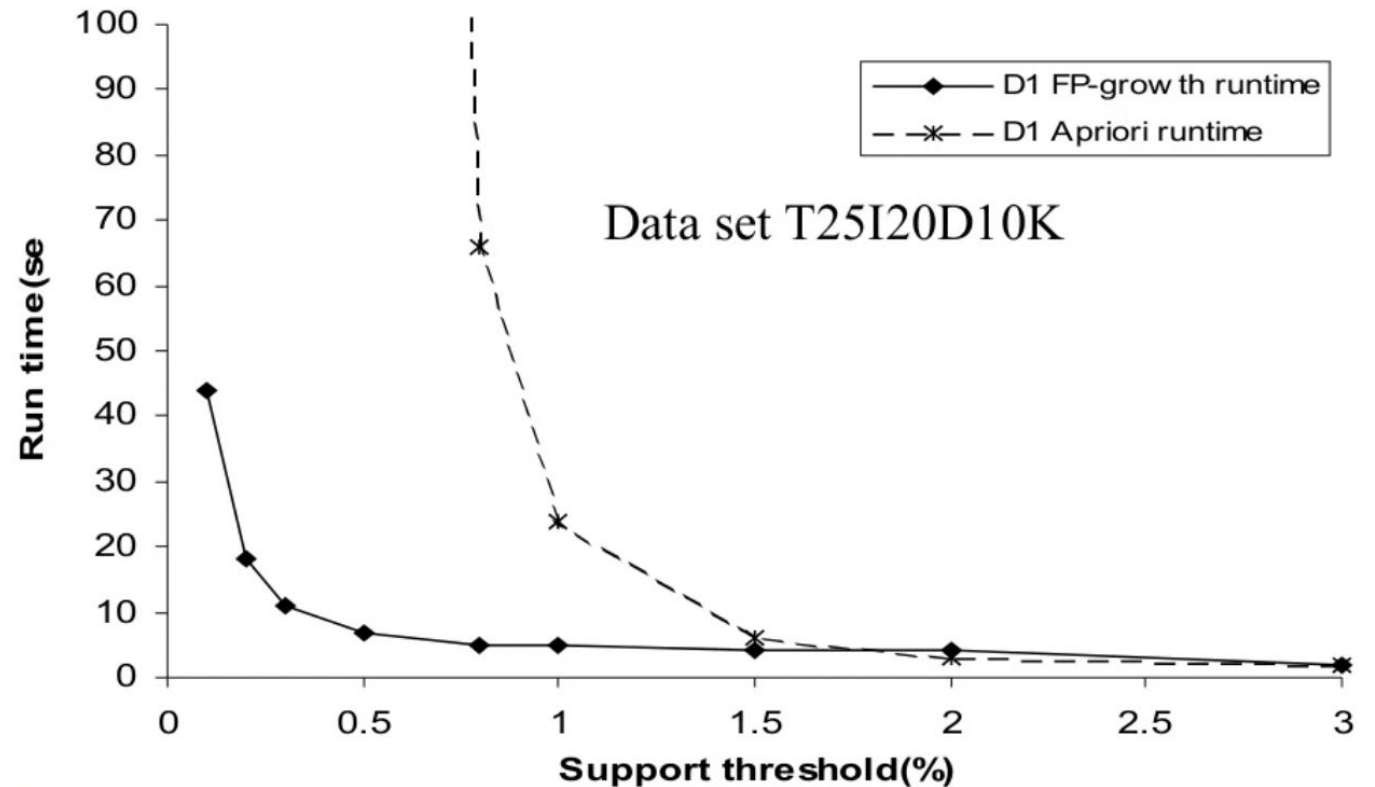


min\_sup = 3



# Why is FP-Growth Fast?

- FP-Growth is an order of magnitude faster than Apriori
  - No candidate generation, no candidate test
  - Use compact data structure
  - Eliminate repeated dataset scan
  - Basic operation is counting and FP-tree building



| #Transactions | Items | Average Transaction Length |
|---------------|-------|----------------------------|
| 250,000       | 1000  | 12                         |

# References

---

- Pattern Mining. Chapter 5. Introduction to Data Mining.

